

Odoo Full-Stack Developer Program Documentation

Detailed explanations, plans, examples, and labs

Scope	Linux · Python · PostgreSQL · Odoo · OWL · Infra
Depth	Lesson-by-lesson teaching documentation
Target Length	~150+ pages of full-page content
Versions	Odoo 17 core · Odoo 18/19 deltas
Edition	July 2026
Companion	Per-section presentation decks

Table of Contents

Part A — Program Framework

Program framework, path, assessment, capstones

Part B — Linux Operating System & CLI Mastery

L01 OS Fundamentals and Kernel Basics
L02 Terminal Navigation with pwd, cd, ls, and tree
L03 File Manipulation: Create, Copy, Move, Link, and Remove
L04 Permissions with chmod, chown, and chgrp
L05 Users, Groups, and sudo
L06 Processes with top, htop, ps, and kill
L07 systemd Services
L08 journalctl and Log Files
L09 Editing Files with nano and vim
L10 Networking with ip, ping, and ss
L11 SSH Keys and Hardening
L12 Environment Variables and bashrc
L13 Cron Jobs
L14 APT Package Management
L15 Disk and Memory Usage with df, du, and free
L16 UFW Firewall
L17 Production Directory Layout for Odoo
L18 Troubleshooting Checklist for Odoo Servers

Part C — Python Programming for Odoo Developers

P01 What Is a Programming Language and Interpreter
P02 Running Python, First Program, and Python 2 vs Python 3
P03 Data Types Overview
P04 Numbers, Math, and Operator Precedence
P05 Variables, Expressions, and Statements
P06 Strings: Concatenation, Formatting, Indexes, and Immutability
P07 Booleans and Type Conversion
P08 Lists: Slicing, Matrix Data, Methods, and Unpacking
P09 Dictionaries: Keys, Values, and Methods
P10 Tuples
P11 Sets
P12 Conditionals: Truthy Values, Ternary Expressions, and Short-Circuiting
P13 for Loops, Iterables, range, and enumerate
P14 while Loops, break, continue, and pass
P15 Functions: Parameters, args, kwargs, return, and Docstrings
P16 Scope, global, and nonlocal
P17 OOP: Classes, __init__, and Methods
P18 classmethod and staticmethod
P19 Encapsulation, Inheritance, Polymorphism, and super
P20 Dunders, MRO, and Multiple Inheritance
P21 map, filter, zip, reduce, and lambdas
P22 Comprehensions
P23 Decorators
P24 Error Handling
P25 Generators
P26 Modules, Packages, Imports, and __name__
P27 pip, venv, and PyPI
P28 Debugging and Python Habits for Odoo Developers

Part D — PostgreSQL Database Administration

DB01 Install PostgreSQL on Ubuntu

DB02 psql CLI, Users, Roles, and Grants
DB03 Create and Manage Databases
DB04 PostgreSQL Data Types for Odoo Developers
DB05 DDL: CREATE, ALTER, and DROP
DB06 Primary Keys, Foreign Keys, and Constraints
DB07 CRUD: INSERT, SELECT, UPDATE, and DELETE
DB08 Filtering with WHERE, LIKE, ILIKE, IN, and BETWEEN
DB09 Joins for Relational Odoo Data
DB10 GROUP BY, HAVING, and Aggregates
DB11 Indexes: B-Tree, GIN, and Composite Design
DB12 Reading EXPLAIN ANALYZE
DB13 Backups with pg_dump
DB14 Restores with pg_restore
DB15 postgresql.conf and pg_hba.conf
DB16 Locks, Connections, Terminating Queries, and Odoo Backup Habits

Part E — Odoo Core Development (Models, Fields, ORM)

O01 Install Odoo 17 on Ubuntu 22.04
O02 Start odoo-bin for the First Time
O03 Odoo CLI Commands Overview
O04 The Odoo Configuration File
O05 Database Manager URLs and Disabling Database Manager
O06 Database CLI and Upgrade CLI Workflows
O07 PyCharm Configuration for Odoo
O08 Scaffold a Module and Use Multiple Addons Paths
O09 Manifest Files, Dependencies, and Auto Install Modules
O10 Custom Module Structure
O11 _name, _table, and Model Identity
O12 Field Overview and Simple Field Options
O13 Text-Oriented Fields: Char, Text, and Html
O14 Boolean, Selection, Integer, Float, and Json Fields
O15 Date and Datetime Fields
O16 Relational Fields: Many2one, One2many, and Many2many
O17 Reference and Related Fields
O18 Compute Fields
O19 Monetary, Binary, and Image Fields
O20 Default ORM Methods and Recordsets
O21 Override create and write
O22 Custom Methods, copy, and unlink
O23 exists, search, and Domain Operators
O24 search_count, read, read_group, and search_read
O25 name_create, default_get, _name_search, and get_view
O26 ensure_one, filtered, mapped, sorted, grouped, fields_get, and get_metadata
O27 search_fetch and Efficient Data Loading
O28 Special Relational Commands 0 through 6
O29 Active Archive Behavior
O30 Environment self.env and Modifying Environment

Part F — Views, Security, Data & QWeb

VS01 Intro to Odoo Views and XML Data Loading
VS02 List Views, Sorting, and Limits
VS03 Editable Lists, Mass Editing, and the Handle Widget
VS04 List Decorations, Field Decorations, Widths, and Hidden Labels
VS05 List Totals, Averages, and Grouped Lists
VS06 Search Views, Default Filters, and Dynamic Date Filters
VS07 Search Panel and Groupby UX
VS08 Form Customization with Groups, Tags, and Widgets

VS09 Notebook Pages, Button Containers, Header Buttons, and Statusbars
VS10 Attrs Migration and Dynamic Readonly, Invisible, Required Rules
VS11 Buttons that Open Forms, URL Actions, and Disabled Actions
VS12 Kanban Views, Templates, and Images
VS13 Kanban Drag and Drop, Handle Ordering, and Progressbars
VS14 Graph, Pivot, Cohort, and Gantt Views
VS15 Security Overview: Six Components, Groups, Categories, and Hierarchy
VS16 Implied Groups and Combobox Group Selection
VS17 Access Rights in CSV and XML
VS18 Record Rules: Why They Exist and How Domains Work
VS19 Record Rules Student and Teacher Example
VS20 Security Inheritance and Groups on Fields, Views, and Actions
VS21 Odoo 19 Privilege Model
VS22 Odoo 18 to 19 Migration Notes for Views and Security
VS23 XML Data, CSV Data, and Choosing Between Them
VS24 View Inheritance Overview and XPath Positions
VS25 ir.sequence for Business Numbers
VS26 QWeb Intro and Core Directives
VS27 QWeb Conditionals, Loops, and Template Inheritance
VS28 Paper Formats, Barcodes, Images, Server Actions, and Email Templates

Part G — OWL Frontend, POS & External APIs

W01 OWL Architecture and Reactive Rendering
W02 Component Structure and Lifecycle Hooks
W03 useState, useRef, Events, and Props
W04 QWeb XML Directives: t-if, t-foreach, and t-model
W05 Custom Form Field Widgets
W06 Overriding JavaScript Components with the Patch Utility
W07 RPC and ORM Services
W08 Custom Kanban and Dashboard OWL Views
W09 POS Custom Buttons and Click Events
W10 Hiding POS Buttons Safely
W11 POS Popups and Dialog Flow
W12 JavaScript Translations
W13 POS RPC and Server Synchronization
W14 Rendering Dynamic POS Data
W15 Clearing Order Lines and Mutating POS Orders
W16 XML-RPC Integration Fundamentals
W17 External API Authentication
W18 Fetch and Export Workflows with XML-RPC
W19 CRUD Operations over XML-RPC
W20 Postman Authentication and CRUD Testing

Part H — Infrastructure, Docker, VPS & Performance

I01 Containers vs Virtual Machines
I02 Installing Docker Compose for Odoo Workloads
I03 Docker CLI, Images, Containers, and Docker Hub
I04 Dockerfile for Odoo
I05 Volumes for PostgreSQL, Filestore, and Addons
I06 Docker Networking for Odoo
I07 Docker Compose Fundamentals
I08 Multi-Container Odoo and PostgreSQL
I09 .env Files and Runtime Configuration
I10 Upgrading Containers Safely
I11 CI/CD with GitHub Actions Basics
I12 VPS Provisioning and Production Directory Layout
I13 SSH Hardening

I14 Nginx Reverse Proxy for Odoo

I15 Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt

I16 systemd odoo.service

I17 Backup Cron for Database and Filestore

I18 Odoo Workers Architecture and Worker Calculation

I19 Memory Limits, Time Limits, PostgreSQL Pooling, and max_connections

I20 Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana

A Program Framework & How to Use This Document

Purpose

This document is the complete written curriculum for the Odoo Full-Stack Developer Program. Unlike a short syllabus, every major topic is explained in teaching depth: why it matters, how seniors apply it, concrete examples, common mistakes, and a lab. Use it as the source of truth for self-study, instructor delivery, and review before capstone work.

Document Structure

Parts B through H map one-to-one with the separate presentation decks. Study a documentation part, then reinforce with that section's slide deck. Each lesson follows the same pedagogical pattern so you always know what to expect.

Part	Domain	Presentation Deck
B	Linux & CLI	01_Linux_CLI_Mastery.pdf
C	Python for Odoo	02_Python_for_Odoo.pdf
D	PostgreSQL	03_PostgreSQL_Database.pdf
E	Odoo Core (ORM/Fields)	04_Odoo_Core_Development.pdf
F	Views, Security, QWeb	05_Views_Security_QWeb.pdf
G	OWL, POS, APIs	06_OWL_POS_APIs.pdf
H	Infra & Performance	07_Infrastructure_DevOps.pdf

How Each Lesson Is Organized

- Objectives — what you can do after the lesson
- Explanation — senior-level narrative of concepts and trade-offs
- Key points — compressed checklist for revision
- Worked examples — commands, Python, XML, SQL, or configs
- Common mistakes — failure modes seen in real projects
- Lab — hands-on practice that produces an artifact

Delivery principle: Do not skip Foundations (Linux/Python/PostgreSQL). Most Odoo production incidents are diagnosed with OS, language, and database skills—not only ORM knowledge.

Recommended Study Path

Week-by-week professional track (12–16 weeks): Weeks 1–4 foundations; weeks 5–7 Odoo install/modules/fields/ORM; weeks 8–9 views and security; weeks 10–11 OWL/POS/APIs/QWeb; week 12 infrastructure and capstone defense. Intensive bootcamps compress each week into 2–3 full days while keeping the same sequence.

Assessment Model

Component	Weight	Evidence
Lesson labs	40%	Working commands/code + notes
Knowledge checks	15%	Short quizzes per part
Mini-projects	20%	Integrated feature slices
Capstone	25%	Secured, documented, demoable module

A2 Capability Map & Capstone Overview

North-Star Graduate Capabilities

- Provision and harden an Ubuntu host suitable for Odoo workloads
- Write idiomatic Python suitable for models, wizards, and connectors
- Administer PostgreSQL for development and production backup/restore
- Install Odoo, author odoo.conf, and operate CLI install/upgrade/shell safely
- Design models/fields and override ORM methods without breaking invariants
- Build list/form/search/kanban UX and migrate attrs-style view logic
- Implement groups, ACLs, and record rules with least privilege
- Extend OWL components and POS screens; integrate via XML-RPC
- Deploy with Docker or systemd+Nginx+SSL and size workers sensibly

Capstone Options

A — Academic Institute Manager: students, courses, enrollments, fees, role-based security, QWeb confirmation PDF, email on confirmation.

B — Retail POS Helper: OWL POS button + popup + RPC-persisted cashier note, manager ACL menu, optional XML-RPC export.

C — Ops Excellence Pack: production layout, systemd, Nginx/longpolling/Certbot, backup/restore rehearsal, worker sizing worksheet.

Rubric focus: Capstones are graded on architecture, correctness, security, UX, operability, and documentation—not on visual decoration.

Part B Linux Operating System & CLI Mastery

This part contains 18 detailed lessons. Work in order. Complete each lab before moving forward when the lab unlocks the next dependency.

Presentation: Companion deck: see [training_materials/presentations/](#) for the Linux Operating System & CLI Mastery slides.

L01 OS Fundamentals and Kernel Basics

Learning Objectives

- Explain the relationship between hardware, kernel, system libraries, and user processes.
- Identify where the shell, services, and Odoo workers fit in the operating system model.
- Use basic inspection commands to confirm kernel and distribution details.
- Describe why kernel resources affect application performance.

Explanation

OS Fundamentals and Kernel Basics matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands the kernel boundary and the role of user space can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to kernel version, distribution release, CPU architecture, memory, and process ownership and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is debugging symptoms at the application layer while ignoring resource or kernel limits, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo workers, PostgreSQL, Nginx, and cron all compete for CPU time, memory, file descriptors, and network sockets connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Start investigations by naming the layer you are testing: hardware, kernel, service manager, process, or application. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- The kernel manages CPU scheduling, memory, devices, filesystems, and network stacks.
- User space contains shells, package managers, systemd, Python, PostgreSQL, Nginx, and Odoo.
- A distribution combines the Linux kernel with tools, defaults, repositories, and service policies.
- Applications fail when kernel limits are exhausted even if their own code is correct.
- Version and architecture details matter when installing drivers, Python wheels, and database packages.
- Good incident notes separate facts observed from assumptions about root cause.

Inspect the operating system baseline

These commands establish the kernel, distribution, CPU, and memory baseline before troubleshooting. Capture this information in support tickets because package names, service defaults, and binary compatibility vary by distribution and version.

```
uname -a
cat /etc/os-release
lscpu | sed -n '1,8p'
free -h
```

See user-space processes

The process tree shows that every running program is a child of an existing process and has a user identity. For Odoo, this makes it clear whether work is running under systemd, an interactive shell, or an accidental manual start.

```
ps -eo pid,ppid,user,stat,comm --sort=pid | head -20
pstree -ap | head -30
```

Common Mistakes

- Treating Linux as a black box and jumping directly into Odoo configuration.
- Assuming two Ubuntu releases behave identically because the commands look similar.
- Ignoring kernel limits such as open files, memory pressure, and socket backlog.
- Changing production settings without first recording the current system state.

Lab / Practice: On a lab VM, record kernel, distribution, CPU, memory, and top-level process information. Write a short note explaining which layer would own a network failure, a Python import failure, and an out-of-memory crash.

L02 Terminal Navigation with pwd, cd, ls, and tree

Learning Objectives

- Navigate absolute and relative paths confidently.
- Use directory listings to understand an unfamiliar server.
- Distinguish the current shell directory from service working directories.
- Build safe habits before editing production files.

Explanation

Terminal Navigation with pwd, cd, ls, and tree matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands filesystem navigation and situational awareness can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to current directory, hidden files, symlinks, ownership, and directory depth and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is editing or deleting the wrong file because the shell was in an unexpected directory, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo source, custom addons, logs, configuration, and backups usually live in different directories with different owners connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Say the path out loud, print it with pwd, and list it before running a destructive command. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- pwd prints the shell's current working directory; it does not tell you where a service runs.
- cd changes directory for the current shell session only.
- ls -la shows hidden files, permissions, owners, sizes, and timestamps.
- tree is useful for understanding project shape but should be limited with -L on large directories.
- Absolute paths start at /, while relative paths start from the current directory.
- Symlinks are common in deployments and should be inspected before editing through them.

Build a safe navigation habit

This sequence confirms the starting point, moves to the expected application directory, and inspects contents before making changes. The limited tree depth gives a useful overview without flooding the terminal on a large addons directory.

```
pwd
cd /opt/odoo
pwd
ls -la
tree -L 2 /opt/odoo
```

Compare absolute and relative paths

The same file can be referenced through different path forms, but only the absolute form is independent of the current directory. Scripts and service files should prefer absolute paths so they behave consistently.

```
cd /etc/odoo
ls -l odoo.conf
ls -l /etc/odoo/odoo.conf
cd /
ls -l etc/odoo/odoo.conf
```

Common Mistakes

- Running `rm`, `mv`, or `chmod` before checking `pwd`.
- Forgetting that hidden dotfiles affect shell and application behavior.
- Using `tree` without a depth limit on large production directories.
- Editing a symlink target without realizing where the real file lives.

Lab / Practice: Create a small directory tree under `/tmp/odoo-navigation-lab`, navigate it using absolute and relative paths, then produce a one-screen map with `tree -L 3` and annotations for config, logs, and addons.

L03 File Manipulation: Create, Copy, Move, Link, and Remove

Learning Objectives

- Create and inspect files safely from the terminal.
- Copy and move files while preserving important metadata when needed.
- Understand hard links, symbolic links, and backup naming.
- Use cautious deletion practices on servers.

Explanation

File Manipulation: Create, Copy, Move, Link, and Remove matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands safe manipulation of files and directories can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to destination paths, overwrite behavior, timestamps, ownership, and symbolic links and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is destroying the only known-good configuration during a rushed fix, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo configuration files, custom modules, attachment filestore data, and backup archives require different handling connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Make reversible changes: copy first, edit second, verify third, and delete only after the replacement is proven. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- touch creates an empty file or updates timestamps.
- cp copies data; cp -a preserves mode, owner, timestamps, and symlinks where permissions allow.
- mv renames within a filesystem and moves across filesystems by copying then removing.
- ln -s creates a symbolic link, which stores a path reference rather than file content.
- rm is permanent at the shell level; production cleanup should be explicit and reviewed.
- Backups need names that show what changed, when, and why.

Back up and edit a configuration file

The archive copy preserves the original metadata and embeds a timestamp in the backup name. A diff after editing keeps the review focused on exactly what changed before the service is restarted.

```
sudo cp -a /etc/odoo/odoo.conf /etc/odoo/odoo.conf.bak.$(date +%Y%m%d-%H%M%S)
sudo nano /etc/odoo/odoo.conf
sudo diff -u /etc/odoo/odoo.conf.bak.* /etc/odoo/odoo.conf | tail -40
```

Use a symlink for active custom addons

A stable symlink such as current lets service configuration point at one path while releases change underneath it. The -n and -f flags replace the link itself rather than following an existing link as a directory.

```
sudo mkdir -p /opt/odoo/custom-addons/releases/2026-07-24
sudo ln -sf /opt/odoo/custom-addons/releases/2026-07-24 /opt/odoo/custom-addons/current
ls -l /opt/odoo/custom-addons/current
```

Common Mistakes

- Copying config files without preserving permissions, then wondering why services cannot read them.

- Using wildcard deletes without first listing what the wildcard expands to.
- Confusing a symlink with a physical copy of a directory.
- Leaving backup files in locations where applications accidentally load them.

Lab / Practice: Create a fake /tmp/odoo-file-lab config file, back it up with a timestamp, edit one setting, compare with diff, then replace a current symlink to point at a new fake release directory.

L04 Permissions with chmod, chown, and chgrp

Learning Objectives

- Read Unix permission bits and ownership from ls output.
- Apply chmod using symbolic and numeric modes.
- Change file owners and groups safely with chown and chgrp.
- Choose permissions suitable for Odoo configuration, source code, and logs.

Explanation

Permissions with chmod, chown, and chgrp matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands ownership and permission enforcement can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to read, write, execute bits, owner, group, others, directories, and recursive changes and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is masking an ownership problem by making sensitive files world-writable, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo should run as a dedicated user that can read its code and config but does not own everything on the server connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Prefer the smallest permission change that makes the intended actor able to do the intended action. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- Permission triplets apply to owner, group, and others in that order.
- Execute on a directory means the user can traverse it; read means the user can list names.
- chmod 640 is common for private configuration files readable by owner and group only.
- chown changes owner and optionally group; chgrp changes only the group.
- Recursive chmod and chown are high-risk operations on production systems.
- The service user, not the admin's interactive user, must have the required access.

Set safe Odoo configuration permissions

The root user owns the file so the service cannot silently rewrite it, while the odoo group can read it. Testing as the odoo user is more reliable than assuming permissions are correct from an admin shell.

```
sudo chown root:odoo /etc/odoo/odoo.conf
sudo chmod 640 /etc/odoo/odoo.conf
ls -l /etc/odoo/odoo.conf
sudo -u odoo test -r /etc/odoo/odoo.conf && echo "odoo can read config"
```

Repair ownership of a custom addon

Directories and files often need different execute bits, so a single recursive chmod is rarely ideal. This pattern keeps source readable by the service while avoiding broad write access.

```
sudo chown -R odoo:odoo /opt/odoo/custom-addons/my_module
find /opt/odoo/custom-addons/my_module -type d -exec chmod 750 {} \;
find /opt/odoo/custom-addons/my_module -type f -exec chmod 640 {} \;
```

Common Mistakes

- Using `chmod 777` as a troubleshooting shortcut and leaving it in place.
- Forgetting execute permission on parent directories.
- Changing ownership of system directories recursively from the wrong path.
- Testing access as root instead of the service user.

Lab / Practice: In `/tmp/odoo-permission-lab`, create a config file and addon directory. Set ownership and permissions so a simulated odoo user can read them but an unrelated user cannot modify them.

L05 Users, Groups, and sudo

Learning Objectives

- Explain why services should run under dedicated users.
- Inspect users, groups, and sudo privileges.
- Use sudo intentionally without hiding ownership problems.
- Separate human admin access from application runtime access.

Explanation

Users, Groups, and sudo matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands identity and privilege separation can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to user IDs, group memberships, service accounts, sudo rules, and command auditing and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is running application processes as root and widening the blast radius of a compromise, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo service users need predictable file access while administrators need controlled elevation for maintenance connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Use sudo to perform administrative tasks, not to make normal application operations depend on root. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- Each process runs with a user ID and group IDs that determine access checks.
- A service account should own only the files it must manage.
- sudo records administrative intent and can restrict which users may run privileged commands.
- Group membership is a clean way to grant shared read or write access.
- Human users should log in individually instead of sharing one admin password.
- Root-owned runtime files often reveal that a command was run with the wrong identity.

Inspect identity and group membership

These commands answer who you are, who the service user is, and what privileged commands are allowed. The output is essential when a file is readable in your shell but not by the Odoo service.

```
id
id odoo
getent passwd odoo
getent group odoo
sudo -l
```

Run a check as the service user

Running a command as the service user reproduces the environment and access rules that systemd will use. The -H flag sets HOME consistently so user-specific configuration does not leak from the admin account.

```
sudo -u odoo -H bash -lc 'whoami; pwd; python3 --version'
sudo -u odoo -H test -x /opt/odoo/venv/bin/python && echo "venv executable"
```

Common Mistakes

- Running Odoo manually with sudo and creating root-owned cache files.
- Sharing one SSH account among administrators.
- Adding broad passwordless sudo rules without command boundaries.
- Changing service file ownership instead of fixing group membership.

Lab / Practice: Create a lab service user, place it in a project group, and demonstrate a file that the group can read but cannot write. Then run a Python version check as that service user.

L06 Processes with top, htop, ps, and kill

Learning Objectives

- List processes and interpret PID, parent PID, state, CPU, and memory columns.
- Use top or htop to observe live server pressure.
- Terminate processes with the least disruptive signal first.
- Connect process behavior to Odoo worker and cron performance.

Explanation

Processes with top, htop, ps, and kill matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands runtime process observation and control can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to PIDs, process states, parent-child relationships, CPU use, RSS memory, and signals and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is killing the wrong worker or masking the underlying cause of resource exhaustion, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo may run multiple HTTP workers, longpolling workers, cron workers, PostgreSQL sessions, and background jobs connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. A kill command is a last step in an investigation, not the investigation itself. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- ps gives a point-in-time process list; top and htop show live changes.
- PID identifies a process instance, while command names may repeat many times.
- SIGTERM asks a process to stop cleanly; SIGKILL stops it immediately and prevents cleanup.
- High CPU, high memory, and uninterruptible IO wait suggest different root causes.
- Parent PID helps distinguish systemd-managed services from manual shells.
- Repeatedly killing processes without logs creates unstable systems and lost evidence.

Find Odoo-related processes

Sorting by memory and filtering by the odoo user quickly reveals whether application workers dominate the server. Elapsed time helps separate a fresh runaway process from a long-running service.

```
ps -eo pid,ppid,user,stat,%cpu,%mem,etime,cmd --sort=-%mem | head -20
pgrep -a -u odoo python
top -o %MEM
```

Signal a process deliberately

TERM gives the process a chance to close files and database connections cleanly. KILL should be explicit and documented because it can interrupt transactions or leave temporary files behind.

```
sudo kill -TERM 12345
sleep 3
ps -p 12345 -o pid,stat,cmd
sudo kill -KILL 12345 # only if the process ignored TERM and impact is understood
```

Common Mistakes

- Using kill -9 as the first response.
- Matching processes by a vague name and terminating unrelated commands.
- Ignoring parent processes that will immediately restart the child.
- Confusing memory percent with actual RSS on large-memory servers.

Lab / Practice: Start a harmless sleep process, find it with ps and pgrep, send TERM, confirm it exited, then repeat with a Python process that prints its PID and handles KeyboardInterrupt.

L07 systemd Services

Learning Objectives

- Explain systemd units, service states, and restart policies.
- Inspect and manage Odoo service lifecycle with systemctl.
- Understand unit files, drop-ins, and daemon reloads.
- Use systemd status output as a first troubleshooting artifact.

Explanation

systemd Services matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands service supervision and repeatable startup can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to unit names, active state, main PID, ExecStart, environment, dependencies, and restart behavior and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is starting Odoo manually and leaving production outside the service manager, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, systemd keeps Odoo, PostgreSQL, and Nginx predictable across reboots and failures connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. If systemd is responsible for the service, always ask systemd what it thinks before changing files. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- A service unit describes how a daemon starts, stops, reloads, and restarts.
- systemctl status combines state, main PID, recent logs, and exit code context.
- After editing unit files or drop-ins, run systemctl daemon-reload.
- Restart policies should match the failure mode and avoid tight crash loops.
- EnvironmentFile and WorkingDirectory affect how Python and Odoo locate resources.
- Manual starts should be used only for controlled diagnostics and then stopped.

Inspect and restart an Odoo service

status answers whether the service is active and shows the immediate failure context if it is not. cat displays the effective unit content, including drop-ins, which prevents debugging the wrong file.

```
sudo systemctl status odoo --no-pager
sudo systemctl cat odoo
sudo systemctl restart odoo
sudo systemctl is-active odoo
```

Create a safe override directory

Drop-ins preserve vendor or deployment unit files while making local changes explicit. daemon-reload is required because systemd caches unit definitions.

```
sudo systemctl edit odoo
# Add only the changed settings in the editor, for example:
# [Service]
# Restart=on-failure
sudo systemctl daemon-reload
sudo systemctl restart odoo
```

Common Mistakes

- Editing a unit file and restarting without daemon-reload.
- Debugging an old service name while the active unit has a different name.
- Running Odoo manually and assuming systemd uses the same environment.
- Setting Restart=always without understanding a crash loop.

Lab / Practice: Inspect a sample odoo.service unit, identify ExecStart, User, Group, WorkingDirectory, and EnvironmentFile, then create a hypothetical drop-in that changes only the restart policy.

L08 journalctl and Log Files

Learning Objectives

- Read systemd journal entries for a service.
- Filter logs by unit, time range, priority, and boot.
- Combine journal output with application log files.
- Preserve useful evidence during Odoo incidents.

Explanation

journalctl and Log Files matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands log-based investigation can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to unit filters, timestamps, boot IDs, priority levels, application log paths, and rotation and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is losing the first failure message by only looking at the last repeated error, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo logs often show Python tracebacks while systemd logs show process exit, restart, and environment failures connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Start with the first error in the relevant time window, then follow the chain forward. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- journalctl -u filters logs for a systemd unit.
- The --since and --until options keep searches tied to the incident window.
- The current boot can be selected with -b, and previous boots with -b -1.
- Application logs may live in /var/log/odoo or another configured path.
- Tracebacks should be read from top context to final exception, not just the last line.
- Log rotation can move older evidence into compressed files.

Read recent service logs

The first command establishes recent context, the second focuses on warnings and errors, and the third stays within the current boot. Time filters prevent unrelated old failures from confusing the incident timeline.

```
sudo journalctl -u odoo --since "30 minutes ago" --no-pager
sudo journalctl -u odoo -p warning..alert --since today --no-pager
sudo journalctl -u odoo -b --no-pager | tail -80
```

Follow application logs during restart

Following the application log during a restart shows startup module loading, database connection failures, and Python exceptions as they happen. Use this with journalctl status so you see both Odoo's view and systemd's view.

```
sudo tail -F /var/log/odoo/odoo.log
# In another terminal:
sudo systemctl restart odoo
```

Common Mistakes

- Searching logs without a time window.
- Reading only the final traceback line and missing the real cause above it.

- Ignoring systemd restart messages when the application log is empty.
- Overwriting logs during cleanup before preserving the incident evidence.

Lab / Practice: Restart a harmless sample service or timer, collect journal entries for the last 10 minutes, identify the first and last relevant lines, and write a five-line incident timeline.

L09 Editing Files with nano and vim

Learning Objectives

- Choose nano or vim based on task complexity and operator comfort.
- Open, edit, save, and exit safely in both editors.
- Avoid accidental configuration damage while editing as root.
- Validate edits before restarting services.

Explanation

Editing Files with nano and vim matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands terminal editing discipline can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to editor modes, save behavior, sudo usage, temporary files, syntax visibility, and post-edit validation and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is making an invisible typo in a production configuration file, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo and Nginx configuration changes often need exact key names, indentation, and path values connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. The best editor is the one you can exit reliably during an incident. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- nano is approachable for small edits and displays common shortcuts.
- vim is modal, powerful, and widely available on servers.
- Always back up important files before editing them as root.
- Use validation commands before restarting services that depend on edited config.
- sudoedit can be safer than running a full editor process as root.
- Editor swap and backup files should not be left where applications will load them.

Safe edit sequence

sudoedit copies the file to a temporary location, opens it with your editor, and writes it back with elevated permissions. The diff provides a review checkpoint before the changed service is restarted.

```
sudo cp -a /etc/odoo/odoo.conf /etc/odoo/odoo.conf.pre-edit
sudoedit /etc/odoo/odoo.conf
sudo diff -u /etc/odoo/odoo.conf.pre-edit /etc/odoo/odoo.conf
```

Essential vim commands

Most vim emergencies come from not recognizing insert mode versus normal mode. Knowing these commands is enough to make a controlled edit or escape safely.

```
vim /tmp/example.conf
# i      enter insert mode
# Esc    return to normal mode
# :w     write file
# :q     quit
# :wq    write and quit
# :q!    quit without saving
```

Common Mistakes

- Opening a root editor without a backup.
- Saving editor temporary files in a directory that is scanned by Odoo.
- Restarting services before checking a diff or validation command.
- Panicking in vim instead of pressing Esc and using :q! to exit without saving.

Lab / Practice: Edit a copied sample config with both nano and vim. Change one value, save it, show the diff, then intentionally abandon a second change without saving.

L10 Networking with ip, ping, and ss

Learning Objectives

- Inspect IP addresses, routes, and listening sockets.
- Use ping for basic reachability without overinterpreting it.
- Use ss to confirm which process owns a port.
- Troubleshoot Odoo, PostgreSQL, and reverse proxy connectivity.

Explanation

Networking with ip, ping, and ss matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands network observation from the host can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to interfaces, addresses, routes, DNS assumptions, TCP ports, process ownership, and listening bind addresses and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is assuming the application is broken when the port is not listening or traffic is routed elsewhere, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo typically listens on localhost or a private interface while Nginx exposes HTTP and HTTPS to users connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Separate reachability, name resolution, listening state, firewall policy, and application response. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- ip addr shows interface addresses; ip route shows where packets will go.
- ping tests ICMP reachability, not whether TCP application ports are open.
- ss -ltnp shows listening TCP sockets and owning processes.
- A service bound to 127.0.0.1 is reachable locally but not from another host.
- PostgreSQL and Odoo ports should usually be private, with Nginx handling public HTTP.
- Network troubleshooting improves when each layer is tested independently.

Inspect addresses, routes, and ports

The commands move from host addressing to routing, reachability, and listening sockets. For Odoo, the socket list often reveals whether the service is bound only to localhost or exposed more broadly.

```
ip addr show
ip route show
ping -c 3 8.8.8.8
ss -ltnp | grep -E ':(8069|8072|80|443|5432)\b'
```

Test an Odoo port locally

A local HTTP check proves the application responds before investigating DNS, load balancers, or public firewalls. The ss filter confirms which process owns the port.

```
curl -I http://127.0.0.1:8069
curl -I http://localhost:8069
ss -ltnp 'sport = :8069'
```

Common Mistakes

- Treating a successful ping as proof that HTTP or PostgreSQL works.

- Forgetting that localhost from one container or VM is not another host's localhost.
- Leaving Odoo bound to a public interface unnecessarily.
- Ignoring DNS and testing only raw IP addresses.

Lab / Practice: Draw a simple network path from browser to Nginx to Odoo to PostgreSQL. On a lab host, collect ip addr, ip route, and ss output and label which ports should be public.

L11 SSH Keys and Hardening

Learning Objectives

- Explain public-key SSH authentication.
- Generate and install SSH keys safely.
- Apply basic sshd hardening for production servers.
- Avoid locking yourself out during access changes.

Explanation

SSH Keys and Hardening matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands secure remote administration can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to key pairs, `authorized_keys`, `sshd_config`, password login, root login, ports, and active sessions and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is weakening or breaking the only administrative access path to a production host, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo servers need secure admin access because shell access often implies database, file, and secret access connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Make SSH changes with a second session open and a rollback path ready. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- The private key stays on the client; the public key is placed on the server.
- `authorized_keys` controls which public keys can log in as a user.
- `PasswordAuthentication no` reduces brute-force risk after keys are confirmed.
- `PermitRootLogin no` encourages named users and `sudo` auditability.
- `sshd` configuration should be tested before reload or restart.
- Firewall and cloud security group rules must match the chosen SSH port.

Create and install a key

Ed25519 keys are compact and strong for modern SSH clients. The final command proves the key works before password login is disabled.

```
ssh-keygen -t ed25519 -C "admin@example.com"
ssh-copy-id admin@odoo-server.example.com
ssh admin@odoo-server.example.com 'whoami; hostname'
```

Test sshd configuration before reload

`sshd -t` validates syntax without applying the configuration. Reload keeps existing sessions alive, which is safer than a restart while verifying access.

```
sudo cp -a /etc/ssh/sshd_config /etc/ssh/sshd_config.pre-hardening
sudoedit /etc/ssh/sshd_config
sudo sshd -t
sudo systemctl reload ssh
```

Common Mistakes

- Disabling password login before confirming key login works.

- Closing the only active session immediately after changing `sshd_config`.
- Sharing one private key across administrators.
- Changing the SSH port without updating firewall or cloud rules.

Lab / Practice: On a disposable VM, create a new SSH key, install it for a non-root user, test login, then draft a hardened `sshd_config` snippet with comments explaining each setting.

L12 Environment Variables and bashrc

Learning Objectives

- Read and set environment variables in a shell.
- Distinguish shell startup files from systemd service environments.
- Use PATH, HOME, and application variables intentionally.
- Avoid hiding production settings in interactive-only files.

Explanation

Environment Variables and bashrc matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands process environment configuration can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to exported variables, shell startup order, PATH lookup, virtual environments, and systemd EnvironmentFile usage and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is testing with variables from .bashrc that the Odoo service never receives, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo configuration may depend on PATH, Python virtual environments, locale, addons paths, and database connection variables connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. If a service needs a value, put it in the service configuration path, not only in an interactive shell profile. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- Environment variables are inherited from parent process to child process.
- export makes a shell variable available to child processes.
- .bashrc affects interactive bash shells, not all services.
- PATH controls command lookup order and can explain different Python binaries.
- systemd units can use Environment and EnvironmentFile for service-specific values.
- Secrets in environment files need strict ownership and permissions.

Inspect shell environment

These commands show which executable the shell will run and which inherited values are visible. A mismatch between command -v and service ExecStart is a frequent source of deployment confusion.

```
echo "$PATH"
printenv | sort | grep -E '^(HOME|LANG|PATH|VIRTUAL_ENV)= '
command -v python3
python3 -c 'import sys; print(sys.executable)'
```

Use an EnvironmentFile with systemd

An environment file makes service configuration explicit and auditable. The permissions allow the odoo group to read values while preventing broad disclosure.

```
sudo install -m 640 -o root -g odoo /dev/null /etc/odoo/odoo.env
echo 'ODOO_RC=/etc/odoo/odoo.conf' | sudo tee -a /etc/odoo/odoo.env
sudo systemctl edit odoo
# [Service]
# EnvironmentFile=/etc/odoo/odoo.env
```

Common Mistakes

- Adding production variables only to `~/.bashrc` and expecting `systemd` to see them.
- Modifying `PATH` globally when only one service needs a specific executable.
- Storing secrets in world-readable environment files.
- Assuming `sudo` preserves the caller's full environment.

Lab / Practice: Create a shell variable and an exported variable, run a child Python command that prints `os.environ`, and explain why only one value is visible. Then draft a service `EnvironmentFile` for a sample Odoo variable.

L13 Cron Jobs

Learning Objectives

- Understand cron timing syntax and execution environment.
- Create safe scheduled maintenance tasks.
- Redirect cron output for observability.
- Avoid conflicts between cron and Odoo's internal scheduler.

Explanation

Cron Jobs matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands scheduled command execution can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to minute, hour, day fields, user crontabs, system crontabs, PATH, working directory, and output handling and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is running a maintenance command with a minimal cron environment and no logs, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo has its own scheduled actions, but server tasks such as backups, cleanup, and certificate checks may use cron connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Every cron job should state who runs it, what environment it expects, where output goes, and how failure is noticed. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- User crontabs run as that user; /etc/cron.d entries include a user field.
- Cron has a small environment and often a different PATH from interactive shells.
- Use absolute paths for commands and files in cron jobs.
- Redirect stdout and stderr to logs or monitoring hooks.
- Locking prevents overlapping runs when a previous job is slow.
- Odoo database backups should be tested for restore, not just scheduled for creation.

A safe backup cron pattern

This system cron entry declares its shell and PATH, runs as postgres, prevents overlap with flock, and captures output. The command still needs a tested script and restore procedure before it is considered production-ready.

```
# /etc/cron.d/odoo-backup
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

15 2 * * * postgres flock -n /var/lock/odoo-backup.lock /usr/local/bin/backup-odoo-db >> /var/log/odoo/backup.log 2>
```

Preview a cron schedule

Servers may use both cron and systemd timers, so list both before assuming how scheduling works. Searching cron directories identifies system-wide jobs that are not visible in a user's crontab.

```
crontab -l
systemctl list-timers --all
grep -R "backup-odoo" /etc/cron* 2>/dev/null
```

Common Mistakes

- Using relative paths in cron commands.
- Ignoring stderr so failures disappear into local mail or nowhere.
- Scheduling backups without restore tests.
- Letting slow jobs overlap and overload the database server.

Lab / Practice: Write a cron entry that runs a harmless script every five minutes, logs stdout and stderr, and uses flock. Then intentionally remove PATH from the script and explain the failure.

L14 APT Package Management

Learning Objectives

- Update package metadata and install packages with apt.
- Understand repositories, package versions, and upgrades.
- Inspect installed packages and service changes.
- Apply package changes safely on Odoo servers.

Explanation

APT Package Management matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands Debian and Ubuntu package lifecycle can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to apt update, apt install, apt upgrade, repositories, package holds, service restarts, and changelog awareness and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is upgrading core packages during business hours without understanding service impact, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo servers depend on OS packages for Python, PostgreSQL clients, Nginx, wkhtmltopdf alternatives, fonts, and build tools connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Treat package changes like deployments: review, schedule, apply, verify, and record. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- apt update refreshes metadata; it does not upgrade installed packages.
- apt install changes the system and may start or restart services.
- apt-cache policy shows candidate versions and repositories.
- apt-mark hold can prevent a sensitive package from upgrading unexpectedly.
- Security updates are important but still need operational timing.
- Build dependencies should be removed or minimized on hardened production hosts.

Inspect and install a package

The policy command shows where the package will come from before installation. After installing, verify both package state and the service that the package may have started.

```
sudo apt update
apt-cache policy nginx
sudo apt install nginx
dpkg -l | grep '^ii' | grep nginx
systemctl status nginx --no-pager
```

Review pending upgrades

The simulated upgrade shows planned actions without changing the server. Use it before maintenance windows to identify packages that may restart services or require reboots.

```
sudo apt update
apt list --upgradable
sudo apt -s upgrade
```

Common Mistakes

- Confusing apt update with apt upgrade.
- Installing from random third-party repositories without trust or maintenance review.
- Upgrading PostgreSQL-related packages without checking database compatibility.
- Leaving build tools installed on minimal production machines unnecessarily.

Lab / Practice: On a lab Ubuntu system, inspect the candidate version of nginx, run a simulated upgrade, install a small package, and record which services changed state.

L15 Disk and Memory Usage with df, du, and free

Learning Objectives

- Measure filesystem capacity with df.
- Find large directories and files with du.
- Interpret memory and swap information with free.
- Connect disk and memory pressure to Odoo failures.

Explanation

Disk and Memory Usage with df, du, and free matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands capacity and pressure diagnosis can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to mounted filesystems, inode use, directory growth, cache memory, swap use, and database storage paths and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is cleaning the wrong directory while the full filesystem remains full, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo attachments, logs, PostgreSQL data, backups, and temporary exports can grow independently connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Always identify the full mount point before deleting data. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- df -h shows filesystem capacity by mount point.
- df -ih shows inode usage, which can be exhausted even when space remains.
- du -sh summarizes directory size; sort helps find the largest consumers.
- free -h separates used memory from available memory and filesystem cache.
- Swap activity can indicate memory pressure but is not automatically bad.
- Database and filestore cleanup require application-aware retention rules.

Find the full mount and largest directories

The -x flag keeps du on one filesystem, which prevents chasing data across mounted volumes. Checking inodes helps diagnose failures caused by too many small files, such as excessive sessions or attachments.

```
df -h
df -ih
sudo du -xhd1 /var | sort -h
sudo du -xhd1 /opt | sort -h
```

Inspect memory pressure

free shows available memory, vmstat shows activity over time, and ps identifies the largest resident processes. Together they distinguish a busy but healthy cache from a server that is swapping heavily.

```
free -h
vmstat 1 5
ps -eo pid,user,%mem,rss,cmd --sort=-rss | head -15
```

Common Mistakes

- Deleting files from a different mount point than the one that is full.

- Ignoring inode exhaustion.
- Clearing application data without retention approval.
- Mistaking Linux filesystem cache for wasted memory.

Lab / Practice: Fill a small lab directory with test files, use `du` to find the largest path, then remove only the test data. Record `df` output before and after and explain why the numbers changed.

L16 UFW Firewall

Learning Objectives

- Explain host firewall allow and deny rules.
- Use UFW to expose only required services.
- Verify firewall status before and after changes.
- Design firewall policy for an Odoo server behind a reverse proxy.

Explanation

UFW Firewall matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands host-level network policy can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to default policies, allowed ports, source restrictions, IPv4 and IPv6, ordering, and rollback and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is locking out SSH or exposing internal Odoo and PostgreSQL ports publicly, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, a typical Odoo host should expose SSH and HTTPS while keeping PostgreSQL and backend Odoo ports private connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Before enabling a firewall, confirm the active SSH rule in the same session. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- UFW is a friendly interface over Linux firewall rules.
- Default deny incoming and allow outgoing is common for application hosts.
- Rules can restrict source IPs for SSH or administrative ports.
- Nginx should usually expose 80 and 443; Odoo 8069 should stay private.
- IPv6 must be considered if the host has public IPv6 addresses.
- Firewall changes need verification from both local and remote perspectives.

Enable a conservative Odoo firewall

This allows administration and public web traffic while denying other incoming connections. OpenSSH should be allowed before enabling UFW to avoid cutting off the active session.

```
sudo ufw status verbose
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status numbered
```

Restrict SSH to an admin network

Source-restricted SSH reduces attack surface when administrators connect from known networks. Apply this only after confirming the actual admin source ranges and any VPN paths.

```
sudo ufw delete allow OpenSSH
sudo ufw allow from 203.0.113.0/24 to any port 22 proto tcp
sudo ufw status numbered
```

Common Mistakes

- Enabling UFW before allowing SSH.
- Exposing PostgreSQL or Odoo backend ports to the entire internet.
- Forgetting IPv6 rules on dual-stack hosts.
- Deleting numbered rules without rechecking numbers after each deletion.

Lab / Practice: Design a UFW policy for a single-server Odoo deployment with Nginx, PostgreSQL local only, and SSH from a VPN range. Explain every allowed port.

L17 Production Directory Layout for Odoo

Learning Objectives

- Map common Odoo directories by responsibility.
- Separate source code, configuration, logs, data, backups, and virtual environments.
- Choose ownership and permissions for each area.
- Make deployments understandable for future maintainers.

Explanation

Production Directory Layout for Odoo matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands filesystem organization for maintainable deployments can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to config paths, addons paths, virtualenvs, logs, filestore, backups, releases, and ownership boundaries and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is mixing code, data, and backups until upgrades and restores become unsafe, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo deployments require clear paths for core source, custom addons, enterprise addons, Python environment, configuration, logs, and filestore connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. A directory layout is operational documentation; if it is surprising, it will fail during incidents. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- `/etc/odoo` is a natural place for root-owned configuration.
- `/opt/odoo` often holds application source, releases, addons, and virtual environments.
- `/var/log/odoo` should contain logs with rotation configured.
- `/var/lib/odoo` holds runtime data such as filestore when using packaged conventions.
- Backups should be isolated from live code and tested for restore.
- Symlinked release directories make rollback easier when used carefully.

Example directory map

This layout separates configuration, source, virtual environment, runtime data, logs, and backups. The exact paths may vary, but the responsibilities should remain explicit.

```
/etc/odoo/odoo.conf
/opt/odoo/src/odoo
/opt/odoo/enterprise
/opt/odoo/custom-addons/current
/opt/odoo/venv
/var/lib/odoo/.local/share/Odoo/filestore
/var/log/odoo/odoo.log
/srv/backups/odoo
```

Check ownership boundaries

`namei` shows permissions on each path component, which catches parent-directory traversal problems. The tests confirm the service can read configuration but cannot casually rewrite it.

```
sudo namei -l /etc/odoo/odoo.conf
sudo namei -l /opt/odoo/custom-addons/current
sudo -u odoo test -r /etc/odoo/odoo.conf && echo readable
sudo -u odoo test -w /etc/odoo/odoo.conf || echo not writable
```

Common Mistakes

- Storing backups inside the live addons path.
- Letting the service user own root-level configuration.
- Putting virtual environments in home directories tied to human users.
- Using undocumented paths that only one administrator understands.

Lab / Practice: Design a directory layout for a production Odoo server. Include owner, group, permission mode, backup policy, and which paths are referenced by the systemd service.

L18 Troubleshooting Checklist for Odoo Servers

Learning Objectives

- Apply a structured checklist during Odoo outages.
- Collect evidence before changing the server.
- Move from infrastructure checks to application checks logically.
- Write concise incident notes and next actions.

Explanation

Troubleshooting Checklist for Odoo Servers matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands systematic production troubleshooting can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to symptom scope, recent changes, service state, logs, ports, resources, database reachability, and rollback options and keeping the command output as evidence.

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is making multiple untracked changes and losing the ability to identify the fix, which is harder to diagnose after several unrelated changes.

In an Odoo deployment, Odoo availability depends on systemd, Python, Odoo configuration, PostgreSQL, Nginx, disk, memory, and network policy connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. A checklist is not bureaucracy; it is how you keep judgment available under pressure. turns Linux from a source of surprises into a stable operating base for Odoo work.

Key Points

- Start by defining the user-visible symptom and scope.
- Check recent changes before assuming random failure.
- Inspect service state, logs, ports, disk, memory, and database connectivity.
- Make one change at a time and record its result.
- Prefer rollback when a recent deployment caused the incident.
- Close the loop with prevention work, not just restoration.

First five minutes of an Odoo outage

This captures time, service state, recent Odoo logs, listening ports, disk, and memory in a repeatable order. It avoids the common mistake of restarting first and destroying useful failure context.

```
date -Is
sudo systemctl status odoo nginx postgresql --no-pager
sudo journalctl -u odoo --since "20 minutes ago" --no-pager | tail -120
ss -ltnp | grep -E ':(80|443|8069|5432)\b'
df -h
free -h
```

Check database reachability as Odoo

Testing from the service user's Python environment verifies more than raw network connectivity. It checks that the expected library, credentials path, and database access all work together.

```
sudo -u odoo -H bash -lc '  
    source /opt/odoo/venv/bin/activate  
    python - <<PY  
import psycopg2  
conn = psycopg2.connect(dbname="postgres", user="odoo", host="127.0.0.1")  
print("database connection ok")  
conn.close()  
PY  
'
```

Common Mistakes

- Restarting every service before collecting evidence.
- Changing several variables at once during an incident.
- Ignoring recent deployments, package upgrades, or certificate renewals.
- Stopping after recovery without writing the prevention task.

Lab / Practice: Use the checklist against a deliberately broken lab Odoo-like service. Record observations, identify the failing layer, make one fix, verify recovery, and write a brief post-incident note.

Part C Python Programming for Odoo Developers

This part contains 28 detailed lessons. Work in order. Complete each lab before moving forward when the lab unlocks the next dependency.

Presentation: Companion deck: see [training_materials/presentations/](#) for the Python Programming for Odoo Developers slides.

P01 What Is a Programming Language and Interpreter

Learning Objectives

- Explain what a programming language gives to humans and machines.
- Describe how an interpreter executes Python source code.
- Distinguish source code, bytecode, runtime objects, and process state.
- Connect interpreted execution to rapid Odoo development.

Explanation

What Is a Programming Language and Interpreter is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding the role of language rules, runtime behavior, and interpretation helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is source code is parsed into instructions that create and manipulate objects while the interpreter manages execution. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is thinking that code is just text rather than instructions with runtime state and side effects, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, server actions, models, controllers, and scripts are all Python instructions executed by a long-running process. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Running small files and inspecting values builds confidence before touching framework code. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- A programming language defines syntax, semantics, and a standard way to express computation.
- The Python interpreter reads code, builds objects, executes statements, and reports errors.
- Interactive execution is useful for experiments, but production code must be repeatable.
- The same source code can behave differently when the environment, imports, or data changes.
- Odoo developers need both language knowledge and framework knowledge.
- Understanding the interpreter explains why restarts are required after many code changes.

Observe Python executing statements

The source code creates a string object, binds it to a name, and passes it to functions. Seeing the type and length makes the invisible runtime state concrete.

```
message = "hello Odoo developer"
print(message)
print(type(message))
print(len(message))
```

Common Mistakes

- Treating Python files as configuration rather than executable code.
- Assuming an interactive experiment automatically matches Odoo's runtime environment.
- Ignoring errors because the interpreter stops at the first failing statement.
- Forgetting that a long-running Odoo process may need a restart to load changed code.

Lab / Practice: Create a file named `interpreter_lab.py` that prints three values and their types. Run it, change one value, run again, and explain what the interpreter did each time.

P02 Running Python, First Program, and Python 2 vs Python 3

Learning Objectives

- Run Python from a file and from an interactive shell.
- Write a first program that accepts input and prints output.
- Explain why Python 3 is the standard for modern Odoo work.
- Recognize Python 2 compatibility problems in old code.

Explanation

Running Python, First Program, and Python 2 vs Python 3 is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding executing Python consistently and recognizing version boundaries helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a command chooses a specific interpreter binary, and that interpreter defines available syntax and libraries. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is running code with the wrong interpreter and misdiagnosing syntax or dependency errors, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, modern Odoo versions use Python 3, while older community snippets may still contain Python 2 assumptions. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Always print `sys.version` and `sys.executable` when the environment is uncertain. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- `python3` usually selects Python 3 on Linux; `python` may vary by system.
- `sys.executable` shows the exact interpreter used for the running process.
- Python 3 treats `print` as a function and text as Unicode strings.
- Python 2 is end of life and should not be used for new Odoo development.
- Virtual environments change which interpreter and packages a command uses.
- A first program should be small enough that every line can be explained.

Confirm interpreter identity

This program proves which Python is running before any dependency debugging begins. It is a simple but valuable habit on servers with system Python, virtual environments, and deployment scripts.

```
import sys

print(sys.version)
print(sys.executable)
print("Hello from Python 3")
```

A small interactive program

`input` returns a string, `strip` removes surrounding whitespace, and an f-string formats the response. Even this tiny program demonstrates data flow from user input to output.

```
name = input("Module name: ").strip()
print(f"Preparing scaffold for {name}")
```

Common Mistakes

- Using python when the server expects python3.
- Copying Python 2 snippets with print statements or old exception syntax.
- Installing packages into one interpreter and running another.
- Skipping version checks during deployment troubleshooting.

Lab / Practice: Write a `first_program.py` file that asks for a developer name and prints the Python executable, version, and a greeting. Run it with the interpreter from a virtual environment if available.

P03 Data Types Overview

Learning Objectives

- Name the common built-in Python data types.
- Explain mutable versus immutable objects.
- Use type inspection to understand unfamiliar values.
- Choose appropriate data structures for Odoo code.

Explanation

Data Types Overview is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding the shape and behavior of runtime values helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is every value is an object with a type, identity, and operations that may or may not mutate the object. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is using a value as if it were another type and producing subtle framework errors, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, recordsets, dictionaries, lists, strings, dates, booleans, and numbers move constantly between models, controllers, and templates. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Inspecting type and representation in small examples prevents wrong assumptions in larger methods. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- Core types include None, bool, int, float, str, list, tuple, dict, set, and custom classes.
- Mutable objects can change in place; immutable objects produce new values when transformed.
- `type(value)` and `isinstance(value, type)` answer different questions.
- Choosing the right type communicates intent to future maintainers.
- Framework APIs often expect a specific type, not merely a similar-looking value.
- Readable code makes conversions explicit at boundaries.

Inspect common values

`repr` shows an unambiguous display of each value, and `type` reveals what operations are valid. This is a useful debugging pattern when values arrive from JSON, forms, or ORM calls.

```
values = [None, True, 42, 19.5, "odoo", ["sale"], ("draft",), {"state": "draft"}, {"crm", "sale"}]

for value in values:
    print(repr(value), type(value).__name__)
```

Common Mistakes

- Assuming a string that looks like a number behaves like a number.
- Mutating a list that is shared by another part of the program.
- Using `type(value) == SomeType` when `isinstance` would handle subclasses better.
- Returning None where a caller expects an empty list or dictionary.

Lab / Practice: Build a table of ten Python values with their type, whether they are mutable, and one valid operation. Include at least one value that would commonly appear in Odoo data.

P04 Numbers, Math, and Operator Precedence

Learning Objectives

- Use integer, float, and decimal-style thinking appropriately.
- Apply arithmetic operators and understand division behavior.
- Explain how operator precedence affects expressions.
- Write clear numeric code for business logic.

Explanation

Numbers, Math, and Operator Precedence is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding numeric operations and expression grouping helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is operators combine operands according to precedence rules unless parentheses make the intended order explicit. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is encoding financial or quantity calculations with unclear grouping or inappropriate float assumptions, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, prices, taxes, quantities, discounts, and stock moves depend on careful numeric logic. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Use parentheses and named intermediate values when a business rule matters. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- `int` represents whole numbers of arbitrary size.
- `float` represents binary floating-point values and can surprise in decimal money calculations.
- `/` returns true division, `//` returns floor division, and `%` returns remainder.
- `**` binds more tightly than multiplication and addition.
- Parentheses are better than relying on readers to remember precedence.
- Odoo monetary logic should usually use framework currency rounding utilities.

Make precedence visible

Both expressions are syntactically valid, but they encode different business rules. Parentheses turn the pricing rule into visible documentation.

```
subtotal = 100
discount = 10
tax_rate = 0.15

wrong = subtotal - discount * (1 + tax_rate)
right = (subtotal - discount) * (1 + tax_rate)

print(wrong)
print(right)
```

Division operators

True division, floor division, and remainder answer different questions. Stock and packaging code often needs all three, but each should be named clearly.

```
quantity = 17
box_size = 5

print(quantity / box_size)
print(quantity // box_size)
print(quantity % box_size)
```

Common Mistakes

- Using floats for money without understanding rounding requirements.
- Relying on precedence in long business expressions.
- Confusing / with // when calculating batches or pages.
- Forgetting that negative numbers affect floor division behavior.

Lab / Practice: Implement a small price calculation with subtotal, discount, and tax. Write two versions, one compact and one using named intermediate values, then compare readability.

P05 Variables, Expressions, and Statements

Learning Objectives

- Define variables as name bindings to objects.
- Distinguish expressions that produce values from statements that perform actions.
- Write assignments that clarify business intent.
- Avoid misleading names and accidental rebinding.

Explanation

Variables, Expressions, and Statements is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding names, values, and executable instructions helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a variable name points to an object, expressions compute values, and statements tell the interpreter to do work. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is believing assignment copies every object or that names and values are the same thing, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, model methods often combine ORM recordsets, computed values, and side-effect statements such as writes. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Trace names through a method before changing logic that writes to records. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- Assignment binds a name to an object; it does not always copy the object.
- An expression can be used where a value is needed.
- A statement changes control flow, binding, or program state.
- Good variable names explain domain meaning, not just type.
- Rebinding a name can be clear or confusing depending on scope and length.
- Side-effect statements should be easy to find in business methods.

Name binding with mutable values

same_tags and tags point to the same list, while copy_tags points to a separate list. This distinction matters when preparing values for ORM create and write calls.

```
tags = ["vip", "renewal"]
same_tags = tags
copy_tags = list(tags)

same_tags.append("priority")

print(tags)
print(copy_tags)
```

Common Mistakes

- Using names like data or temp for important business concepts.
- Assuming assignment makes a deep copy.
- Hiding database writes among many harmless expressions.
- Reusing one variable name for unrelated meanings in the same method.

Lab / Practice: Write a short script that builds an order summary from named variables. Then refactor it so each variable name explains a business concept rather than a data type.

P06 Strings: Concatenation, Formatting, Indexes, and Immutability

Learning Objectives

- Create and combine strings safely.
- Use f-strings for readable formatting.
- Index and slice strings without mutating them.
- Handle text values carefully in Odoo fields and logs.

Explanation

Strings: Concatenation, Formatting, Indexes, and Immutability is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding text values and immutable sequence behavior helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a string is an immutable sequence of characters, so operations produce new strings instead of changing the original. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is building unclear or unsafe text output with manual concatenation and hidden conversions, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, names, emails, XML IDs, domains, log lines, and translated labels are all text-heavy surfaces. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Use formatting for display and structured values for logic. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- Strings can be indexed and sliced like sequences.
- Immutability means assigning to a string index is not allowed.
- f-strings are usually the clearest formatting tool for local variables.
- join is preferable for combining many strings.
- strip, lower, and replace return new strings.
- Text used in SQL, XML, or HTML needs the appropriate framework escaping rules.

Format a user-facing label

The f-string keeps the formatting rule close to the values and makes decimal display explicit. The slice returns a new string; it does not change the original label.

```
partner = "Azure Interior"
amount = 1250.5
currency = "USD"

label = f"{partner}: {amount:,.2f} {currency}"
print(label)
print(label[0:14])
```

Join many parts

join communicates that a separator is being placed between existing pieces. It is clearer and more efficient than repeated concatenation in loops.

```
parts = ["sale", "order", "confirmed"]
xml_id = ".".join(parts)
print(xml_id)
```

Common Mistakes

- Concatenating strings and numbers without explicit conversion or formatting.
- Expecting string methods to modify the original value.
- Using slicing indexes without naming why the slice matters.
- Manually building SQL or HTML strings instead of using safe APIs.

Lab / Practice: Create formatted labels for three fake invoices using f-strings. Include amount formatting, then demonstrate that calling upper on a string returns a new value.

P07 Booleans and Type Conversion

Learning Objectives

- Use True and False values in decisions.
- Explain truthy and falsy values.
- Convert between common types explicitly.
- Avoid ambiguous conversions in business rules.

Explanation

Booleans and Type Conversion is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding truth values and explicit boundaries between types helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is Python asks objects for truth value in conditionals, while conversion functions build new values of requested types. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is treating empty values, strings, numbers, and None as interchangeable, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, forms and integrations often deliver strings that must become booleans, numbers, dates, or empty values intentionally. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Normalize external input at the boundary and keep core logic typed clearly. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- False, None, zero, empty strings, and empty containers are falsy.
- Non-empty strings are truthy, even the string 'False'.
- bool, int, float, str, list, and dict perform conversions when valid.
- Explicit conversion makes validation errors visible.
- Use is None when testing for the absence marker specifically.
- Odoo field values may distinguish False, 0, empty strings, and empty recordsets.

Truthiness surprises

The non-empty strings are truthy even though humans may read them as false-like. External text input should be parsed with domain-specific rules instead of passed directly to bool.

```
samples = [False, None, 0, "", [], "False", "0"]

for sample in samples:
    print(repr(sample), bool(sample))
```

Validate conversion

Conversion and validation are separate steps. This makes invalid input fail early instead of spreading string values through numeric logic.

```
raw_quantity = "12"
quantity = int(raw_quantity)

if quantity <= 0:
    raise ValueError("quantity must be positive")

print(quantity * 2)
```

Common Mistakes

- Using `bool('False')` and expecting `False`.
- Testing if value when `None` and zero mean different things.
- Converting user input without handling `ValueError`.
- Letting raw JSON strings reach ORM write values unchecked.

Lab / Practice: Write a parser for a yes/no text value that accepts yes, no, true, false, 1, and 0. Make invalid input raise `ValueError` with a clear message.

P08 Lists: Slicing, Matrix Data, Methods, and Unpacking

Learning Objectives

- Create and modify lists safely.
- Use indexes, slices, and common methods.
- Represent matrix-like data with nested lists.
- Unpack list values clearly.

Explanation

Lists: Slicing, Matrix Data, Methods, and Unpacking is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding ordered mutable collections helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a list stores references in order and can be changed in place by index, slice, or method calls. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is mutating shared lists or confusing shallow copies with independent nested data, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo code often builds command lists, report rows, import buffers, and wizard selections. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Copy deliberately and keep list mutation close to where the list is created. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- `append` adds one item; `extend` adds items from an iterable.
- Slicing returns a new list, while slice assignment mutates the original list.
- Nested lists can represent rows, but each row must be a separate list.
- Unpacking assigns multiple names from a sequence in one statement.
- `sort` mutates a list; `sorted` returns a new sorted list.
- Lists are good for ordered, repeatable collections.

List methods and slicing

`append` and item assignment change the list in place, while the slice creates a new list. This distinction is important when one function should not modify a caller's list.

```
states = ["draft", "sent", "sale"]
states.append("done")
first_two = states[:2]
states[1] = "quoted"

print(states)
print(first_two)
```

Unpack rows

Unpacking gives names to positions and makes row structure obvious. If the row shape changes, Python raises an error instead of silently producing the wrong report.

```
rows = [  
    ["S0001", 100.0],  
    ["S0002", 250.0],  
]  
  
for order_name, total in rows:  
    print(f"{order_name}: {total:.2f}")
```

Common Mistakes

- Using one inner list repeatedly when building a matrix.
- Calling sort when the original order must be preserved.
- Appending a list when extend was intended.
- Changing a list passed into a function without documenting that side effect.

Lab / Practice: Build a list of invoice rows, slice the first three, sort a copy by amount, and unpack each row into named variables while printing a summary.

P09 Dictionaries: Keys, Values, and Methods

Learning Objectives

- Use dictionaries for named data.
- Access keys safely with direct lookup and get.
- Iterate over keys, values, and items.
- Prepare dictionaries for ORM create and write operations.

Explanation

Dictionaries: Keys, Values, and Methods is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding mapping keys to values helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a dictionary stores unique hashable keys, and each key identifies one current value. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is using loose dictionaries without knowing which keys are required or optional, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo create and write methods receive dictionaries whose keys must match field names and whose values must be valid for those fields. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Treat dictionaries at framework boundaries as contracts, not bags of random values. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- Dictionary keys must be unique and hashable.
- `dict[key]` raises `KeyError` when missing; `dict.get(key)` can provide a default.
- `items` returns key-value pairs for iteration.
- `update` merges values and overwrites existing keys.
- `in` checks keys, not values.
- Clear key names make business payloads easier to review.

Build ORM values

The dictionary mirrors the field-value contract used by Odoo's create method. Checking required keys before calling the ORM produces clearer errors.

```
partner_vals = {
    "name": "Azure Interior",
    "email": "info@example.com",
    "customer_rank": 1,
}

required = ["name", "email"]
missing = [key for key in required if not partner_vals.get(key)]
print(missing)
print(partner_vals.items())
```

Common Mistakes

- Using `get` for required keys and accidentally accepting missing data.
- Overwriting a key during update without noticing.
- Assuming dictionaries preserve a business order in older Python contexts.
- Passing unknown field names into ORM write values.

Lab / Practice: Create a dictionary for a fake product with name, default_code, list_price, and active. Validate required keys and print each key-value pair in a readable format.

P10 Tuples

Learning Objectives

- Create tuples and understand their immutability.
- Use tuples for fixed-size grouped values.
- Unpack tuples in assignments and loops.
- Recognize tuple syntax in Odoo domains and commands.

Explanation

Tuples is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding fixed grouped values helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a tuple is an immutable ordered container, often used when position and size are part of the meaning. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is confusing tuple immutability with immutability of objects inside the tuple, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, domains, selection values, and relational command patterns use tuple-shaped data frequently. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Use tuples when the shape of the data is part of the contract. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- A one-item tuple needs a trailing comma.
- Tuples are immutable, but contained mutable objects can still change.
- Tuple unpacking documents expected position meanings.
- Tuples are hashable only if all contained values are hashable.
- Odoo domains commonly use three-item tuples such as ('field', '=', value).
- Fixed shapes should be validated before dynamic construction.

Domain tuple shape

Each tuple has a fixed three-part meaning, and unpacking enforces that expectation. This mirrors a common Odoo domain pattern.

```
domain = [
    ("state", "=", "sale"),
    ("amount_total", ">", 1000),
]

for field, operator, value in domain:
    print(field, operator, value)
```

Common Mistakes

- Writing ('draft') and thinking it is a tuple instead of a string.
- Trying to assign to a tuple element.
- Assuming mutable objects inside a tuple cannot change.
- Using positional tuples where a dictionary would be clearer.

Lab / Practice: Build three domain tuples and unpack them in a loop. Add one malformed tuple and observe the error, then explain why early failure is useful.

P11 Sets

Learning Objectives

- Create sets for unique values.
- Use membership and set operations.
- Choose sets over lists for uniqueness and fast lookup.
- Apply sets to validation and import cleanup tasks.

Explanation

Sets is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding unordered unique collections helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a set stores unique hashable values and supports mathematical operations such as union and intersection. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is using lists for uniqueness checks and producing duplicate business actions, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, imports, access checks, tags, and external identifiers often need duplicate detection. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Convert to a set when uniqueness is the rule, then sort if presentation order matters. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- Sets remove duplicates automatically.
- Membership checks in sets are efficient.
- Union, intersection, and difference express comparison logic clearly.
- Sets are unordered and should not drive display order directly.
- Only hashable values can be set members.
- An empty set is created with `set()`, not `{}`.

Find duplicate external IDs

The seen set makes duplicate detection direct and efficient. This pattern is useful before import jobs where duplicates would create confusing failures.

```
external_ids = ["sale.order_1", "sale.order_2", "sale.order_1"]
seen = set()
duplicates = set()

for xml_id in external_ids:
    if xml_id in seen:
        duplicates.add(xml_id)
    seen.add(xml_id)

print(duplicates)
```

Common Mistakes

- Expecting a set to preserve insertion order for presentation.
- Trying to put dictionaries or lists inside a set.
- Using `{}` when an empty set was intended.
- Converting to a set too early and losing meaningful duplicates.

Lab / Practice: Given two lists of module technical names, find modules in both lists, modules only in the first list, and the sorted union of all modules.

P12 Conditionals: Truthy Values, Ternary Expressions, and Short-Circuiting

Learning Objectives

- Write if, elif, and else branches clearly.
- Use truthy values deliberately.
- Apply ternary expressions only when they improve readability.
- Explain and use short-circuit boolean evaluation.

Explanation

Conditionals: Truthy Values, Ternary Expressions, and Short-Circuiting is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding controlled branching based on conditions helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is Python evaluates conditions to a truth value and executes only the matching branch, while and and or may skip later expressions. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is compressing business rules until nobody can see which case is handled, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, computed fields, constraints, button actions, and controllers often branch by state, permissions, and input completeness. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Prefer explicit branches for business policy and compact expressions for simple value selection. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- if starts a branch, elif adds alternatives, and else catches the remainder.
- and returns when the first falsy value is found; or returns when the first truthy value is found.
- A ternary expression has the form a if condition else b.
- Short-circuiting can prevent unsafe access to missing values.
- Complex conditionals deserve named boolean variables.
- Condition order should reflect business priority and safety.

Readable business branch

The branches read like a policy and make the exceptional cases visible. This is better than hiding the same rule in a dense expression.

```
state = "draft"
amount_total = 1500

if state != "draft":
    action = "readonly"
elif amount_total > 1000:
    action = "manager_approval"
else:
    action = "confirm"

print(action)
```

Short-circuit guard

The f-string is only built when the email value is truthy. Short-circuiting is helpful, but it should remain easy to read.

```
partner = {"name": "Azure", "email": ""}
message = partner.get("email") and f"Send mail to {partner['email']}"
print(message or "No email available")
```

Common Mistakes

- Using truthiness when zero, False, and None have different meanings.
- Writing nested ternary expressions for business rules.
- Forgetting that and and or return operands, not always True or False.
- Placing expensive or unsafe checks before cheap guard checks.

Lab / Practice: Write a decision function that returns an invoice approval state based on amount, customer status, and whether required fields are present. Refactor complex conditions into named booleans.

P13 for Loops, Iterables, range, and enumerate

Learning Objectives

- Iterate over sequences and other iterables.
- Use range for counted loops.
- Use enumerate when an index is genuinely needed.
- Write loop bodies that avoid accidental side effects.

Explanation

for Loops, Iterables, range, and enumerate is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding repeated work over iterable values helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a for loop asks an iterable for values one at a time and binds each value to the loop variable. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is looping by index when direct iteration would be safer and clearer, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo recordsets, lists of values, import rows, and report lines are all iterated constantly. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Choose the loop form that matches the data: values, index plus value, or generated counts. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- for item in items is the default readable loop.
- range produces integer sequences without storing a full list.
- enumerate provides index and value together.
- Loop variables are ordinary names and remain bound after the loop.
- Avoid modifying a list while iterating over it unless the pattern is deliberate.
- Odoo recordsets can be iterated directly like collections of records.

Enumerate import rows

enumerate keeps the human line number next to the row data without manual counters. This is ideal for import validation errors.

```
rows = [
    {"name": "A", "price": 10},
    {"name": "B", "price": 20},
]

for line_number, row in enumerate(rows, start=1):
    print(f"Line {line_number}: {row['name']} costs {row['price']}")
```

Common Mistakes

- Using range(len(items)) when the item itself is all that is needed.
- Changing the list being iterated and skipping elements unexpectedly.
- Forgetting start=1 when reporting human line numbers.
- Using loop variables after the loop without considering empty input.

Lab / Practice: Validate a list of CSV-like row dictionaries. Use `enumerate(start=1)` to collect readable error messages that include line numbers.

P14 while Loops, break, continue, and pass

Learning Objectives

- Use while loops for condition-driven repetition.
- Exit loops intentionally with break.
- Skip to the next iteration with continue.
- Use pass as an explicit placeholder only when appropriate.

Explanation

while Loops, break, continue, and pass is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding looping until a condition changes helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a while loop keeps executing while its condition remains truthy, so the body must move the program toward completion. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is creating infinite loops or hiding unfinished code behind pass, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, polling jobs, batch processing, and retry loops appear in integration and maintenance scripts around Odoo. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Make loop exit conditions visible and test them with small limits. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- while is best when the number of iterations is not known upfront.
- break exits the nearest loop immediately.
- continue skips the rest of the current iteration.
- pass does nothing and should not hide missing implementation.
- Loop counters and retry limits prevent runaway scripts.
- Sleep and backoff are important in real polling loops.

Batch loop with explicit progress

The loop condition is tied to the remaining work, and each iteration removes one item. continue skips one item without stopping the entire batch.

```
remaining = [1, 2, 3, 4, 5]
processed = []

while remaining:
    item = remaining.pop(0)
    if item == 3:
        continue
    processed.append(item)

print(processed)
```

Common Mistakes

- Writing while True without a clear break condition.
- Using pass where a real implementation or explicit error should exist.
- Forgetting to update the condition variable.

- Catching all errors inside a loop and continuing forever.

Lab / Practice: Write a retry loop that attempts a fake operation up to three times. Use break on success, continue on a recoverable validation issue, and raise an error after all retries fail.

P15 Functions: Parameters, args, kwargs, return, and Docstrings

Learning Objectives

- Define reusable functions with clear parameters.
- Use positional, keyword, *args, and **kwargs appropriately.
- Return values deliberately.
- Write docstrings that explain behavior and constraints.

Explanation

Functions: Parameters, args, kwargs, return, and Docstrings is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding packaging behavior behind a callable interface helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a function receives arguments, binds them to parameter names, executes a body, and optionally returns a value. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is creating functions with vague contracts that callers misuse, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo model methods are functions with framework-provided self, decorators, context, and return conventions. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Write the call site first in a small example, then design the function signature to make that call obvious. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- Parameters define what information a function needs.
- return sends a value back to the caller and stops function execution.
- *args collects extra positional arguments; **kwargs collects extra keyword arguments.
- Default arguments are evaluated once when the function is defined.
- Docstrings should state purpose, important parameters, return value, and notable errors.
- Small pure functions are easier to test than large methods with hidden side effects.

A clear calculation function

The signature exposes the required subtotal and optional discount, while validation protects the business rule. The docstring describes behavior rather than repeating the code.

```
def compute_discounted_total(subtotal, discount_rate=0.0):
    """Return subtotal after applying a decimal discount rate."""
    if not 0 <= discount_rate <= 1:
        raise ValueError("discount_rate must be between 0 and 1")
    return subtotal * (1 - discount_rate)

print(compute_discounted_total(100, discount_rate=0.15))
```

Forward keyword options

**kwargs is useful for flexible metadata, but the function still needs a clear purpose. Sorting details gives stable output for logs and tests.

```
def log_action(action, **details):
    print(action)
    for key, value in sorted(details.items()):
        print(f" {key}: {value}")

log_action("confirm_order", order="S0001", user="admin")
```

Common Mistakes

- Using mutable default arguments such as [] or {}.
- Returning different types for similar outcomes without documenting them.
- Accepting **kwargs and silently ignoring misspelled options.
- Writing docstrings that say what is obvious instead of why constraints exist.

Lab / Practice: Write a function that accepts product price, quantity, and optional discount. Validate inputs, return a total, and include a docstring with one example call.

P16 Scope, global, and nonlocal

Learning Objectives

- Explain local, enclosing, global, and built-in scopes.
- Understand name resolution with the LEGB rule.
- Use global and nonlocal sparingly and deliberately.
- Avoid hidden shared state in Odoo modules.

Explanation

Scope, global, and nonlocal is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding where names are looked up and rebound helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is Python searches local, enclosing, global, and built-in scopes for a name, while assignment normally binds in the local scope. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is using hidden module-level state that behaves unpredictably under long-running workers, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo workers may serve many requests, so module globals can leak state between users or transactions. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Pass values explicitly unless shared state is truly part of the design. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- LEGB means local, enclosing, global, built-in.
- Reading a global name is different from rebinding it.
- global tells Python assignment should target the module scope.
- nonlocal targets an enclosing function scope.
- Module constants are acceptable; mutable module caches need careful invalidation.
- Framework context should usually travel through self.env.context, parameters, or records.

Local and enclosing scope

nonlocal allows the inner function to rebind a name from the enclosing function. This is powerful, but in application code explicit objects are often easier to reason about.

```
def make_counter():
    count = 0

    def next_value():
        nonlocal count
        count += 1
        return count

    return next_value

counter = make_counter()
print(counter())
print(counter())
```

Common Mistakes

- Using global to avoid passing parameters.
- Storing request-specific data in module-level variables.
- Shadowing built-in names such as list, dict, or id.
- Assuming each web request starts with fresh module globals.

Lab / Practice: Write a function that accidentally shadows a built-in, then fix the name. Next, create a small closure with nonlocal and explain why you would avoid the same pattern for request data.

P17 OOP: Classes, `__init__`, and Methods

Learning Objectives

- Define classes and create instances.
- Use `__init__` to initialize object state.
- Write instance methods that use `self` correctly.
- Connect Python classes to Odoo models.

Explanation

OOP: Classes, `__init__`, and Methods is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding bundling data and behavior into objects helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a class defines behavior, an instance holds state, and methods receive the instance as `self`. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is placing behavior in disconnected helper code when object state would make the design clearer, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo models are Python classes whose fields and methods are extended by the framework metaclass and registry. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Practice with plain classes first so Odoo model classes feel less magical. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- `class` creates a new type.
- `__init__` runs when an instance is created.
- `self` is the current instance passed automatically to instance methods.
- Instance attributes belong to one object unless shared deliberately.
- Methods should operate on the object's coherent state.
- Odoo model methods receive recordsets as `self`, not always one record.

Plain class with state

The instance owns `number` and `total`, and the method formats state through `self`. This mirrors how model methods organize behavior around record data.

```
class InvoiceSummary:
    def __init__(self, number, total):
        self.number = number
        self.total = total

    def label(self):
        return f"{self.number}: {self.total:.2f}"

summary = InvoiceSummary("INV001", 250.0)
print(summary.label())
```

Common Mistakes

- Forgetting `self` in method definitions.
- Using class attributes when instance attributes were intended.
- Doing too much work in `__init__` instead of keeping construction simple.
- Assuming an Odoo recordset method always receives a single record.

Lab / Practice: Create a plain ProductLabel class with name, default_code, and price. Add a method that returns a formatted label and test it with two instances.

P18 classmethod and staticmethod

Learning Objectives

- Distinguish instance methods, class methods, and static methods.
- Use classmethod for alternate constructors or class-level behavior.
- Use staticmethod only when no instance or class state is needed.
- Avoid unnecessary method types in framework code.

Explanation

classmethod and staticmethod is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding choosing the right method binding helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is instance methods receive self, class methods receive cls, and static methods receive neither automatically. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is using method decorators to hide unclear responsibilities, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo has its own API decorators, so plain Python method decorators should be used only when the semantics are clear. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Prefer ordinary instance methods until a class-level or stateless behavior is truly needed. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- Instance methods are the default and receive self.
- classmethod receives the class and can create instances polymorphically.
- staticmethod is namespaced on the class but has no automatic access to state.
- Alternate constructors are a common classmethod use case.
- Utility functions do not always need to live inside a class.
- Framework decorators and Python decorators solve different problems.

Alternate constructor

from_string receives cls, so subclasses could reuse the constructor correctly. The currency check is static because it does not need instance or class state.

```
class Money:
    def __init__(self, amount, currency):
        self.amount = amount
        self.currency = currency

    @classmethod
    def from_string(cls, text):
        amount_text, currency = text.split()
        return cls(float(amount_text), currency)

    @staticmethod
    def is_supported_currency(currency):
        return currency in {"USD", "EUR"}

money = Money.from_string("10.50 USD")
print(money.amount, money.currency)
print(Money.is_supported_currency("USD"))
```

Common Mistakes

- Using staticmethod to avoid thinking about where a function belongs.
- Using classmethod when no class-level behavior exists.
- Confusing Python decorators with Odoo api decorators.
- Putting business logic in utility methods that bypass record rules or environment context.

Lab / Practice: Create a small CodeParser class with a classmethod from_text and a staticmethod is_valid_code. Explain which method would become an instance method if it needed stored state.

P19 Encapsulation, Inheritance, Polymorphism, and super

Learning Objectives

- Explain encapsulation as controlled access to behavior and state.
- Use inheritance to specialize behavior.
- Recognize polymorphism through shared method contracts.
- Call parent behavior correctly with super.

Explanation

Encapsulation, Inheritance, Polymorphism, and super is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding object-oriented extension patterns helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is objects expose behavior through methods, subclasses can override methods, and callers can rely on shared method names. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is overriding framework methods without preserving parent behavior or contracts, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo customization frequently inherits models and extends methods such as create, write, unlink, and action methods. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Before overriding, read the parent contract and decide whether to run code before or after super. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- Encapsulation keeps callers focused on behavior rather than internal representation.
- Inheritance models an is-a relationship or framework extension point.
- Polymorphism lets different classes respond to the same method call.
- super delegates to the next implementation in the method resolution order.
- Overridden methods must preserve expected arguments, return values, and side effects.
- Odoo super calls are essential for keeping core framework behavior intact.

Extend behavior with super

The subclass adds invoice-specific behavior without discarding the parent checks. This is the same mindset needed when extending Odoo model methods.

```
class Document:
    def validate(self):
        return ["base checks"]

class Invoice(Document):
    def validate(self):
        checks = super().validate()
        checks.append("invoice checks")
        return checks

print(Invoice().validate())
```

Common Mistakes

- Overriding a method and forgetting super when parent behavior is required.
- Changing the return type of an inherited method.

- Using inheritance where composition or a helper function would be clearer.
- Accessing internal attributes from outside instead of using methods.

Lab / Practice: Build a base Approval class and two subclasses with different validation rules. Ensure each subclass calls super and returns the same type.

P20 Dunders, MRO, and Multiple Inheritance

Learning Objectives

- Recognize common double-underscore methods.
- Explain method resolution order.
- Understand the risks and uses of multiple inheritance.
- Design cooperative methods that work with super.

Explanation

Dunders, MRO, and Multiple Inheritance is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding Python's data model and inheritance lookup helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is dunder methods let objects participate in Python protocols, and MRO defines the order used to find methods. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is creating inheritance diamonds that break because methods do not cooperate, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo models combine Python inheritance with framework inheritance mechanisms, so method lookup must be respected. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Use multiple inheritance only when each class has a narrow, cooperative responsibility. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- `__repr__`, `__str__`, `__len__`, and `__bool__` customize common Python operations.
- MRO can be inspected with `ClassName.mro()`.
- `super` follows MRO, not simply the textual parent class.
- Multiple inheritance works best when methods accept compatible signatures.
- Mixins should be small and focused.
- Odoo's `_inherit` is not identical to plain Python inheritance, but Python method rules still matter.

Inspect MRO

The MRO shows the actual lookup order, and `super` follows that order cooperatively. This is why mixins should call `super` when they participate in a shared method chain.

```
class AuditMixin:
    def save(self):
        print("audit")
        return super().save()

class Base:
    def save(self):
        print("base")
        return True

class Document(AuditMixin, Base):
    pass

print([cls.__name__ for cls in Document.mro()])
Document().save()
```

Readable object representation

`__repr__` makes debugging output precise and reconstructive. Useful representations save time when inspecting collections of domain objects.

```
class PartnerRef:
    def __init__(self, name, ref):
        self.name = name
        self.ref = ref

    def __repr__(self):
        return f"PartnerRef(name={self.name!r}, ref={self.ref!r})"

print(PartnerRef("Azure", "AZ001"))
```

Common Mistakes

- Assuming super always calls the immediate parent class.
- Writing mixins that do not call super in cooperative chains.
- Using dunder methods for cleverness instead of protocol clarity.
- Creating multiple inheritance hierarchies with incompatible method signatures.

Lab / Practice: Create two mixins and a base class with a cooperative process method. Print the MRO and show the order in which each method runs.

P21 map, filter, zip, reduce, and lambdas

Learning Objectives

- Use functional helpers where they improve clarity.
- Write simple lambda expressions.
- Combine related iterables with zip.
- Recognize when comprehensions or explicit loops are clearer.

Explanation

map, filter, zip, reduce, and lambdas is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding functional iteration tools helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is functions can be passed around as values, and iterable helpers apply them lazily or combine iterable streams. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is choosing clever expressions that hide business logic and complicate debugging, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo code benefits from readable transformations of records and values, but framework recordsets already provide rich APIs. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Use these tools for small transformations, not for hiding multi-step business rules. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- map applies a function to each item.
- filter keeps items whose predicate is truthy.
- zip pairs items from iterables by position.
- functools.reduce combines items into one result but can be less readable than a loop.
- lambda creates a small anonymous function expression.
- In many cases, comprehensions are more Pythonic than map or filter.

Transform and pair values

map performs a simple transformation, while zip pairs two related sequences. Converting map to a list is necessary when you want to inspect all results immediately.

```
names = ["sale", "crm", "stock"]
labels = list(map(str.upper, names))
enabled = [True, False, True]

for module, is_enabled in zip(names, enabled):
    print(module, is_enabled)

print(labels)
```

Reduce with care

reduce works, but sum(amounts) would be clearer for this particular operation. Choose the tool that makes the intent most obvious.

```
from functools import reduce

amounts = [10, 20, 30]
total = reduce(lambda current, amount: current + amount, amounts, 0)
print(total)
```

Common Mistakes

- Using lambda for complex logic that deserves a named function.
- Forgetting that map and filter are lazy iterators in Python 3.
- Using zip when input lengths must be validated and may differ.
- Using reduce where sum, any, all, or a loop is clearer.

Lab / Practice: Transform a list of product dictionaries into display names using both map and a comprehension. Explain which version your team should keep and why.

P22 Comprehensions

Learning Objectives

- Write list, dict, and set comprehensions.
- Use conditional clauses in comprehensions responsibly.
- Replace simple loops with clear comprehensions.
- Avoid dense comprehensions for business-heavy logic.

Explanation

Comprehensions is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding concise collection construction helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a comprehension builds a new collection by iterating over an input and optionally filtering or transforming items. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is packing too much logic into one line and making errors hard to locate, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo developers often build domains, value lists, summaries, and validation messages from records or imported rows. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. A comprehension is good when it reads like a sentence; otherwise use a loop. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- List comprehensions build lists.
- Set comprehensions build unique sets.
- Dict comprehensions build key-value mappings.
- Filters appear after the for clause with if.
- Nested comprehensions should be used sparingly.
- Do not use comprehensions only for side effects.

Build validation messages

The comprehension cleanly maps invalid rows into error messages. If validation had multiple rules and side effects, a normal loop would be easier to maintain.

```
rows = [
    {"line": 1, "name": "A", "price": 10},
    {"line": 2, "name": "", "price": 5},
]

errors = [f"Line {row['line']}: missing name" for row in rows if not row["name"]]
print(errors)
```

Create a lookup dictionary

The dict comprehension creates a direct lookup table from a list of records. This is a common pattern before matching imported rows to existing data.

```
products = [
    {"default_code": "A001", "name": "Desk"},
    {"default_code": "A002", "name": "Chair"},
]

by_code = {product["default_code"]: product for product in products}
print(by_code["A001"]["name"])
```

Common Mistakes

- Using comprehensions for print, write, or other side effects.
- Writing nested comprehensions that require careful mental parsing.
- Forgetting duplicate keys overwrite earlier values in dict comprehensions.
- Hiding validation rules inside a long conditional expression.

Lab / Practice: Given a list of order dictionaries, create a list of confirmed order names, a set of customer names, and a dictionary mapping order name to total.

P23 Decorators

Learning Objectives

- Explain decorators as functions that wrap or modify callables.
- Write a simple decorator with `functools.wraps`.
- Understand why decorators affect call behavior.
- Relate Python decorators to Odoo api decorators carefully.

Explanation

Decorators is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding callable wrapping and metadata-preserving behavior helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a decorator receives a function and returns a replacement callable, often adding work before or after the original call. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is using decorators without understanding that they change the callable being executed, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo uses decorators such as `api.depends`, `api.constrains`, and `api.model` to tell the framework how methods participate in ORM behavior. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Read decorator documentation before changing method signatures or return expectations. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- The `@decorator` syntax is assignment sugar around function wrapping.
- `functools.wraps` preserves name, docstring, and metadata.
- Decorators can add logging, validation, caching, or framework registration.
- The wrapper should accept compatible arguments.
- Odoo decorators are framework contracts, not merely style markers.
- Decorator order matters when multiple decorators are stacked.

Simple timing decorator

The decorator adds timing around the original function while returning its result. `wraps` keeps debugging and documentation output tied to the original function name.

```
from functools import wraps
from time import perf_counter

def timed(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            elapsed = perf_counter() - start
            print(f"{func.__name__} took {elapsed:.4f}s")
    return wrapper

@timed
def compute_total(amounts):
    return sum(amounts)

print(compute_total([10, 20, 30]))
```

Common Mistakes

- Forgetting to return the original function's result from the wrapper.
- Not using `functools.wraps` and losing useful metadata.
- Changing arguments in a wrapper without a clear contract.
- Copying Odoo decorators without understanding what the ORM expects.

Lab / Practice: Write a decorator that logs function arguments and return value for a pure calculation function. Then explain why the same decorator might be unsafe around sensitive production data.

P24 Error Handling

Learning Objectives

- Raise and catch exceptions intentionally.
- Use try, except, else, and finally appropriately.
- Design error messages for operators and users.
- Avoid swallowing framework errors.

Explanation

Error Handling is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding explicit handling of exceptional control flow helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is exceptions interrupt normal execution and travel up the call stack until code handles them or the program reports failure. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is catching broad exceptions and hiding the actual cause of data or transaction problems, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo translates many exceptions into user-visible errors and rolls back failed transactions. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Catch only errors you can handle locally, and let unexpected errors remain visible. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- raise creates an exception.
- except handles matching exception types.
- else runs when no exception occurred.
- finally runs whether success or failure occurred.
- Specific exception types are safer than bare except.
- Logging should preserve traceback information for unexpected failures.

Validate input with a specific exception

The function catches only conversion errors and raises a clearer domain message while preserving the original cause. Validation after conversion keeps separate failure modes understandable.

```
def parse_positive_int(text):
    try:
        value = int(text)
    except ValueError as exc:
        raise ValueError(f"expected a whole number, got {text!r}") from exc
    if value <= 0:
        raise ValueError("value must be positive")
    return value

print(parse_positive_int("12"))
```

Always close local resources

finally ensures cleanup even if writing fails. In real code, a with statement is usually better, but finally explains the control-flow guarantee.

```
path = "/tmp/python-error-lab.txt"
handle = open(path, "w")
try:
    handle.write("hello\n")
finally:
    handle.close()
```

Common Mistakes

- Using bare except and hiding KeyboardInterrupt or system exit.
- Catching an error only to print it and continue with corrupt state.
- Raising generic Exception when a specific type would be clearer.
- Discarding the original exception cause during re-raise.

Lab / Practice: Write a parser for imported quantities that reports line numbers on invalid input. Catch only expected conversion errors and collect all row errors for display.

P25 Generators

Learning Objectives

- Define generator functions with yield.
- Explain lazy iteration and one-pass consumption.
- Use generators for streaming large data.
- Avoid generator surprises in debugging and reuse.

Explanation

Generators is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding lazy production of values helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a generator function pauses at yield and resumes later, producing values only as the caller requests them. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is assuming a generator is a reusable list and accidentally consuming it once, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, large exports, imports, and reports can benefit from streaming rather than loading everything into memory. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Use generators when memory or incremental processing matters, and materialize them only at the boundary that needs a list. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- yield turns a function into a generator function.
- Generators are iterators and are consumed as they are iterated.
- Lazy evaluation can reduce memory use.
- list(generator) materializes all values and consumes the generator.
- Generator expressions are concise lazy expressions.
- Debugging generators often requires careful inspection because values appear over time.

Stream cleaned rows

The generator yields only valid cleaned rows and skips empty names without building an intermediate list. This pattern scales well for import pipelines.

```
def cleaned_rows(raw_rows):
    for row in raw_rows:
        name = row.get("name", "").strip()
        if not name:
            continue
        yield {"name": name, "price": float(row.get("price", 0))}

rows = [{"name": " Desk ", "price": "10.5"}, {"name": "", "price": "2"}]
for row in cleaned_rows(rows):
    print(row)
```

Common Mistakes

- Iterating a generator once and expecting values to remain for a second pass.
- Materializing a huge generator too early with list.
- Putting hidden side effects inside a generator and forgetting when they execute.

- Returning a list when the caller expects lazy behavior.

Lab / Practice: Write a generator that yields valid invoice rows from raw dictionaries. Demonstrate that iterating it twice without recreating it produces values only the first time.

P26 Modules, Packages, Imports, and `__name__`

Learning Objectives

- Organize code into modules and packages.
- Use import forms responsibly.
- Explain how Python finds modules.
- Use `__name__ == '__main__'` for script entry points.

Explanation

Modules, Packages, Imports, and `__name__` is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding code organization and import execution helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a module is executed once when imported, names become attributes, and packages group modules with importable paths. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is putting side effects at import time and surprising tests, scripts, or Odoo module loading, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo addons are Python packages whose `__init__.py` files import models, controllers, and other framework registrations. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Keep imports cheap and predictable, and put script-only behavior behind a main guard. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- A `.py` file is a module; a package is a directory Python can import.
- `import module` keeps names qualified; `from module import name` brings a name directly into scope.
- Python searches `sys.path` to find modules.
- Module top-level code runs at import time.
- `__name__` is `'__main__'` when a file is run as a script.
- Circular imports usually indicate misplaced responsibilities.

Use a main guard

The main guard prevents script behavior from running when the module is imported elsewhere. This is essential for reusable utilities and safe tests.

```
def main():
    print("run as a script")

if __name__ == "__main__":
    main()
```

Prefer qualified imports for clarity

The qualified `pathlib.Path` call shows where the name comes from. This reduces ambiguity in modules with many imports.

```
import pathlib

config_path = pathlib.Path("/etc/odoo/odoo.conf")
print(config_path.exists())
```

Common Mistakes

- Executing database or network work at import time.
- Using wildcard imports and hiding where names come from.
- Creating circular imports between modules.
- Forgetting to import a new Odoo models file in the package `__init__.py`.

Lab / Practice: Create a two-file package with a helper module and a script module. Import the helper from the script and protect executable behavior with a main guard.

P27 pip, venv, and PyPI

Learning Objectives

- Create and activate a Python virtual environment.
- Install packages with pip into the intended environment.
- Explain PyPI as a package index.
- Manage dependencies safely for Odoo projects.

Explanation

pip, venv, and PyPI is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding isolated dependency management helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a virtual environment provides a separate interpreter context with its own installed packages. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is installing dependencies into the wrong environment or accepting unreviewed packages, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo deployments depend on exact Python packages for Odoo itself, custom addons, integrations, and tooling. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Record dependencies and reproduce environments instead of fixing imports manually on one server. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- venv creates an isolated environment directory.
- Activating a venv changes PATH so python and pip point into that environment.
- `python -m pip` ensures pip belongs to the selected interpreter.
- PyPI hosts public Python packages, but packages still require review.
- requirements files should be pinned or constrained for production.
- System Python packages and project packages should not be mixed casually.

Create and inspect a virtual environment

Using `python -m pip` ties package installation to the currently selected interpreter. Printing `sys.executable` confirms the shell is using the virtual environment.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -c "import sys; print(sys.executable)"
python -m pip list
```

Install from a requirements file

A requirements file makes dependencies explicit and repeatable. Production projects usually pin versions or use a lock process after security review.

```
cat > requirements.txt <<'REQ'  
requests  
REQ  
  
python -m pip install -r requirements.txt  
python -m pip show requests
```

Common Mistakes

- Running pip without checking which Python it targets.
- Installing project packages into the system Python on a production host.
- Depending on unpinned package versions for repeatable deployments.
- Adding packages without license, maintenance, or security review.

Lab / Practice: Create a virtual environment, install one small package, print the interpreter path, freeze dependencies, deactivate, and explain why the package is not necessarily available to system Python.

P28 Debugging and Python Habits for Odoo Developers

Learning Objectives

- Apply a disciplined debugging workflow.
- Use print, logging, pdb, and tests appropriately.
- Develop Python habits that fit Odoo's ORM and transaction model.
- Turn fixes into maintainable changes.

Explanation

Debugging and Python Habits for Odoo Developers is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding evidence-driven debugging and framework-aware habits helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is debugging is a loop of observing, forming a hypothesis, making a minimal test, and verifying the result. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is changing code until the symptom disappears without understanding the cause, especially when code runs inside a large framework with many inherited conventions.

In Odoo work, Odoo code runs inside transactions, environments, caches, access rules, and long-running worker processes. This means a small Python choice can affect database records, scheduled jobs, access rules, view rendering, and upgrade safety.

Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data, which errors are expected, and how the code will behave when a module is installed, upgraded, or called by many users.

The practice habit is to write a tiny experiment, inspect the result, then move the idea into production code only after the behavior is understood. Keep experiments small, keep production changes reversible, and write the regression test when the bug is understood. is the kind of repetition that makes Python reliable under real Odoo constraints.

Key Points

- Start with the exact error, input, user, company, and database state.
- Use logging for server code where print may be hidden or noisy.
- pdb is useful locally but dangerous if left in server code.
- Respect recordsets: methods may receive zero, one, or many records.
- Do not bypass the ORM unless you understand transactions, security, and cache invalidation.
- Readable, boring Python is a strength in business systems.

Use logging instead of print in server code

Logging keeps diagnostic output integrated with server log configuration and preserves structured context. The message uses lazy formatting so values are formatted only when the log level needs them.

```
import logging

_logger = logging.getLogger(__name__)

def confirm_order(order_name, amount_total):
    _logger.info("confirming order %s amount=%s", order_name, amount_total)
    if amount_total <= 0:
        raise ValueError("amount_total must be positive")
    return True
```

Small reproduction before framework change

A small pure function lets the team test the rule outside Odoo before connecting it to models or views. This reduces the number of moving parts during debugging.

```
def normalize_code(value):  
    return value.strip().upper().replace(" ", "_")  
  
assert normalize_code(" sale order ") == "SALE_ORDER"  
assert normalize_code("crm") == "CRM"  
print("normalization rules understood")
```

Common Mistakes

- Leaving breakpoint calls in code that may run on a server.
- Debugging as administrator only and missing access-rule failures.
- Calling `ensure_one` reflexively instead of designing for recordsets.
- Fixing a symptom without adding a regression test or prevention note.

Lab / Practice: Take a failing Python function, write down the expected input and output, add two focused assertions, fix the function, then describe how the same habit applies before changing an Odoo model method.

Part D PostgreSQL Database Administration

This part contains 16 detailed lessons. Work in order. Complete each lab before moving forward when the lab unlocks the next dependency.

Presentation: Companion deck: see `training_materials/presentations/` for the PostgreSQL Database Administration slides.

DB01 Install PostgreSQL on Ubuntu

Learning Objectives

- Install PostgreSQL packages on Ubuntu in a repeatable way.
- Understand the system service, default cluster, and postgres OS user.
- Verify readiness before connecting Odoo to the server.

Explanation

A clean PostgreSQL installation starts with knowing which package source you trust. Ubuntu ships PostgreSQL packages that are stable and easy to patch, while the upstream PostgreSQL APT repository gives access to newer major versions sooner.

On Ubuntu, PostgreSQL is managed as a systemd service and normally creates a default cluster during installation. The cluster is a data directory plus configuration files, not just a running process.

The postgres Linux user is intentionally separate from your application user. It owns the database files and can connect locally as the postgres database superuser through peer authentication.

For Odoo training and development, keep the database server local unless there is a clear operational reason to separate it. Localhost reduces network complexity and makes pg_hba.conf easier to reason about.

After installation, verify the service state, listening port, and psql connection before installing Odoo. Debugging Odoo connection errors is much faster when the database layer is already proven healthy.

Production installations should also consider disk layout, backups, monitoring, and upgrade policy before real data is loaded. PostgreSQL is easy to install, but moving a busy Odoo database later is not always simple.

Key Points

- Use apt for predictable installs and security updates.
- The default service is usually postgresql, with one or more versioned clusters behind it.
- The postgres OS account is for database administration, not for running Odoo.
- Always verify the server with psql before blaming Odoo.

Ubuntu package installation

This installs PostgreSQL, starts the service, and proves that the local postgres administrator can execute SQL.

```
sudo apt update
sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable --now postgresql
systemctl status postgresql --no-pager
sudo -u postgres psql -c "SELECT version();"

```

Common Mistakes

- Installing PostgreSQL but never checking whether the service is actually running.
- Running Odoo as the postgres Linux user instead of creating a dedicated Odoo database role.
- Mixing Ubuntu packages and upstream packages without documenting the version source.
- Opening PostgreSQL to the network before authentication rules and firewall rules are understood.

Lab / Practice: Install PostgreSQL on a fresh Ubuntu machine, confirm the service is active, and capture the output of `SELECT version()`.

DB02 psql CLI, Users, Roles, and Grants

Learning Objectives

- Connect with psql using local and password authentication.
- Create roles suitable for Odoo development.
- Grant only the privileges an application role needs.

Explanation

psql is the fastest diagnostic tool for PostgreSQL because it removes the application layer from the problem. A developer who can inspect roles, databases, schemas, and locks from psql can solve most Odoo database issues directly.

PostgreSQL uses roles for both users and groups. A role can log in when it has LOGIN, and it can own objects, inherit privileges, or be granted to other roles depending on how you design access.

Odoo usually connects with one role that owns or can create its databases in development. In production, many teams restrict the role to a specific database and avoid broad CREATEDB permissions after deployment.

Grants are scoped to databases, schemas, tables, sequences, and functions. Granting a database privilege does not automatically give table privileges inside every schema.

Password authentication requires both a password on the role and a pg_hba.conf rule that permits password-based access. When either side is missing, Odoo errors often look like a generic authentication failure.

Use psql meta commands such as \du, \l, \dn, and \dt to inspect the server quickly. These commands are not SQL, but they are invaluable during support and training.

Key Points

- A PostgreSQL role is the central identity and privilege unit.
- LOGIN makes a role usable as a user account.
- CREATEDB is convenient for Odoo development but should be reviewed for production.
- psql meta commands are part of a practical database debugging workflow.

Create an Odoo database role

This creates a login role that can create databases, then confirms that password authentication works over TCP.

```
sudo -u postgres createuser --createdb --pwprompt odoo17
psql -h 127.0.0.1 -U odoo17 -d postgres

-- Inside psql
\conninfo
\du
SELECT current_user;
```

Grant schema privileges

The example separates ownership from application access, which is closer to how production systems are often controlled.

```
CREATE DATABASE training_owner OWNER postgres;
CREATE ROLE training_app LOGIN PASSWORD 'change-me';
\c training_owner
GRANT USAGE, CREATE ON SCHEMA public TO training_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO training_app;
```

Common Mistakes

- Assuming a database grant automatically grants privileges on every table.
- Forgetting that peer authentication uses the Linux user name, not the typed database password.
- Giving SUPERUSER to application roles to avoid learning the real permission issue.
- Leaving training passwords in shell history or shared notes.

Lab / Practice: Create a role named `odoo17_lab` with `CREATEDB`, connect with `psql` over `127.0.0.1`, and list all roles with `\du`.

DB03 Create and Manage Databases

Learning Objectives

- Create databases with the correct owner and encoding.
- Understand templates, connection limits, and database ownership.
- Prepare databases for Odoo import and development workflows.

Explanation

A PostgreSQL database is an isolation boundary for Odoo data. Each Odoo database contains its own registry metadata, business records, installed modules, and attachments references.

The database owner matters because Odoo creates and updates many database objects during module installation and upgrades. If the role lacks ownership or enough privileges, upgrade failures can appear deep inside ORM operations.

UTF8 encoding is the normal choice for Odoo. Training environments should make this explicit so later collation, translation, and text search behavior are easier to explain.

Template databases allow PostgreSQL to copy an existing database structure at creation time. For ordinary Odoo work, template0 or the default template1 is enough, but polluted templates can introduce surprising extensions or objects.

Connection limits can protect a shared database server from one workload consuming every backend process. For local training, low limits are annoying, but for production they can prevent cascading failure.

Dropping an Odoo database is destructive and should be treated like deleting a customer environment. Always confirm you have the right database name and a tested backup before destructive maintenance.

Key Points

- One Odoo database maps to one Odoo instance dataset.
- Use explicit owners for predictable module installation and upgrades.
- UTF8 is expected for modern Odoo deployments.
- Do not drop or rename databases casually in shared environments.

Create databases from SQL and CLI

The CLI is concise for daily work, while SQL makes ownership, encoding, template, and limits explicit.

```
createdb -h 127.0.0.1 -U odoo17 --encoding=UTF8 odoo17_training

psql -U postgres -d postgres -c "
CREATE DATABASE odoo17_reporting
    WITH OWNER odoo17
    ENCODING 'UTF8'
    TEMPLATE template0
    CONNECTION LIMIT 50;
"
```

Common Mistakes

- Creating an Odoo database owned by postgres and then running Odoo as a different restricted role.
- Using mixed encodings or locale settings across environments without documenting them.
- Dropping a database from the wrong terminal session.
- Copying a database while users or cron jobs are still writing to it.

Lab / Practice: Create two training databases owned by your Odoo role, list them with \l, and remove one after confirming its owner.

DB04 PostgreSQL Data Types for Odoo Developers

Learning Objectives

- Recognize PostgreSQL types commonly produced by Odoo fields.
- Choose SQL types for reporting tables and integration staging tables.
- Avoid type choices that damage money, date, or JSON accuracy.

Explanation

Odoo hides most column definitions behind its ORM, but PostgreSQL types still affect performance, correctness, and reporting. Senior Odoo developers should be able to inspect a table and understand what the ORM generated.

Text-like fields commonly map to `varchar` or `text`. The practical distinction is less about storage and more about validation, indexing, and how the Odoo field communicates intent to users.

Numeric choices matter for money and quantities. Use numeric for exact decimal values when precision matters, and avoid float for values that must reconcile financially.

Date and timestamp columns support business workflows such as accounting periods, scheduled actions, and SLA calculations. Odoo stores datetimes in UTC and converts them for users at the application layer.

JSONB is useful for flexible integration payloads, analytic properties, and denormalized metadata. It is powerful, but it should not become a hiding place for core relational data that needs constraints and joins.

Arrays, UUIDs, and specialized types can be useful in custom SQL tables, but they should be introduced deliberately. Odoo's ORM works best when custom models use standard field types and clear relationships.

Key Points

- Understand the SQL type behind each important Odoo field.
- Use numeric for exact business decimals.
- Datetimes are stored in UTC by Odoo.
- JSONB is useful, but relational data still belongs in relations.

Inspect generated Odoo column types

The query teaches developers to inspect real Odoo tables, while the staging table shows conservative type choices for integrations.

```
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'res_partner'
ORDER BY ordinal_position;

CREATE TABLE integration_payload (
    id bigserial PRIMARY KEY,
    external_id varchar NOT NULL,
    amount numeric(16, 2) NOT NULL,
    received_at timestamp without time zone NOT NULL,
    payload jsonb NOT NULL
);
```

Common Mistakes

- Using float for money because it looks simpler in test data.
- Storing dates as text and then losing reliable filtering and sorting.
- Putting frequently joined business identifiers only inside JSONB.
- Assuming Odoo user timezone conversion changes how datetimes are stored in PostgreSQL.

Lab / Practice: Inspect five columns from `res_partner` or another Odoo table and map each SQL type back to the likely Odoo field type.

DB05 DDL: CREATE, ALTER, and DROP

Learning Objectives

- Use DDL safely in development and maintenance tasks.
- Understand how DDL locks affect live Odoo systems.
- Plan schema changes with reversibility and backups in mind.

Explanation

Data Definition Language changes the structure of the database. In Odoo, most application schema changes should come from module code and migrations, not from manual SQL typed into production.

CREATE statements add tables, indexes, schemas, views, and functions. They are safe when planned, but object names should follow clear conventions so DBAs and developers can identify ownership.

ALTER statements can be cheap or expensive depending on the operation and PostgreSQL version. Adding a nullable column is usually fast, while changing a type or setting a not-null constraint may scan or rewrite data.

DROP statements are final from the perspective of the current transaction once committed. A backup is not optional when dropping objects that may contain customer or audit data.

DDL takes locks, and those locks can block Odoo workers. Even short DDL can create visible downtime if it waits behind a long transaction and then blocks new writes.

For custom Odoo modules, prefer declarative model fields and migration scripts that can be reviewed. Manual DDL is best reserved for diagnostics, reporting objects, and carefully controlled maintenance.

Key Points

- Let Odoo module definitions manage normal model tables.
- DDL can block live application traffic.
- Review ALTER operations for rewrite or scan behavior.
- Treat DROP as a backup-required operation.

DDL for a reporting helper table

This illustrates common DDL statements in a non-Odoo schema where manual SQL may be appropriate.

```
CREATE SCHEMA IF NOT EXISTS reporting;

CREATE TABLE reporting.daily_sales_snapshot (
    snapshot_date date PRIMARY KEY,
    order_count integer NOT NULL DEFAULT 0,
    untaxed_total numeric(16, 2) NOT NULL DEFAULT 0
);

ALTER TABLE reporting.daily_sales_snapshot
    ADD COLUMN company_id integer;

DROP TABLE reporting.daily_sales_snapshot;
```

Common Mistakes

- Manually altering Odoo-owned tables and then being surprised when module upgrades disagree.
- Running ALTER TABLE during business hours without checking locks and table size.
- Dropping a column before confirming whether computed fields, reports, or integrations use it.
- Leaving experimental tables in public schema with unclear ownership.

Lab / Practice: Create a reporting schema and table, alter it with one new column, inspect it with \d, then drop it after confirming the object name.

DB06 Primary Keys, Foreign Keys, and Constraints

Learning Objectives

- Explain how constraints protect relational integrity.
- Use primary keys, foreign keys, unique constraints, and checks.
- Connect database constraints to Odoo model design.

Explanation

Constraints are the database's way of refusing impossible data. They remain active regardless of whether data is written by Odoo, an import script, psql, or an integration job.

A primary key uniquely identifies each row and is the target for many relationships. Odoo models normally use an integer id primary key generated by the database.

Foreign keys preserve references between tables. In Odoo, Many2one fields often create foreign keys unless the field definition or module history prevents it.

Unique constraints prevent duplicates that application checks can miss under concurrency. For example, two users creating the same external reference at the same time should be stopped by the database, not only by Python code.

Check constraints enforce local row rules such as positive quantities or valid ranges. They are simple, fast, and useful for invariants that should never be bypassed.

Good Odoo modules combine ORM validations with SQL constraints. Python constraints give friendly messages and cross-record logic, while SQL constraints provide final protection against race conditions.

Key Points

- Constraints are stronger than application-only validation.
- Unique constraints are important under concurrent writes.
- Foreign keys keep relational references honest.
- Use SQL constraints for invariants that must never be violated.

Relational constraints

The schema protects identity, references, positive amounts, and invoice uniqueness even if data is loaded outside Odoo.

```
CREATE TABLE training_customer (  
    id bigserial PRIMARY KEY,  
    email varchar NOT NULL UNIQUE,  
    credit_limit numeric(16, 2) NOT NULL DEFAULT 0,  
    CHECK (credit_limit >= 0)  
);  
  
CREATE TABLE training_invoice (  
    id bigserial PRIMARY KEY,  
    customer_id bigint NOT NULL REFERENCES training_customer(id) ON DELETE RESTRICT,  
    number varchar NOT NULL,  
    total numeric(16, 2) NOT NULL CHECK (total >= 0),  
    UNIQUE (customer_id, number)  
);
```

Common Mistakes

- Relying only on Python validation for uniqueness in concurrent workflows.
- Using ON DELETE CASCADE without considering accounting and audit requirements.
- Adding constraints to dirty data without first finding violations.
- Creating nullable foreign keys when the business relationship is actually required.

Lab / Practice: Build two related tables with a foreign key and a unique constraint, then attempt inserts that prove each constraint works.

DB07 CRUD: INSERT, SELECT, UPDATE, and DELETE

Learning Objectives

- Write basic CRUD SQL confidently.
- Understand transactions and RETURNING clauses.
- Relate direct SQL changes to Odoo ORM safety rules.

Explanation

CRUD operations are the foundation of every database-backed application. Even when Odoo's ORM writes SQL for you, understanding the underlying statements helps you debug logs and performance plans.

INSERT creates rows and can return generated values such as id. RETURNING is especially useful in scripts because it avoids a second query to discover what was inserted.

SELECT retrieves data and should be shaped intentionally. Pulling every column from a large Odoo table is rarely the right diagnostic habit when you only need ids, names, and a status field.

UPDATE changes existing rows and is dangerous without a precise WHERE clause. In Odoo databases, direct UPDATE can bypass computed fields, mail tracking, access rules, onchange logic, and business validations.

DELETE removes rows and should be used with even more caution. Many Odoo records should be archived with active = false rather than deleted, especially if they are referenced by accounting, inventory, or chatter history.

Transactions let several statements succeed or fail together. In psycopg, BEGIN, COMMIT, and ROLLBACK are essential safety tools when testing data changes.

Key Points

- Always write SELECT first to confirm the target rows before UPDATE or DELETE.
- RETURNING is useful for inserts and updates.
- Direct SQL bypasses Odoo ORM behavior.
- Use transactions for safe experimentation.

Safe CRUD workflow

The transaction proves the statements and then rolls them back, which is a safe habit when learning or diagnosing.

```
BEGIN;  
  
INSERT INTO training_customer (email, credit_limit)  
VALUES ('ada@example.com', 5000.00)  
RETURNING id, email;  
  
SELECT id, email, credit_limit  
FROM training_customer  
WHERE email = 'ada@example.com';  
  
UPDATE training_customer  
SET credit_limit = credit_limit + 1000  
WHERE email = 'ada@example.com'  
RETURNING id, credit_limit;  
  
ROLLBACK;
```

Common Mistakes

- Running UPDATE or DELETE without a WHERE clause.
- Changing Odoo data directly and expecting ORM computed fields or tracking to run.
- Using SELECT * on large tables during production incidents.
- Forgetting to COMMIT a successful transaction or ROLLBACK a test transaction.

Lab / Practice: Insert a row in a transaction, update it with RETURNING, verify the result, and roll back so the table returns to its original state.

DB08 Filtering with WHERE, LIKE, ILIKE, IN, and BETWEEN

Learning Objectives

- Use common filtering operators precisely.
- Understand case-sensitive and case-insensitive pattern matching.
- Avoid filters that defeat indexes or return unintended records.

Explanation

Filtering is where SQL becomes practical for support work. Good WHERE clauses narrow the problem to the exact company, date range, state, or external identifier you need to investigate.

LIKE is case-sensitive pattern matching in PostgreSQL, while ILIKE is case-insensitive. ILIKE is convenient for support searches but can be slower unless supported by the right index strategy.

IN is clearer than a long chain of OR comparisons when matching a known set of values. It is common in Odoo diagnostics because record ids are often copied from logs, URLs, or failed queue jobs.

BETWEEN includes both boundaries. That is useful for dates when you mean a closed range, but for datetimes many teams prefer `>= start` and `< next_start` to avoid edge mistakes.

NULL requires IS NULL or IS NOT NULL, not equals comparisons. This distinction matters in Odoo because optional Many2one fields are stored as NULL when unset.

Filters should be written with indexes in mind when tables are large. Wrapping indexed columns in functions or starting text patterns with a wildcard can force PostgreSQL to inspect far more rows.

Key Points

- LIKE is case-sensitive; ILIKE is case-insensitive.
- BETWEEN includes both ends of the range.
- Use IS NULL for missing optional values.
- Index-friendly filters matter on large Odoo tables.

Odoo-style support filters

The examples use practical filters for partners and orders while avoiding an ambiguous datetime BETWEEN boundary.

```
SELECT id, name, email
FROM res_partner
WHERE active IS TRUE
      AND email ILIKE '%@example.com'
      AND company_id IN (1, 3, 7)
ORDER BY name
LIMIT 50;

SELECT id, name, date_order
FROM sale_order
WHERE date_order >= timestamp '2026-01-01'
      AND date_order < timestamp '2026-02-01'
      AND state IN ('sale', 'done');
```

Common Mistakes

- Expecting LIKE to be case-insensitive.
- Using BETWEEN with datetimes and accidentally including or excluding boundary records.
- Writing column = NULL instead of column IS NULL.
- Starting every text search with % and then wondering why indexes are not used.

Lab / Practice: Write three SELECT queries against an Odoo database: one with ILIKE, one with IN, and one date range using `>=` and `<`.

DB09 Joins for Relational Odoo Data

Learning Objectives

- Choose INNER JOIN and LEFT JOIN correctly.
- Trace Odoo Many2one and Many2many relationships in SQL.
- Read joined results without multiplying rows accidentally.

Explanation

Joins combine rows from related tables, which is essential in Odoo because business data is highly relational. A sale order links to partners, companies, currencies, users, order lines, stock moves, invoices, and more.

INNER JOIN returns rows only when both sides match. It is appropriate when the relationship is required and missing related data would indicate invalid or irrelevant records.

LEFT JOIN keeps the left-side row even when the related row is missing. This is useful for optional fields, incomplete setup, and diagnostics where missing data is part of the question.

Many2one relationships are usually simple joins from a foreign key column to the related table's id. Many2many relationships use an intermediate relation table that contains pairs of ids.

One-to-many joins can multiply rows because each parent may have many children. That is expected, but it becomes a bug when totals are summed after joining to another one-to-many table without pre-aggregation.

When debugging Odoo reports, inspect the join path carefully before blaming the ORM. Many incorrect totals come from SQL joins that accidentally duplicate business rows.

Key Points

- INNER JOIN filters to matching rows.
- LEFT JOIN preserves unmatched left-side rows.
- Many2many joins require a relation table.
- One-to-many joins can duplicate parent values in result sets.

Join sale orders to customers and tags

The first query follows Many2one fields, and the second shows the relation table pattern behind a partner tag Many2many.

```
SELECT so.id, so.name, rp.name AS customer, rc.name AS company
FROM sale_order so
JOIN res_partner rp ON rp.id = so.partner_id
LEFT JOIN res_company rc ON rc.id = so.company_id
WHERE so.state IN ('sale', 'done')
ORDER BY so.date_order DESC
LIMIT 20;
```

```
SELECT rp.name, category.name AS tag
FROM res_partner rp
JOIN res_partner_res_partner_category_rel rel
  ON rel.partner_id = rp.id
JOIN res_partner_category category
  ON category.id = rel.category_id
WHERE rp.id = 42;
```

Common Mistakes

- Using INNER JOIN where optional data requires LEFT JOIN.
- Summing parent totals after joining multiple child tables.
- Guessing Many2many relation table names instead of inspecting the model or database.
- Joining on display names instead of stable ids.

Lab / Practice: Join sale_order to res_partner, then write a separate query that follows one Many2many relation table in your database.

DB10 GROUP BY, HAVING, and Aggregates

Learning Objectives

- Summarize Odoo data with aggregate functions.
- Use GROUP BY and HAVING in the correct order.
- Avoid aggregation mistakes caused by joins and NULL values.

Explanation

Aggregates turn detail records into business signals such as totals, counts, minimums, maximums, and averages. Odoo dashboards and reports often reduce to the same concepts expressed through ORM `read_group` or SQL.

GROUP BY defines the grain of the result. If you group sales by company and month, every selected non-aggregate column must belong to that grain.

WHERE filters rows before grouping, while HAVING filters groups after aggregation. This difference is important when you want only customers whose total confirmed sales exceed a threshold.

COUNT(*) counts rows, while COUNT(column) counts non-null values. In optional Odoo fields, the difference can change your conclusion during data quality checks.

Aggregating after joins requires care. If a sale order is joined to both order lines and invoices, totals may be multiplied unless one side is aggregated first.

For application code, prefer Odoo's `read_group` when you need ORM security, context, and computed grouping behavior. Use SQL for diagnostics, migrations, and carefully reviewed reporting queries.

Key Points

- GROUP BY sets the result grain.
- HAVING filters aggregate results.
- COUNT(*) and COUNT(column) answer different questions.
- Pre-aggregate child tables before joining multiple one-to-many relationships.

Monthly sales summary

The query filters confirmed orders first, groups by month and company, then keeps only meaningful sales groups.

```
SELECT
    date_trunc('month', so.date_order)::date AS month,
    so.company_id,
    COUNT(*) AS order_count,
    SUM(so.amount_total) AS total_amount
FROM sale_order so
WHERE so.state IN ('sale', 'done')
GROUP BY 1, 2
HAVING SUM(so.amount_total) > 10000
ORDER BY month DESC, total_amount DESC;
```

Common Mistakes

- Selecting columns that are not part of GROUP BY and not aggregated.
- Using HAVING for simple row filters that belong in WHERE.
- Forgetting that COUNT(column) ignores NULL.
- Joining detail tables before aggregation and inflating totals.

Lab / Practice: Create a grouped sales or invoice query with COUNT and SUM, then add a HAVING clause that filters low-value groups.

DB11 Indexes: B-Tree, GIN, and Composite Design

Learning Objectives

- Explain how indexes speed up reads and slow down writes.
- Choose B-Tree, GIN, and composite indexes for common cases.
- Design indexes around Odoo domains and reporting queries.

Explanation

Indexes are lookup structures that help PostgreSQL find rows without scanning an entire table. They are essential for large Odoo installations because tables such as `account_move_line`, `mail_message`, and `stock_move` can grow quickly.

B-Tree indexes are the default and support equality, range comparisons, ordering, and many common joins. They are usually the right first choice for ids, foreign keys, dates, states, and unique constraints.

Composite indexes include more than one column and should match real filter patterns. An index on (`company_id`, `state`, `date_order`) can help a domain that filters company and state before sorting or ranging by date.

GIN indexes are useful for containment and full-text style lookups, especially with `jsonb`, arrays, and trigram search when the extension is enabled. They are heavier than B-Tree indexes, so use them when the query pattern justifies the cost.

Every index has a write cost because PostgreSQL must maintain it during `INSERT`, `UPDATE`, and `DELETE`. Over-indexing an Odoo table can slow imports, stock validation, accounting posting, and module upgrades.

Good index work starts from a slow query or a known high-volume domain, not from guesswork. Use `EXPLAIN ANALYZE` and production-like data before adding indexes that will live for years.

Key Points

- B-Tree is the default index type for most relational filters.
- Composite index column order should match query patterns.
- GIN is useful for `jsonb`, arrays, and specialized text search.
- Indexes speed reads but add write and storage cost.

Indexes for Odoo query patterns

The examples show B-Tree indexes for common domains and a GIN index for JSONB containment queries.

```
CREATE INDEX sale_order_company_state_date_idx
ON sale_order (company_id, state, date_order DESC);

CREATE INDEX account_move_line_partner_date_idx
ON account_move_line (partner_id, date);

CREATE INDEX integration_payload_payload_gin_idx
ON integration_payload USING gin (payload);

EXPLAIN ANALYZE
SELECT id, name
FROM sale_order
WHERE company_id = 1
  AND state IN ('sale', 'done')
ORDER BY date_order DESC
LIMIT 50;
```

Common Mistakes

- Adding indexes without measuring the query before and after.
- Creating many overlapping indexes on busy Odoo tables.
- Putting composite index columns in an order that does not match real filters.
- Expecting a normal B-Tree index to make leading-wildcard `ILIKE` searches fast.

Lab / Practice: Choose one slow-looking domain from Odoo logs, write the SQL equivalent, run EXPLAIN ANALYZE, add an index in a test database, and compare the plan.

DB12 Reading EXPLAIN ANALYZE

Learning Objectives

- Use EXPLAIN ANALYZE to inspect real query execution.
- Interpret scans, joins, row estimates, timing, and buffers.
- Turn query plans into practical optimization decisions.

Explanation

EXPLAIN shows PostgreSQL's planned execution strategy, while EXPLAIN ANALYZE runs the query and reports what actually happened. The ANALYZE form is more truthful but must be used carefully for writes because it executes them.

Plans are trees, and each node contributes work to its parent. Sequential scans, index scans, bitmap scans, nested loops, hash joins, and sorts all tell you how the optimizer chose to access and combine data.

Estimated rows versus actual rows is one of the most important signals. Large differences can mean stale statistics, correlated filters, missing indexes, or a query shape the planner cannot estimate well.

Timing tells you where time was spent, but buffers often explain why. A plan that reads many shared buffers may be doing heavy logical I/O even if the current run is warmed by cache.

An index scan is not automatically good, and a sequential scan is not automatically bad. Scanning a small table or a large percentage of a table can be cheaper than bouncing through an index.

For Odoo, connect slow SQL logs to EXPLAIN work. The best optimizations usually come from understanding a repeated ORM domain, adding a targeted index, reducing selected columns, or avoiding an accidental join explosion.

Key Points

- EXPLAIN ANALYZE executes the query.
- Compare estimated rows to actual rows.
- Look at buffers as well as timing.
- Sequential scans can be correct for small or broad queries.

Analyze a read query safely

The first command reports actual execution details, and the second refreshes statistics if estimates look stale.

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT id, name, date_order
FROM sale_order
WHERE company_id = 1
      AND state = 'sale'
ORDER BY date_order DESC
LIMIT 20;

ANALYZE sale_order;
```

Common Mistakes

- Running EXPLAIN ANALYZE on UPDATE or DELETE in production without wrapping and rolling back.
- Assuming the largest cost number is always the real bottleneck.
- Ignoring row estimate errors.
- Adding an index because a sequential scan appears, even when the query returns most rows.

Lab / Practice: Run EXPLAIN (ANALYZE, BUFFERS) for a filtered query before and after ANALYZE, then document the scan type and row estimates.

DB13 Backups with pg_dump

Learning Objectives

- Create logical backups with pg_dump.
- Choose custom, directory, and plain SQL dump formats.
- Prepare Odoo databases for consistent backup routines.

Explanation

pg_dump creates a logical backup by reading database objects and data. It is portable across machines and useful for Odoo migrations, staging refreshes, and point-in-time training snapshots.

The custom format is usually the best default for Odoo because it is compressed, flexible, and restorable with pg_restore. Plain SQL is readable but less flexible for selective restore and parallel jobs.

A consistent backup must include the PostgreSQL database and Odoo filestore. The database stores attachment metadata, but the binary files may live on disk under the Odoo data directory.

Backups should be automated and verified, not performed only when someone remembers. A backup that has never been restored is an assumption, not a recovery plan.

Large databases need attention to runtime, disk space, compression, and impact on the database server. pg_dump is online, but it still consumes I/O and CPU that can affect users.

Security matters because dumps contain customer data, passwords hashes, tokens, messages, and accounting records. Encrypt backups at rest and control who can download them.

Key Points

- Use custom format for flexible pg_restore workflows.
- Back up the Odoo filestore with the database.
- Automate backups and test restores.
- Protect dumps as sensitive production data.

Custom-format Odoo backup

The database dump and filestore archive are both needed for a realistic Odoo recovery.

```
export DB=odoo17_prod
export BACKUP_DIR=/var/backups/odoo
sudo mkdir -p "$BACKUP_DIR"

pg_dump -h 127.0.0.1 -U odoo17 -F c -Z 6 -f "$BACKUP_DIR/${DB}_$(date +%F).dump" "$DB"

sudo tar -C /var/lib/odoo/.local/share/Odoo/filestore -czf "$BACKUP_DIR/${DB}_filestore_$(date +%F).tar.gz" "$DB"
```

Common Mistakes

- Backing up only PostgreSQL and forgetting the Odoo filestore.
- Using plain SQL dumps for every case and losing pg_restore flexibility.
- Keeping backups on the same disk as the database with no off-host copy.
- Never testing whether the backup can be restored.

Lab / Practice: Create a custom-format dump of a training database and record the dump size, duration, and exact command used.

DB14 Restores with pg_restore

Learning Objectives

- Restore custom-format dumps safely.
- Use clean, create, jobs, and selective restore options.
- Validate an Odoo database after restore.

Explanation

Restore practice is where backup confidence becomes real. A team that restores regularly can estimate recovery time, detect missing filestore data, and find permission problems before an emergency.

pg_restore works with custom and directory-format dumps. It can create objects, clean existing objects, run parallel jobs, and restore selected schemas or tables when the dump format supports it.

Restoring into a fresh database is usually safer than overwriting an existing one. It lets you validate the result before swapping names, updating configuration, or pointing Odoo at the restored database.

Ownership and role availability matter during restore. If the dump references owners that do not exist on the target server, you may need --no-owner or a prepared role setup.

For Odoo, restore the filestore directory with the exact database name expected by the instance. If the database is renamed, the filestore path must match or attachments will appear missing.

After restore, validate at multiple levels: psql connection, Odoo registry startup, module list, attachment access, scheduled actions, and a few business workflows. A database that restores without SQL errors may still be incomplete operationally.

Key Points

- Practice restore before a real incident.
- Prefer restoring into a fresh database for validation.
- Use --no-owner when target roles differ.
- Match the filestore directory name to the restored Odoo database.

Restore a custom dump to a test database

The restore targets a separate database and performs a simple Odoo metadata check afterward.

```
createdb -h 127.0.0.1 -U odoo17 odoo17_restore_test

pg_restore -h 127.0.0.1 -U odoo17 --dbname=odoo17_restore_test --no-owner --jobs=4 /var/backups/odoo17/odoo17_restore_test.dump

psql -h 127.0.0.1 -U odoo17 -d odoo17_restore_test -c "SELECT latest_version FROM ir_module_module WHERE name = 'base'"
```

Common Mistakes

- Restoring over the only available copy of a database.
- Ignoring missing target roles and ownership errors.
- Forgetting to restore or rename the filestore.
- Declaring success after pg_restore exits without checking Odoo startup.

Lab / Practice: Restore a dump into a new training database, start Odoo against it, and open one attachment to prove the filestore is present.

DB15 postgresql.conf and pg_hba.conf

Learning Objectives

- Locate and edit the main PostgreSQL configuration files.
- Configure listening addresses, memory settings, logging, and authentication.
- Reload or restart PostgreSQL safely after configuration changes.

Explanation

postgresql.conf controls server behavior such as listening addresses, memory, logging, autovacuum, and connection limits. pg_hba.conf controls who can connect, from where, to which database, and by which authentication method.

Configuration files are cluster-specific on Ubuntu. The common path includes the PostgreSQL major version, such as /etc/postgresql/15/main, so always confirm the active cluster before editing.

listen_addresses determines the network interfaces PostgreSQL listens on. Binding to localhost is safest for single-host Odoo; binding to all addresses requires firewalling and strong authentication rules.

pg_hba.conf is evaluated top to bottom, and the first matching rule wins. A permissive rule above a strict rule can silently weaken the configuration.

Many changes only need a reload, while others require a restart. Reloading is less disruptive, but you must know whether the setting is reloadable before assuming it took effect.

Odoo deployments benefit from logging slow queries and connection information during tuning. Do not enable extremely verbose logging permanently without checking disk growth and privacy implications.

Key Points

- postgresql.conf tunes server behavior.
- pg_hba.conf controls connection authentication.
- pg_hba.conf rule order matters.
- Reload when possible, restart when required.

Odoo-friendly local configuration snippets

The snippets keep PostgreSQL local to the application host and enable slow-query visibility for tuning.

```
-- Locate files
SHOW config_file;
SHOW hba_file;

-- postgresql.conf examples
listen_addresses = '127.0.0.1'
max_connections = 200
shared_buffers = 1GB
log_min_duration_statement = 500

-- pg_hba.conf example
# TYPE DATABASE USER ADDRESS METHOD
host all odoo17 127.0.0.1/32 scram-sha-256

-- Apply reloadable changes
SELECT pg_reload_conf();
```

Common Mistakes

- Editing the wrong versioned cluster directory.
- Adding a strict pg_hba.conf rule below a broader rule that already matches.
- Using trust authentication on shared or production machines.
- Restarting the service for every change without knowing whether reload is enough.

Lab / Practice: Find config_file and hba_file with SHOW commands, add a localhost rule for your Odoo role in a lab environment, reload, and reconnect.

DB16 Locks, Connections, Terminating Queries, and Odoo Backup Habits

Learning Objectives

- Inspect active connections and blocking locks.
- Terminate unsafe or stuck sessions carefully.
- Apply Odoo-specific database habits and backup strategy.

Explanation

Every PostgreSQL connection uses server resources, and every transaction can hold locks. Odoo workers, cron jobs, imports, reports, and shell sessions can all contribute to lock contention.

Locks are normal; blocking is the operational concern. A short row lock during a write is expected, while an idle transaction holding a table lock can stop module upgrades or business workflows.

`pg_stat_activity` shows what sessions are doing, how long they have been running, and whether they are waiting. Combined with `pg_locks`, it helps identify blockers rather than only victims.

Terminating a backend is a production action, not a reflex. Prefer canceling a query when possible, communicate with users when needed, and understand whether the application will retry safely.

Odoo-specific database habits include avoiding direct writes to business tables, stopping Odoo during risky restores, disabling cron during staging refreshes, and running upgrades from the application layer. Respect the ORM unless you are deliberately doing maintenance.

A strong Odoo backup strategy combines scheduled `pg_dump` or physical backups, filestore backups, off-host copies, retention, encryption, and restore drills. The strategy should define recovery point objective, recovery time objective, and who is allowed to perform recovery.

Key Points

- Use `pg_stat_activity` to understand sessions before acting.
- Find blockers, not just blocked queries.
- Cancel or terminate with intent and an audit trail.
- Odoo backups must include both database and filestore.
- Restore drills are part of the backup strategy.

Inspect and control active sessions

Inspect first, cancel when possible, and terminate only when the risk of leaving the session running is greater than killing it.

```
SELECT pid, username, datname, state, wait_event_type, wait_event,
       now() - query_start AS runtime,
       left(query, 120) AS query
FROM pg_stat_activity
WHERE datname = 'odoo17_prod'
ORDER BY runtime DESC NULLS LAST;

SELECT pg_cancel_backend(12345);
SELECT pg_terminate_backend(12345);
```

Simple Odoo backup policy outline

The outline connects backup creation to restore testing, which is the habit that actually proves recoverability.

```
Daily:
- pg_dump custom format for each production database
- tar and compress matching filestore directories
- upload encrypted artifacts to off-host storage

Weekly:
- restore latest backup to an isolated server
- run Odoo with --stop-after-init
- verify login, attachments, and key reports
```

Common Mistakes

- Killing sessions without checking whether they are blockers or victims.
- Leaving psql transactions open with state idle in transaction.
- Refreshing staging from production while Odoo cron jobs are active.
- Backing up databases and filestores on different schedules so they no longer match.
- Keeping no written RPO, RTO, retention, or restore procedure.

Lab / Practice: Open two psql sessions, create a blocking transaction in one, observe it from `pg_stat_activity` in the other, then cancel or roll back safely.

Part E Odoo Core Development (Models, Fields, ORM)

This part contains 30 detailed lessons. Work in order. Complete each lab before moving forward when the lab unlocks the next dependency.

Presentation: Companion deck: see `training_materials/presentations/` for the Odoo Core Development (Models, Fields, ORM) slides.

O01 Install Odoo 17 on Ubuntu 22.04

Learning Objectives

- Prepare Ubuntu 22.04 for Odoo 17 development.
- Install system packages, Python dependencies, Node tooling, and wkhtmltopdf.
- Connect the installation to a PostgreSQL role.

Explanation

An Odoo installation is a Python application, a PostgreSQL database client, a set of system libraries, and a front-end asset toolchain. Treat the server setup as part of the application, because missing libraries often appear later as PDF, image, or asset errors.

Ubuntu 22.04 is a common base for Odoo 17 because it provides a stable Python and package ecosystem. Even in training, use a documented setup so students can reproduce bugs and compare results.

A virtual environment keeps Odoo's Python packages separate from the operating system. This matters when multiple Odoo versions or experiments live on the same machine.

PostgreSQL should already have a dedicated role for Odoo. The role usually needs CREATEDB in development, while production permissions should be reviewed more strictly.

wkhtmltopdf remains important for many Odoo PDF reports. Use a known compatible package and test report rendering early instead of discovering the issue during invoice printing.

After installation, start with a small database and install only base modules needed for the lab. A minimal first boot is easier to debug than a heavily customized environment.

Key Points

- Document every system package required by the Odoo instance.
- Use a Python virtual environment for source-based installs.
- Prepare PostgreSQL before first Odoo startup.
- Test PDF rendering as part of installation validation.

Source install outline

This prepares a source checkout suitable for development and keeps Python dependencies inside the local virtual environment.

```
sudo apt update
sudo apt install -y git python3.10-venv python3-dev build-essential libxml2-dev libxslt1-dev libldap2-dev libsasl2-dev

git clone --depth 1 --branch 17.0 https://github.com/odoo/odoo.git /opt/odoo/odoo17
cd /opt/odoo/odoo17
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip wheel
pip install -r requirements.txt
```

Common Mistakes

- Installing Python packages globally and later mixing Odoo versions.
- Skipping wkhtmltopdf validation until the first customer invoice must be printed.
- Using the postgres superuser as the Odoo database user.
- Starting from too many addons before the base installation is proven.

Lab / Practice: Install Odoo 17 from source on Ubuntu 22.04, activate the virtual environment, and run `odoo-bin --version`.

O02 Start odoo-bin for the First Time

Learning Objectives

- Run Odoo from the source tree with explicit options.
- Understand addons paths, database host options, and startup logs.
- Use `--stop-after-init` for controlled initialization.

Explanation

`odoo-bin` is the main entry point for source-based Odoo. Running it manually is the best way to learn what the server is loading before hiding it behind `systemd` or an IDE.

The addons path determines which modules Odoo can discover. A wrong path can make a valid custom module invisible, while duplicate module names across paths can create confusing behavior.

Database options tell Odoo how to reach PostgreSQL. In local development, host `127.0.0.1` with a dedicated user is explicit and avoids peer authentication surprises.

Startup logs show configuration, addons discovery, database connections, HTTP routes, and module loading. Senior developers read the first failure in the log, not only the final stack trace.

`--stop-after-init` is useful for installing or upgrading modules from the CLI and then exiting. It gives deterministic command-line workflows for CI, migrations, and training labs.

A first run should use a fresh database and a small module set. Once the server can start cleanly, add custom addons and debugging tools incrementally.

Key Points

- `odoo-bin` is the source checkout entry point.
- Addons path order affects module discovery.
- `--stop-after-init` is essential for scripted installs and upgrades.
- Read startup logs from the first error forward.

Create and start a training database

The first command initializes the database and exits; the second starts the server using a configuration file.

```
./odoo-bin --addons-path=addons --db_host=127.0.0.1 --db_user=odoo17 --db_password=odoo17 --data_dir=/var/lib/odoo17
./odoo-bin -c /etc/odoo17.conf --database=odoo17_training
```

Common Mistakes

- Letting Odoo guess database settings and then debugging the wrong authentication path.
- Putting a custom addons path before core addons without understanding name collisions.
- Ignoring warnings during startup because the web UI appears to load.
- Using `--init` repeatedly when `--update` is the intended operation.

Lab / Practice: Start Odoo once with explicit CLI options and once with a config file, then compare the first 30 log lines.

O03 Odoo CLI Commands Overview

Learning Objectives

- Recognize the most useful odoo-bin command-line options.
- Install, update, test, scaffold, and shell from the CLI.
- Choose command options appropriate for development and maintenance.

Explanation

The Odoo CLI is more than a way to start the HTTP server. It is also how developers install modules, run upgrades, create scaffolds, execute tests, open shells, and control database selection.

Common options such as `-c`, `-d`, `-i`, `-u`, `--addons-path`, and `--stop-after-init` are the daily vocabulary of Odoo development. Knowing them makes deployments repeatable instead of dependent on browser clicks.

The shell command opens an Odoo environment in Python. It is powerful for diagnostics, but it can change real data, so use it with the same care as `psql`.

Test options allow module tests to run without manually clicking through workflows. A serious custom module should have at least a small test surface that can be run from the CLI.

CLI commands should be copied into documentation, deployment scripts, or CI jobs once they are proven. If a release depends on a developer's memory, it is not a release process.

Different Odoo versions add or change options, so always check `--help` for the installed source tree. Training should teach the pattern, not just one memorized command.

Key Points

- `-c` selects the configuration file.
- `-d` selects the database.
- `-i` installs modules; `-u` upgrades modules.
- `--stop-after-init` makes install or upgrade commands exit.
- `odoo-bin shell` gives direct ORM access.

Daily CLI commands

These commands cover the most common lifecycle actions: inspect, install, upgrade, debug, scaffold, and test.

```
./odoo-bin --help
./odoo-bin -c odoo.conf -d odoo17_training -i sale --stop-after-init
./odoo-bin -c odoo.conf -d odoo17_training -u my_module --stop-after-init
./odoo-bin shell -c odoo.conf -d odoo17_training
./odoo-bin scaffold academy_course /opt/odoo/custom_addons
./odoo-bin -c odoo.conf -d odoo17_training --test-enable --stop-after-init
```

Common Mistakes

- Using `-i` on a module that is already installed when `-u` is needed.
- Running shell commands against the wrong database.
- Forgetting `--stop-after-init` in scripts and leaving a server process running.
- Assuming browser-based module upgrades are equivalent to documented release commands.

Lab / Practice: Run `--help`, scaffold a throwaway module, and write the exact command that would update it in a named database.

004 The Odoo Configuration File

Learning Objectives

- Create an `odoo.conf` suitable for development.
- Configure database, addons, logging, workers, and admin password options.
- Keep secrets and environment-specific values out of source control.

Explanation

The configuration file turns a long command line into a repeatable server definition. It should make the runtime obvious to another developer who has never used your laptop.

Database settings define where Odoo connects and which credentials it uses. In development they are often explicit, while production may use environment management or secret files around the config.

`addons_path` is one of the most important settings in the file. It must include core addons, enterprise addons if used, and custom addons in a deliberate order.

`admin_passwd` protects database management operations such as creating, duplicating, dropping, and backing up databases through the web manager. It is not the same as an Odoo user password.

Worker and limit settings should reflect the deployment mode. Development commonly uses one process with live reload, while production needs worker sizing, memory limits, time limits, and reverse proxy settings.

Never commit production secrets into a training repository or custom module. Provide a sample config and document how real values are supplied on each machine.

Key Points

- Use `odoo.conf` to make startup repeatable.
- `admin_passwd` controls database manager actions.
- `addons_path` must include every module location in the right order.
- Keep production secrets outside versioned code.

Development `odoo.conf`

This sample is explicit enough for training while still showing which values should be changed for real environments.

```
[options]
addons_path = /opt/odoo/odoo17/addons,/opt/odoo/custom_addons
data_dir = /var/lib/odoo17
db_host = 127.0.0.1
db_port = 5432
db_user = odoo17
db_password = odoo17
admin_passwd = change-this-dev-password
log_level = info
logfile = /var/log/odoo/odoo17.log
xmlrpc_port = 8069
limit_time_cpu = 120
limit_time_real = 240
```

Common Mistakes

- Committing real `db_password` or `admin_passwd` values.
- Forgetting one addons path and wondering why a module does not appear.
- Using production worker settings for local debugging.
- Changing config values but starting Odoo with a different `-c` file.

Lab / Practice: Create a local `odoo.conf`, start Odoo with `-c`, and prove from the logs that your custom addons path was loaded.

O05 Database Manager URLs and Disabling Database Manager

Learning Objectives

- Use the Odoo database manager routes safely in development.
- Understand the risk of exposing database management in production.
- Disable database manager access with configuration.

Explanation

Odoo includes browser routes for creating, duplicating, restoring, backing up, and dropping databases. They are convenient in training but must be treated as administrative tools.

The database manager is usually reached through `/web/database/manager`, with related routes for create, backup, restore, duplicate, and drop. Access is protected by the master password, but exposure still increases risk.

In production, many teams disable `list_db` and block database manager routes at the reverse proxy. The principle is simple: customers should reach the application, not database administration screens.

`dbfilter` also matters because it controls which database a request can select. A good `dbfilter` prevents one Odoo service from accidentally offering unrelated databases on the same PostgreSQL server.

The master password must be strong and private if database manager remains enabled. It authorizes destructive actions, so it belongs in secret management rather than a shared wiki.

For training, keep database manager enabled only while teaching the feature. Then show how to disable it so learners understand both convenience and operational safety.

Key Points

- `/web/database/manager` exposes administrative database actions.
- `admin_passwd` is the master password for these operations.
- `list_db = False` hides database listing and is common in production.
- Reverse proxies can block database manager routes.

Production-oriented database manager settings

The config hides database listing and the reverse proxy blocks database manager routes before they reach Odoo.

```
[options]
admin_passwd = use-a-secret-manager-value
list_db = False
dbfilter = ^%d$
proxy_mode = True

# Nginx example
location ~* /web/database/ {
    deny all;
}
```

Common Mistakes

- Leaving database manager visible on a public production domain.
- Using a weak `admin_passwd` because normal users cannot see it after login.
- Setting `dbfilter` incorrectly and making valid databases unreachable.
- Assuming `list_db = False` replaces network and proxy hardening.

Lab / Practice: Open the database manager in a lab instance, create a database, then set `list_db = False` and confirm the list no longer appears.

O06 Database CLI and Upgrade CLI Workflows

Learning Objectives

- Create, initialize, update, and test databases from the command line.
- Use upgrade commands safely for module development.
- Separate database operations from browser habits.

Explanation

Database work in Odoo should be scriptable. A command that creates a database, installs modules, and exits is easier to review than a sequence of browser clicks.

The `-i` option installs modules into a database and should be used for first installation. The `-u` option updates already installed modules and is the normal path after changing Python fields, views, security, or data files.

`--stop-after-init` turns Odoo into a batch command. It loads the registry, performs install or update work, runs enabled tests if requested, and exits with a status code.

For module upgrades, keep the database selected explicitly with `-d`. Accidentally upgrading a customer copy, staging database, or wrong training database is a common and preventable mistake.

When tests are part of the command, failures should stop the release. Do not hide test failures behind a server that keeps running and makes the deployment look successful.

A good upgrade habit includes taking a backup, stopping extra workers or cron where needed, running the update command, reviewing logs, and doing a focused smoke test.

Key Points

- `-i` is for installation; `-u` is for updates.
- Always specify the target database.
- `--stop-after-init` makes commands automation-friendly.
- Upgrade workflows should include backup and smoke testing.

Repeatable module upgrade

The first command updates a module, and the second adds test execution for a release-quality upgrade check.

```
DB=odoo17_training
CONF=/etc/odoo17.conf

./odoo-bin -c "$CONF" -d "$DB" -u academy_course --stop-after-init

./odoo-bin -c "$CONF" -d "$DB" -u academy_course --test-enable --stop-after-init --log-level=test
```

Common Mistakes

- Running `-u` all casually on a large or customized production database.
- Not backing up before structural module upgrades.
- Forgetting that Odoo shell and upgrade commands can write real data.
- Using browser upgrades as the only deployment procedure.

Lab / Practice: Install a scaffolded module with `-i`, change its manifest or view, then update it with `-u` and `--stop-after-init`.

O07 PyCharm Configuration for Odoo

Learning Objectives

- Configure a Python interpreter and run configuration for Odoo.
- Pass Odoo CLI parameters from the IDE.
- Debug server startup and custom module code.

Explanation

An IDE configuration should mirror the command line that already works. If PyCharm starts Odoo differently than the terminal, debugging becomes a second environment problem.

Point PyCharm at the virtual environment used for the Odoo checkout. This gives code completion for installed dependencies and prevents accidental imports from the system Python.

The script path is usually `odoo-bin`, and parameters should include `-c` and the target database or development flags. Working directory should be the Odoo source root unless your project layout intentionally differs.

Breakpoints work best when workers are disabled and code reload behavior is understood. In development, start with a single process before adding worker-based production settings.

Mark custom addons and the Odoo source as project roots when useful for navigation. Good indexing makes model inheritance, XML ids, and controller code easier to inspect.

Keep IDE-only configuration out of deployment assumptions. The IDE helps developers move faster, but the terminal command remains the source of truth for automation.

Key Points

- Use the same virtual environment in terminal and PyCharm.
- Set `odoo-bin` as the script path.
- Pass `-c` and `-d` explicitly in run parameters.
- Disable workers for straightforward breakpoint debugging.

PyCharm run configuration values

These settings make the IDE run the same Odoo server that the terminal command would run.

```
Script path:
/opt/odoo/odoo17/odoo-bin

Parameters:
-c /etc/odoo17.conf -d odoo17_training --dev=xml,qweb

Working directory:
/opt/odoo/odoo17

Environment:
PYTHONUNBUFFERED=1
```

Common Mistakes

- Using a different interpreter in PyCharm than the one used to install `requirements.txt`.
- Debugging with workers enabled and wondering why breakpoints are skipped.
- Leaving the database name empty and attaching to the wrong database.
- Changing PyCharm parameters without updating the documented CLI command.

Lab / Practice: Create a PyCharm run configuration that starts your training database and stops at a breakpoint in a custom model method.

O08 Scaffold a Module and Use Multiple Addons Paths

Learning Objectives

- Generate a module scaffold with odoo-bin.
- Understand custom module directory structure.
- Configure multiple addons paths safely.

Explanation

Scaffolding creates a conventional starting point for a custom module. It is not magic, but it saves time and teaches the files Odoo expects.

A custom module is discovered because its directory sits under one of the configured addons paths and contains a manifest file. The directory name is the technical module name used in CLI commands.

Multiple addons paths are normal in real projects. A typical setup includes community addons, enterprise addons, first-party custom addons, and sometimes vendor addons.

Path order matters when modules share a technical name. Avoid duplicate names entirely because they make upgrades and debugging unpredictable.

The scaffold contains models, views, security, controllers, demo, and tests placeholders depending on the template. Remove or fill placeholders so the module is intentional, not a pile of unused files.

A clean addons layout makes onboarding easier. Developers should know where core code ends, where vendor code lives, and where the team's custom ownership begins.

Key Points

- A module is a directory with a manifest.
- The directory name is the technical module name.
- addons_path can contain several comma-separated roots.
- Duplicate module names across paths are dangerous.

Scaffold and configure paths

The scaffold command creates the module directory, and the config makes Odoo discover it.

```
mkdir -p /opt/odoo/custom_addons
./odoo-bin scaffold academy_course /opt/odoo/custom_addons

# odoo.conf
addons_path = /opt/odoo/odoo17/addons,/opt/odoo/enterprise,/opt/odoo/custom_addons

./odoo-bin -c /etc/odoo17.conf -d odoo17_training -i academy_course --stop-after-init
```

Common Mistakes

- Putting the module directory itself in addons_path instead of its parent directory.
- Using spaces around paths that are later copied incorrectly into service files.
- Keeping duplicate technical module names in different paths.
- Installing a scaffold before reviewing its manifest, security, and views.

Lab / Practice: Scaffold academy_course, add its parent directory to addons_path, update the app list, and install the module from the CLI.

009 Manifest Files, Dependencies, and Auto Install Modules

Learning Objectives

- Write a correct `__manifest__.py` for Odoo 17.
- Declare dependencies and data files deliberately.
- Use `auto_install` only for glue modules that should install automatically.

Explanation

The manifest is the module contract Odoo reads before loading code and data. It tells Odoo the module's name, version, dependencies, category, data files, assets, license, and install behavior.

Dependencies control load order and registry availability. If your model inherits `sale.order`, the `sale` module belongs in `depends`; relying on a user's installed apps is fragile.

Data file order matters because XML records can reference records loaded earlier in the same module or in dependencies. Security files typically load before views that expose the model.

`auto_install` is for modules that connect other modules when their dependencies are present. It is not a shortcut to force every database to install a feature.

The manifest should be boring and accurate. Inflated dependencies slow installations, while missing dependencies create intermittent failures depending on what happens to be installed.

Version values help release and migration discipline. Even if Odoo does not enforce semantic versioning for custom modules, your team should use versions to communicate change scope.

Key Points

- `depends` declares modules required before this module loads.
- data file order is load order.
- `auto_install` is for glue modules.
- Keep manifests accurate and minimal.

Odoo 17 manifest

The manifest declares precise dependencies, ordered data files, and normal install behavior for an application module.

```
{
  "name": "Academy Course Management",
  "version": "17.0.1.0.0",
  "category": "Education",
  "summary": "Manage training courses and sessions",
  "depends": ["base", "mail", "product"],
  "data": [
    "security/ir.model.access.csv",
    "views/academy_course_views.xml",
    "views/academy_session_views.xml",
    "views/menu.xml",
  ],
  "demo": ["demo/academy_demo.xml"],
  "application": True,
  "installable": True,
  "auto_install": False,
  "license": "LGPL-3",
}
```

Common Mistakes

- Forgetting a dependency because the module happens to be installed in the developer database.
- Loading views before security access rules.
- Setting `auto_install` to `True` on a normal business application module.
- Leaving `installable` `False` after copying a scaffold.

Lab / Practice: Create a manifest for a training module that depends on mail, includes security before views, and installs cleanly from the CLI.

O10 Custom Module Structure

Learning Objectives

- Organize Python, XML, security, data, demo, and test files.
- Import models correctly through `__init__.py` files.
- Keep module ownership clear as the codebase grows.

Explanation

A module structure is a map for both Odoo and future developers. Consistent directories reduce cognitive load when a bug report mentions a model, view, security rule, or scheduled action.

Python packages must be imported for model classes to register. The top-level `__init__.py` imports models, and `models/__init__.py` imports the individual Python files.

Security files define who can access the model and under what record rules. Without access rights, a beautiful view can still fail with an access error as soon as a normal user opens it.

Views describe the user interface and should reference model fields that already exist at module load time. XML ids become stable references used by menus, actions, tests, and other modules.

Data and demo files serve different purposes. Data is normally required for the module to work, while demo should be safe sample content for training and development databases.

Tests belong close to the module because they describe expected module behavior. Even a small test for create, access, and a computed field is valuable during upgrades.

Key Points

- `__init__.py` imports are required for model registration.
- `security/ir.model.access.csv` is usually loaded before views.
- data is functional; demo is sample content.
- Tests should travel with the module.

Module layout

The layout separates responsibilities and shows the import chain Odoo needs to register models.

```
academy_course/  
  __init__.py  
  __manifest__.py  
  models/  
    __init__.py  
    academy_course.py  
    academy_session.py  
  security/  
    ir.model.access.csv  
    academy_security.xml  
  views/  
    academy_course_views.xml  
    menu.xml  
  data/  
    sequence.xml  
  demo/  
    academy_demo.xml  
  tests/  
    __init__.py  
    test_academy_course.py  
  
# academy_course/__init__.py  
from . import models  
  
# academy_course/models/__init__.py  
from . import academy_course  
from . import academy_session
```

Common Mistakes

- Creating a Python model file but forgetting to import it.
- Putting required records in demo data and then missing them in production.
- Letting one large models.py file grow without domain boundaries.
- Adding views without corresponding access rights for target users.

Lab / Practice: Create the directories for academy_course, add import files, and confirm a simple model appears in Settings > Technical > Models.

O11 `_name`, `_table`, and Model Identity

Learning Objectives

- Define a new Odoo model with `_name`.
- Understand default table naming and when to use `_table`.
- Choose technical names that support long-term maintenance.

Explanation

`_name` is the model's technical identifier in the Odoo registry. It is used in Python, XML actions, security files, domains, external APIs, and logs.

By default, Odoo turns dots in `_name` into underscores for the PostgreSQL table name. A model named `academy.course` normally stores rows in `academy_course`.

`_table` can override the physical table name. Use it rarely, usually for legacy integration, SQL views, or situations where a stable existing table name must be preserved.

Technical names should be unique and module-scoped. A good name such as `academy.course` is clearer than `course.course` and less likely to collide with another addon.

Changing `_name` after data exists is a migration, not a rename in one file. Security XML ids, views, relations, `ir.model` metadata, and existing tables may all need migration work.

Use `_description` to make technical screens and logs friendlier. Small metadata choices help administrators understand custom models years later.

Key Points

- `_name` identifies the model in the registry.
- Default table name replaces dots with underscores.
- `_table` overrides the physical table name.
- Changing model identity after release requires migration.

Model identity

The class defines a new registry model and uses an explicit table name that matches Odoo's default convention.

```
from odoo import fields, models

class AcademyCourse(models.Model):
    _name = "academy.course"
    _description = "Academy Course"
    _table = "academy_course"

    name = fields.Char(required=True)
    code = fields.Char(required=True, index=True)
    active = fields.Boolean(default=True)
```

Common Mistakes

- Renaming `_name` in a released module without a migration plan.
- Using generic model names that collide with vendor or future modules.
- Setting `_table` unnecessarily and hiding the default naming convention.
- Forgetting `_description` and making technical screens harder to read.

Lab / Practice: Create `academy.course`, install the module, and inspect the generated `academy_course` table in PostgreSQL.

O12 Field Overview and Simple Field Options

Learning Objectives

- Recognize common field declaration options.
- Use required, readonly, index, help, default, copy, and tracking deliberately.
- Understand how fields connect Python models to database columns and views.

Explanation

Odoo fields describe both storage and business semantics. A field declaration can create a database column, expose metadata to views, enable search behavior, and participate in tracking or computed logic.

required enforces input at the ORM and user interface level, while SQL constraints may still be needed for deep invariants. index improves common search patterns but adds database write cost.

readonly affects how the field is edited through views and ORM conventions, but it is not a security boundary by itself. Access rights and record rules remain the real permission controls.

default values can be constants, callables, or context-driven helpers. Use defaults to reduce user work, but avoid hiding business decisions that should be explicit.

copy controls whether a field is duplicated when a record is copied. This is important for sequence numbers, external references, and state fields that should not carry over.

tracking integrates with mail.thread to log changes in the chatter. It is valuable for business audit trails, but excessive tracking can create noise and mail_message growth.

Key Points

- Field declarations carry storage, UI, and ORM metadata.
- Indexes should match real searches.
- readonly is not a security mechanism.
- copy=False is important for unique references and workflow state.

Common field options

The fields demonstrate common options that affect validation, indexing, duplication, help text, defaults, and chatter tracking.

```
from odoo import fields, models

class AcademyCourse(models.Model):
    _name = "academy.course"
    _inherit = ["mail.thread"]
    _description = "Academy Course"

    name = fields.Char(required=True, tracking=True)
    code = fields.Char(required=True, index=True, copy=False)
    active = fields.Boolean(default=True)
    notes = fields.Text(help="Internal teaching notes.")
    state = fields.Selection(
        [("draft", "Draft"), ("published", "Published")],
        default="draft",
        tracking=True,
    )
```

Common Mistakes

- Adding index=True to fields that are rarely searched.
- Expecting readonly=True to stop writes from server-side code.
- Letting copy=True duplicate external identifiers.
- Tracking every field and making chatter unusable.

Lab / Practice: Add five fields with different options to academy.course and confirm their metadata with fields_get in Odoo shell.

O13 Text-Oriented Fields: Char, Text, and Html

Learning Objectives

- Choose between Char, Text, and Html fields.
- Use size, translate, sanitize, and help options appropriately.
- Render text fields safely in views.

Explanation

Char is for short values such as names, codes, references, phone numbers, and labels. It communicates that the content is concise and often searchable.

Text is for longer plain text such as internal notes, descriptions, and instructions. It does not imply rich formatting, which keeps content simpler for exports and integrations.

Html stores rich text and is common for website content, email bodies, terms, and formatted descriptions. Because it can contain markup, sanitization and trust boundaries matter.

`translate=True` is useful for user-facing labels or descriptions that should differ by language. Do not mark operational codes or external references as translatable.

The widget chosen in XML affects the editing experience. A Text field can use a textarea-like widget, while Html commonly uses Odoo's rich text editor.

Search behavior differs in practice because short indexed fields are easier to optimize than large bodies of text. Use the field that matches the business meaning rather than storing everything as Html.

Key Points

- Char is for short structured text.
- Text is for long plain text.
- Html is for rich formatted content and needs sanitization awareness.
- `translate=True` belongs on user-facing language content.

Course text fields and XML

The model separates searchable codes, plain descriptions, and rich syllabus content instead of using one text type for everything.

```
class AcademyCourse(models.Model):
    _name = "academy.course"
    _description = "Academy Course"

    name = fields.Char(required=True, translate=True)
    code = fields.Char(required=True, size=32, index=True)
    description = fields.Text(translate=True)
    syllabus_html = fields.Html(string="Syllabus", sanitize=True, translate=True)

<!-- views/academy_course_views.xml -->
<field name="description" placeholder="Plain text course summary"/>
<field name="syllabus_html" widget="html"/>
```

Common Mistakes

- Using Html for plain operational notes.
- Making external codes translatable.
- Expecting a Char size to replace a real business validation.
- Rendering user-provided HTML without considering sanitization.

Lab / Practice: Add Char, Text, and Html fields to a model, place them in a form view, and compare how each appears in the UI.

O14 Boolean, Selection, Integer, Float, and Json Fields

Learning Objectives

- Model flags, states, counts, quantities, and flexible payloads.
- Choose numeric fields carefully for business meaning.
- Use Json fields without replacing relational design.

Explanation

Boolean fields represent yes-or-no state and are often used for toggles such as active, published, billable, or requires approval. They are simple, but too many booleans can hide a workflow that should be a Selection.

Selection fields represent a controlled set of values. They are ideal for workflow state when the allowed values are known and transitions can be managed in methods.

Integer fields are appropriate for counts and whole-number settings. Float fields are appropriate for approximate decimal quantities, but financial amounts usually need Monetary fields.

Float precision in Odoo is normally controlled through decimal precision settings or digits. Do not assume Python floating point behavior is acceptable for accounting decisions.

Json fields store structured Python dictionaries and lists as JSON data. They are useful for integration payloads, configuration fragments, or analytics data that does not deserve first-class relational fields yet.

Json should not become a dumping ground for data that users search, secure, relate, or report on every day. Once a JSON key becomes core business data, promote it to a real field.

Key Points

- Use Selection for controlled workflow states.
- Use Integer for whole counts and Float for non-money quantities.
- Use Monetary for currency amounts instead of bare Float.
- Json is flexible but weaker than relational fields for core data.

Operational scalar fields

The fields show a workflow state, whole-number capacity, decimal duration, and optional integration metadata.

```
class AcademySession(models.Model):
    _name = "academy.session"
    _description = "Academy Session"

    active = fields.Boolean(default=True)
    state = fields.Selection(
        [("draft", "Draft"), ("confirmed", "Confirmed"), ("done", "Done")],
        default="draft",
        required=True,
    )
    seat_count = fields.Integer(default=12)
    duration_hours = fields.Float(digits=(16, 2))
    integration_payload = fields.Json(copy=False)
```

Common Mistakes

- Using several booleans where one Selection state would be clearer.
- Using Float for currency amounts that must reconcile.
- Storing searchable operational data only inside Json.
- Changing Selection keys without migrating existing database values.

Lab / Practice: Create a state field with three values, add a button method that changes the state, and store one sample integration payload in a Json field.

O15 Date and Datetime Fields

Learning Objectives

- Use Date and Datetime fields correctly in Odoo 17.
- Understand UTC storage and user timezone conversion.
- Write safe defaults and domains for date ranges.

Explanation

Date fields store calendar dates without time of day. They are appropriate for birthdays, accounting dates, deadlines, validity dates, and business days.

Datetime fields store a timestamp and Odoo treats them as UTC internally. The web client displays them in the user's timezone, which is helpful but can surprise developers inspecting raw PostgreSQL values.

Use `fields.Date.today` and `fields.Datetime.now` for defaults rather than evaluating Python date functions at import time. Defaults must run when the record is created, not when the server starts.

Date range domains should be explicit about inclusive and exclusive boundaries. For Datetime fields, a half-open range is often safer than trying to include the last second of a day.

Context-aware date helpers are useful when user timezone matters for business interpretation. A scheduled action running at midnight UTC may not match a user's local business day.

Avoid storing date strings in Char fields. Real Date and Datetime fields give better validation, widgets, searches, grouping, and reporting.

Key Points

- Date has no time; Datetime has timestamp semantics.
- Odoo stores datetimes in UTC.
- Use callable defaults such as `fields.Date.today`.
- Use half-open ranges for Datetime domains.

Date fields and safe domains

The example uses callable defaults and a half-open monthly range that avoids boundary confusion.

```
class AcademySession(models.Model):
    _name = "academy.session"

    start_date = fields.Date(default=fields.Date.today, required=True)
    start_at = fields.Datetime(default=fields.Datetime.now, required=True)
    deadline = fields.Date()

    def sessions_this_month(self):
        start = fields.Date.start_of(fields.Date.today(), "month")
        end = fields.Date.add(start, months=1)
        return self.search([
            ("start_date", ">=", start),
            ("start_date", "<", end),
        ])
```

Common Mistakes

- Writing `default=datetime.now()` with parentheses in the field declaration.
- Comparing raw UTC database values to a user's local expectation without conversion.
- Using Char fields for dates.
- Using `<= end_of_day` strings for Datetime ranges instead of a clear next boundary.

Lab / Practice: Add Date and Datetime fields, create records from two user timezones, and compare the UI value to the raw PostgreSQL value.

O16 Relational Fields: Many2one, One2many, and Many2many

Learning Objectives

- Model core relationships with Odoo relational fields.
- Understand inverse fields and relation tables.
- Use ondelete, domain, context, and check_company carefully.

Explanation

Relational fields are the heart of Odoo's data model. Most business workflows are built by connecting documents, partners, companies, products, users, and accounting records through relationships.

Many2one stores a foreign key to one related record. It is the most common relation and often appears in database tables as a column ending with `_id`.

One2many is the inverse view of a Many2one. It does not store a separate list in the parent table; it shows child records whose inverse field points back to the parent.

Many2many stores pairs of ids in a relation table. It is useful for tags, attendees, allowed users, and other cases where both sides can have many records.

ondelete controls what should happen when a referenced record is removed. In business systems, restrict or set null is often safer than cascade unless child data is purely dependent.

Domains and contexts on relational fields improve user experience by filtering choices and defaulting values. They should support business rules, not replace server-side validation where integrity matters.

Key Points

- Many2one stores the foreign key.
- One2many depends on an inverse Many2one.
- Many2many uses a relation table.
- Choose ondelete behavior based on business data ownership.

Course relationships

The parent course sees sessions through One2many, while each session stores the real `course_id` foreign key.

```
class AcademyCourse(models.Model):
    _name = "academy.course"

    name = fields.Char(required=True)
    session_ids = fields.One2many("academy.session", "course_id")
    tag_ids = fields.Many2many("academy.tag", string="Tags")

class AcademySession(models.Model):
    _name = "academy.session"

    name = fields.Char(required=True)
    course_id = fields.Many2one(
        "academy.course",
        required=True,
        ondelete="restrict",
        index=True,
    )
    instructor_id = fields.Many2one(
        "res.partner",
        domain=[("is_company", "=", False)],
        context={"default_is_company": False},
    )
```

Common Mistakes

- Defining One2many without a valid inverse Many2one field.
- Using cascade deletion on records that have audit or accounting meaning.
- Expecting a Many2many to preserve ordering like line items.

- Using UI domains as the only protection for business integrity.

Lab / Practice: Create course, session, and tag models, then inspect the generated foreign key column and Many2many relation table.

O17 Reference and Related Fields

Learning Objectives

- Use Reference fields for controlled polymorphic links.
- Use Related fields to expose values from linked records.
- Decide when stored related fields are worth the cost.

Explanation

A Reference field can point to records from different models. It is useful for generic links, but it is weaker than a normal Many2one because the database cannot enforce a single foreign key.

Use Reference only when polymorphism is truly part of the business requirement. Many designs become simpler and safer with explicit Many2one fields to the expected model.

Related fields expose a value through a relationship path, such as `session.course_id.company_id`. They reduce duplication and keep the UI close to the data users need.

A non-stored related field is computed on access and does not create a column. A stored related field is searchable and groupable more efficiently, but it must be recomputed when dependencies change.

Related fields can be readonly by default because their source is another record. If users should edit the source value, make the relationship and ownership clear in the view.

For reporting and domains, store related fields only when there is a real search or performance reason. Storing every convenience value adds recomputation work and schema noise.

Key Points

- Reference fields are polymorphic but less constrained.
- Related fields expose values through relation paths.
- `store=True` improves searchability at recomputation cost.
- Prefer explicit Many2one when only one target model is valid.

Reference and related fields

The stored `course_id` can support search or grouping, while the instructor remains a lightweight display convenience.

```
class AcademyActivity(models.Model):
    _name = "academy.activity"

    name = fields.Char(required=True)
    resource_ref = fields.Reference(
        selection=[("academy.course", "Course"), ("academy.session", "Session")],
        string="Related Resource",
    )
    session_id = fields.Many2one("academy.session")
    course_id = fields.Many2one(related="session_id.course_id", store=True)
    instructor_id = fields.Many2one(related="session_id.instructor_id", store=False)
```

Common Mistakes

- Using Reference fields when a normal Many2one would enforce the relationship.
- Storing related fields without a search or reporting reason.
- Expecting database foreign keys from Reference values.
- Making related fields editable without explaining where the value is actually stored.

Lab / Practice: Add a related course field on session activity, make one version stored and one non-stored, then compare `fields_get` metadata.

O18 Compute Fields

Learning Objectives

- Write computed fields with correct dependencies.
- Choose between stored and non-stored compute fields.
- Use inverse and search methods when the business case requires them.

Explanation

Computed fields derive their value from other fields or records. They keep business logic in Python instead of forcing users or integrations to maintain redundant values manually.

@api.depends tells Odoo which changes should trigger recomputation. Missing dependencies create stale values, while overly broad dependencies waste time during writes.

Non-stored computed fields are calculated when read. Stored computed fields create database columns and are recomputed when dependencies change, which makes them searchable and groupable in more cases.

Every record in self must receive a value in the compute method. Compute methods that accidentally assign only some records create cache errors that can be difficult for beginners to understand.

Inverse methods allow a computed field to be edited by translating the entered value back to source fields. Use them only when that reverse mapping is clear and stable.

Search methods let non-stored computed fields participate in domains. They should return an equivalent domain, not perform arbitrary recordset filtering in Python for large datasets.

Key Points

- Use @api.depends accurately.
- Assign a value for every record in the compute method.
- store=True trades recomputation cost for search and grouping benefits.
- Inverse and search methods are advanced tools for specific cases.

Stored computed field

The compute method assigns every course a count and stores the result for efficient display and search.

```
from odoo import api, fields, models

class AcademyCourse(models.Model):
    _name = "academy.course"

    session_ids = fields.One2many("academy.session", "course_id")
    session_count = fields.Integer(compute="_compute_session_count", store=True)

    @api.depends("session_ids")
    def _compute_session_count(self):
        for course in self:
            course.session_count = len(course.session_ids)
```

Common Mistakes

- Forgetting @api.depends and wondering why values are stale.
- Assigning a computed value only inside an if branch.
- Setting store=True on expensive fields without considering write impact.
- Using compute fields to hide data that should be normalized into real relational fields.

Lab / Practice: Add session_count to academy.course, update sessions, and confirm the count changes after module upgrade.

O19 Monetary, Binary, and Image Fields

Learning Objectives

- Represent currency amounts with Monetary fields.
- Store files and images with Binary and Image fields.
- Understand attachment storage and UI widgets.

Explanation

Monetary fields pair an amount with a currency field. This is the correct pattern for prices, fees, budgets, and totals that users expect to format by currency.

A Monetary field normally references a `res.currency` record through `currency_field`. Without a clear currency, an amount is just a number and can be misread in multi-company environments.

Binary fields store file-like data and often use `attachment=True` so data is stored as an `ir.attachment` rather than directly in the model table. This keeps large payloads out of frequently scanned business tables.

Image fields build on binary storage with image-specific resizing and widgets. They are appropriate for logos, profile pictures, product images, and course thumbnails.

Files and images affect backup strategy because Odoo attachments commonly live in the filestore. A database dump without the filestore can leave Binary and Image fields broken.

Access to attachments must match business expectations. Do not assume that because a file is stored indirectly it is automatically safe for every user to retrieve.

Key Points

- Use Monetary with a currency field for money.
- Binary fields often store data as attachments.
- Image fields support image-specific UI and processing.
- Database and filestore backups must stay together.

Money and files

The example links amounts to company currency and stores course files through Odoo's attachment mechanism.

```
class AcademyCourse(models.Model):
    _name = "academy.course"

    currency_id = fields.Many2one(
        "res.currency",
        default=lambda self: self.env.company.currency_id,
        required=True,
    )
    fee = fields.Monetary(currency_field="currency_id")
    brochure = fields.Binary(attachment=True)
    brochure_filename = fields.Char()
    image_1920 = fields.Image(max_width=1920, max_height=1920)

<!-- XML -->
<field name="fee"/>
<field name="brochure" filename="brochure_filename"/>
<field name="image_1920" widget="image"/>
```

Common Mistakes

- Using Float for prices instead of Monetary.
- Forgetting the currency field in multi-company data.
- Backing up the database but not the filestore.
- Storing large files directly on heavily used models without considering attachment behavior.

Lab / Practice: Add a fee, currency, brochure, and image to `academy.course`, then upload a file and confirm an `ir.attachment` was created.

O20 Default ORM Methods and Recordsets

Learning Objectives

- Understand recordsets and default model method style.
- Use browse, create, write, copy, and unlink safely.
- Recognize when methods operate on multiple records.

Explanation

Odoo's ORM represents records as recordsets, even when there is only one record. This design lets methods work consistently on zero, one, or many records.

Model methods receive self as a recordset or model environment depending on how they are called. A method should be clear about whether it supports multiple records or requires exactly one.

browse creates a recordset from ids without immediately verifying that every id exists. exists is often used later when the code must discard deleted or inaccessible records.

create, write, copy, and unlink are core lifecycle methods. They are powerful extension points but also high-risk because many modules may override the same methods in the inheritance chain.

The ORM handles access rules, field conversions, computed fields, invalidation, and relational commands. Direct SQL skips these behaviors and should be reserved for carefully controlled cases.

A senior Odoo developer thinks in recordsets, not loops over raw ids. This leads to clearer code and fewer unnecessary database queries.

Key Points

- self is usually a recordset.
- browse does not guarantee records still exist.
- Core lifecycle methods should be extended carefully.
- Prefer ORM operations unless direct SQL is deliberately justified.

Recordset basics in shell

The shell example shows recordset search, batch write, existence checking, copy, and create through the ORM.

```
courses = env["academy.course"].search([("active", "=", True)], limit=10)
courses.write({"state": "published"})

course = env["academy.course"].browse(42)
if course.exists():
    duplicate = course.copy({"name": "%s (Copy)" % course.name})

env["academy.course"].create({
    "name": "Odoo Developer Bootcamp",
    "code": "ODOO-B00T",
})
```

Common Mistakes

- Assuming self always contains exactly one record.
- Looping over ids and performing one search per id.
- Using direct SQL for changes that need access rules or recomputation.
- Calling browse and then assuming the record exists in the database.

Lab / Practice: In Odoo shell, create three courses, batch update them with write, duplicate one, and check exists on a deleted id.

O21 Override create and write

Learning Objectives

- Override create with `@api.model_create_multi`.
- Override write without breaking batch behavior.
- Call super and preserve the inheritance chain.

Explanation

`create` and `write` are the most common lifecycle overrides in custom modules. They are also easy to damage because they sit on hot paths used by imports, forms, automated actions, and other modules.

In Odoo 17, `create` should normally use `@api.model_create_multi` so one call can create several records efficiently. Code that assumes a single `vals` dictionary can slow imports or break batch creation.

`write` receives one `vals` dictionary applied to every record in `self`. If behavior depends on old values, capture them before super because after super the recordset reflects the new data.

Always call super unless you are intentionally replacing core behavior, which is rare. The inheritance chain may include mail tracking, website behavior, accounting rules, and other installed modules.

Validation in `create` or `write` should be targeted and explainable. Prefer constraints for invariant rules and use overrides for workflow side effects, sequence assignment, or integration hooks.

Avoid writing to the same records inside `write` without guards because recursion is easy. If a follow-up update is required, make it explicit and ensure it will not call itself forever.

Key Points

- Use `@api.model_create_multi` for create overrides.
- `write` applies `vals` to every record in `self`.
- Capture old values before super when needed.
- Call super to preserve installed module behavior.

Safe create and write overrides

The `create` override supports batch creation, and `write` captures old state before calling super.

```
from odoo import api, fields, models

class AcademyCourse(models.Model):
    _name = "academy.course"

    name = fields.Char(required=True)
    code = fields.Char(copy=False)
    published_at = fields.Datetime(copy=False)
    state = fields.Selection([("draft", "Draft"), ("published", "Published")], default="draft")

    @api.model_create_multi
    def create(self, vals_list):
        for vals in vals_list:
            vals.setdefault("code", self.env["ir.sequence"].next_by_code("academy.course"))
        return super().create(vals_list)

    def write(self, vals):
        old_states = {record.id: record.state for record in self}
        result = super().write(vals)
        if vals.get("state") == "published":
            to_stamp = self.filtered(lambda record: old_states.get(record.id) != "published")
            to_stamp.write({"published_at": fields.Datetime.now()})
        return result
```

Common Mistakes

- Using `@api.model` and a single `vals` dict for batch create code.
- Forgetting to return the result of super.

- Reading old values after super and losing the transition information.
- Creating recursive writes without guards.

Lab / Practice: Override create to assign a sequence and write to stamp a publication datetime only when the state first becomes published.

O22 Custom Methods, copy, and unlink

Learning Objectives

- Write business methods that operate on recordsets.
- Override copy to reset duplicated values.
- Override unlink to protect important records.

Explanation

Custom methods should express business actions in the language of the module. Button handlers such as `action_confirm` or `action_archive` are clearer than placing workflow logic directly in views.

Methods called by buttons often require one record, but batch actions may pass many. Use `ensure_one` only when the method truly cannot operate on multiple records.

`copy` duplicates a record and lets you provide default values for the duplicate. Reset fields such as state, sequence code, external references, and counters that should not be cloned.

`unlink` deletes records and should be restricted when records have business history. In many Odoo workflows, archiving with `active=False` is safer than permanent deletion.

Raise `UserError` or `ValidationError` with messages that help the user fix the problem. A stack trace or generic access error is not a good business experience.

Keep custom methods small enough to test. If an action performs validation, state change, notification, and external API calls, split helper methods around clear responsibilities.

Key Points

- Custom methods should express business actions.
- Use `ensure_one` only for truly single-record actions.
- Override `copy` to reset duplicated business identifiers.
- Prefer `archive` over `unlink` for historical business records.

Business methods and deletion protection

The methods implement a publish action, safe duplication defaults, and deletion protection for published courses.

```
from odoo import _, models
from odoo.exceptions import UserError

class AcademyCourse(models.Model):
    _name = "academy.course"

    def action_publish(self):
        for course in self:
            if not course.session_ids:
                raise UserError(_("Add at least one session before publishing."))
            self.write({"state": "published"})

    def copy(self, default=None):
        default = dict(default or {})
        default.update({"state": "draft", "code": False})
        return super().copy(default)

    def unlink(self):
        if any(course.state == "published" for course in self):
            raise UserError(_("Published courses cannot be deleted. Archive them instead."))
        return super().unlink()
```

Common Mistakes

- Calling `ensure_one` at the top of every method by habit.
- Letting `copy` duplicate unique business references.
- Deleting records that should remain for audit history.

- Raising technical exceptions that do not guide the user.

Lab / Practice: Add a publish button method, prevent deletion of published records, and verify duplicate records return to draft state.

O23 exists, search, and Domain Operators

Learning Objectives

- Use exists to clean recordsets.
- Write search domains with common operators.
- Combine domain logic with &, |, and !.

Explanation

search is the main ORM method for finding records. It accepts a domain, optional offset, limit, order, and count behavior through related methods.

A domain is a list representation of logical conditions. Each tuple contains a field, an operator, and a value, while prefix operators such as | and ! combine expressions.

Common operators include =, !=, in, not in, like, ilike, child_of, >=, <=, and =?. The right operator makes code more readable and can change whether indexes are useful.

exists returns the subset of a recordset that still exists and is accessible. It is useful after browse on external ids, user input, or records that may have been deleted by another transaction.

Domains are translated to SQL with ORM context, access rights, active_test, and company behavior in mind. A domain is not just a string filter; it is part of Odoo's security-aware data access layer.

Complex domains should be named or built in helper methods. A long inline domain copied across models is a maintenance risk and often hides subtle logic differences.

Key Points

- search takes a domain and returns a recordset.
- Domain operators describe SQL-like comparisons and Odoo hierarchy logic.
- Prefix | means OR, & means AND, and ! means NOT.
- exists filters out missing records from a recordset.

Search domains

The examples show normal AND behavior, explicit OR behavior, ordering, limiting, and existence cleanup.

```
Course = self.env["academy.course"]

published = Course.search([
    ("active", "=", True),
    ("state", "=", "published"),
    ("name", "ilike", "odoo"),
], order="name", limit=20)

draft_or_expensive = Course.search([
    "|",
    ("state", "=", "draft"),
    ("fee", ">=", 1000),
])

records = Course.browse([1, 2, 999999]).exists()
```

Common Mistakes

- Building domains with string concatenation instead of domain lists.
- Forgetting that active_test may hide inactive records.
- Using ilike for every search even when exact equality is intended.
- Browsing arbitrary ids and assuming all records exist.

Lab / Practice: Write three domains: exact state, OR condition, and inactive-inclusive search using with_context(active_test=False).

O24 search_count, read, read_group, and search_read

Learning Objectives

- Choose the right read method for lists, counts, and aggregates.
- Use read_group for grouped reporting.
- Avoid loading unnecessary fields or records.

Explanation

Not every data question needs full recordsets with every field prefetched. Odoo provides specialized methods for counts, raw reads, combined search-and-read, and grouped aggregation.

search_count returns the number of records matching a domain without fetching all records. It is ideal for badges, warnings, and decisions that only need a count.

read returns dictionaries for fields on an existing recordset. It is useful for RPC-like output or integration code that needs simple structures rather than record methods.

search_read combines search and read in one call and is common in controllers and API-style endpoints. It should still use fields, limit, and domain deliberately.

read_group performs grouped aggregation similar to SQL GROUP BY while respecting ORM access and context. It is useful for dashboards and summary widgets.

Performance depends on shape. Fetching 10 fields from 80 records is different from fetching 80 fields from 10,000 records, so be precise about fields and limits.

Key Points

- Use search_count for counts.
- Use read for dictionaries from known records.
- Use search_read for search plus field extraction.
- Use read_group for grouped aggregates.

Reading and grouping

The methods answer count, list, and grouped reporting questions without loading more data than needed.

```
Course = self.env["academy.course"]

draft_count = Course.search_count([("state", "=", "draft")])

rows = Course.search_read(
    [("state", "=", "published")],
    fields=["name", "code", "fee"],
    limit=10,
    order="name",
)

summary = Course.read_group(
    domain=[("active", "=", True)],
    fields=["fee:sum", "id:count"],
    groupby=["state"],
)
```

Common Mistakes

- Using len(search(domain)) instead of search_count for large datasets.
- Calling search_read without a limit in controller code.
- Expecting read_group output to look exactly like SQL rows.
- Fetching all fields when only names and ids are required.

Lab / Practice: Build a small dashboard method that returns draft count, the ten latest published courses, and fees grouped by state.

O25 name_create, default_get, _name_search, and get_view

Learning Objectives

- Customize quick creation and default values.
- Improve Many2one search behavior with `_name_search`.
- Understand when `get_view` customization is appropriate.

Explanation

`name_create` supports quick creation from relational widgets. It should create a valid minimal record, not bypass required business setup that the user must provide.

`default_get` computes default values for new records based on fields, context, company, user, and business rules. It is a good place for defaults that need more logic than a field declaration.

`_name_search` controls how records are found when users type in Many2one fields. It should return ids matching the user's text across meaningful fields such as code and name.

Search customization must respect access rules and performance. A convenient `_name_search` that scans large text fields can slow every form that uses the model.

`get_view` can modify view architecture before it reaches the client. It is powerful but should be used sparingly because dynamic XML manipulation is harder to reason about than normal view inheritance.

Prefer declarative XML view inheritance and context defaults first. Use these hooks when the behavior is model-level and cannot be expressed cleanly in standard configuration.

Key Points

- `name_create` handles quick creation from relational fields.
- `default_get` returns context-aware defaults.
- `_name_search` improves lookup behavior.
- `get_view` is powerful and should be rare.

Lookup and defaults hooks

The model supports quick creation, company-aware defaults, and lookup by either course name or code.

```
from odoo import api, models

class AcademyCourse(models.Model):
    _name = "academy.course"
    _rec_name = "name"

    @api.model
    def name_create(self, name):
        record = self.create({"name": name, "code": name.upper().replace(" ", "-")[:32]})
        return record.name_get()[0]

    @api.model
    def default_get(self, fields_list):
        values = super().default_get(fields_list)
        if "company_id" in fields_list:
            values["company_id"] = self.env.company.id
        return values

    @api.model
    def _name_search(self, name="", args=None, operator="ilike", limit=100, name_get_uid=None):
        args = list(args or [])
        domain = ["|", ("name", operator, name), ("code", operator, name)] if name else []
        return self._search(domain + args, limit=limit, access_rights_uid=name_get_uid)
```

Common Mistakes

- Using `name_create` to create incomplete records that later fail business rules.
- Overriding `default_get` and ignoring the requested `fields_list`.

- Writing `_name_search` domains that ignore args passed by the caller.
- Using `get_view` for changes that normal XML inheritance can handle.

Lab / Practice: Make `academy.course` searchable by code in a Many2one field and add a default company through `default_get`.

O26 ensure_one, filtered, mapped, sorted, grouped, fields_get, and get_metadata

Learning Objectives

- Use recordset helper methods idiomatically.
- Inspect model metadata from the ORM.
- Avoid unnecessary loops and single-record assumptions.

Explanation

Recordset helpers make Odoo code concise and expressive. They operate on recordsets and usually preserve ORM behavior better than manually building lists of ids.

ensure_one asserts that a method is working with exactly one record. It is appropriate for single-record actions and computed display helpers, but it should not be used to avoid thinking about batch behavior.

filtered keeps records matching a predicate or field expression, mapped extracts fields or calls, sorted orders records, and grouped partitions records by a key. These methods make business code read closer to the business question.

fields_get returns field metadata such as type, string, required, readonly, selection, and help. It is useful for dynamic UIs, debugging, and training exercises about what Odoo knows about a model.

get_metadata returns technical metadata such as XML ids and creation or update information. It helps when diagnosing data loaded from XML or deciding whether a record is safe to reference externally.

Use helpers on reasonably sized recordsets. For very large datasets, push filtering and grouping into search domains or read_group so PostgreSQL does the heavy work.

Key Points

- ensure_one is an assertion, not a default habit.
- filtered, mapped, sorted, and grouped express recordset transformations.
- fields_get exposes field metadata.
- get_metadata helps trace XML ids and record provenance.

Recordset helpers

The code shows common helpers and a correct single-record action that uses ensure_one intentionally.

```
courses = self.env["academy.course"].search([])

published = courses.filtered(lambda course: course.state == "published")
names = published.mapped("name")
ordered = published.sorted(key=lambda course: course.fee, reverse=True)
by_state = courses.grouped("state")

field_info = self.env["academy.course"].fields_get(["name", "state", "fee"])
metadata = published[:1].get_metadata()

def action_open_form(self):
    self.ensure_one()
    return {
        "type": "ir.actions.act_window",
        "res_model": self._name,
        "res_id": self.id,
        "view_mode": "form",
    }
```

Common Mistakes

- Calling ensure_one in methods that could naturally support batch actions.
- Using filtered on thousands of records when a search domain would be better.
- Assuming mapped always returns a recordset; scalar fields return Python lists.
- Ignoring metadata tools and hardcoding guesses about fields or XML ids.

Lab / Practice: Use Odoo shell to filter published courses, map their names, group by state, and print fields_get metadata for three fields.

O27 search_fetch and Efficient Data Loading

Learning Objectives

- Use search_fetch when records and selected fields are needed together.
- Understand prefetching and field loading behavior.
- Reduce query chatter in read-heavy custom code.

Explanation

Odoo's ORM uses caching and prefetching to reduce repetitive database access. Most normal code benefits automatically, but high-volume loops can still create avoidable query chatter.

search_fetch searches for records and fetches specified fields in one deliberate operation. It is useful when the next code path will read a known set of fields from every returned record.

The method keeps recordset semantics while making field-loading intent explicit. This can be cleaner than search followed by manual reads when the code still wants record methods.

Do not use search_fetch everywhere by habit. If code only needs ids, search may be enough; if it needs dictionaries for an API response, search_read may be clearer.

Performance work should be measured with logs, tests, or query counters when possible. Odoo caches can make a change look faster in a warm shell while offering little benefit in a real request.

Efficient loading is especially valuable in computed fields, scheduled actions, exports, and controllers. These paths often touch many records and can amplify small ORM mistakes.

Key Points

- search_fetch combines search with deliberate field fetching.
- Use it when you need a recordset and known fields immediately.
- search_read is better when dictionaries are the desired output.
- Measure performance changes before making broad refactors.

Fetch fields for a batch job

The batch job declares the fields it will use before looping over the recordset.

```
def _send_course_digest(self):
    courses = self.env["academy.course"].search_fetch(
        [("state", "=", "published")],
        ["name", "code", "fee", "currency_id"],
        limit=500,
        order="name",
    )
    for course in courses:
        body = "%s [%s] costs %s" % (course.name, course.code, course.fee)
        self.env["mail.mail"].create({
            "subject": "Course digest",
            "body_html": body,
            "email_to": "training@example.com",
        })
```

Common Mistakes

- Using search_fetch when only a count is needed.
- Fetching many fields just in case.
- Optimizing without measuring query count or runtime.
- Replacing clear search_read API code with recordset code that callers do not need.

Lab / Practice: Write one method with search and one with search_fetch for the same field access pattern, then compare SQL logs in a small benchmark.

O28 Special Relational Commands 0 through 6

Learning Objectives

- Use Odoo's relational command tuples correctly.
- Apply commands to One2many and Many2many fields.
- Prefer Command helpers for readable Odoo 17 code.

Explanation

Relational commands describe changes to One2many and Many2many fields in create and write calls. They let one ORM call create, update, link, unlink, clear, or replace related records.

The classic tuple commands are numbered 0 through 6. They are compact but easy to misread, so Odoo's Command helper is preferred in modern code for clarity.

Command 0 creates a new related record, command 1 updates an existing related record, and command 2 deletes the related record. For One2many, these are common when editing line items.

Command 3 unlinks a relation without deleting the target record, command 4 links an existing record, command 5 clears all links, and command 6 replaces all links with a given id list. Many2many fields use 3, 4, 5, and 6 frequently.

Be careful with command 2 because it deletes the related record, not just the relationship. On business documents, that distinction can affect audit history and accounting integrity.

Relational commands should be built from validated data. Do not accept arbitrary command lists from untrusted external clients without checking allowed operations and target records.

Key Points

- 0 creates, 1 updates, 2 deletes.
- 3 unlinks, 4 links, 5 clears, 6 replaces.
- Use `odoo.fields.Command` for readable code.
- Delete and unlink are different operations.

Command helper examples

The helper names make the intent clearer than raw numbers while still producing the same ORM commands.

```
from odoo.fields import Command

course = self.env["academy.course"].create({
    "name": "Advanced Odoo",
    "session_ids": [
        Command.create({"name": "ORM Deep Dive"}),
        Command.create({"name": "Performance Lab"}),
    ],
    "tag_ids": [
        Command.link(existing_tag_id),
    ],
})

course.write({
    "tag_ids": [Command.set([tag_a.id, tag_b.id])],
    "session_ids": [
        Command.update(course.session_ids[0].id, {"name": "ORM Masterclass"}),
    ],
})

# Classic tuple equivalent for replacing many2many links:
course.write({"tag_ids": [(6, 0, [tag_a.id, tag_b.id])])})
```

Common Mistakes

- Using command 2 when command 3 was intended.
- Replacing all Many2many links with command 6 when the code meant to add one link.
- Passing raw external command lists into write without validation.

- Forgetting that One2many requires values compatible with the child model.

Lab / Practice: Create a course with two sessions using `Command.create`, add two tags with `Command.set`, then remove one tag with `Command.unlink`.

O29 Active Archive Behavior

Learning Objectives

- Use the active field for archiving records.
- Understand active_test context in searches and relational fields.
- Choose archive or unlink based on business history.

Explanation

Many Odoo models use an active Boolean to archive records instead of deleting them. Archived records remain in the database but are hidden from normal searches and views.

The ORM context key active_test controls whether inactive records are filtered automatically. By default, active_test is usually true, so search results may exclude archived records without an explicit domain.

Archiving is safer for records referenced by historical documents. A partner, product, course, or analytic account may no longer be selectable for new work but still needs to appear on old records.

Views often provide Archive and Unarchive actions automatically when a model has active. This creates a consistent user experience across custom and standard models.

Search domains that explicitly mention active can interact with active_test in confusing ways. When diagnosing archived data, use with_context(active_test=False) and then filter intentionally.

Do not use active as a substitute for workflow state. Active means visible or usable, while state represents where a record is in a business process.

Key Points

- active=False archives records.
- active_test controls automatic filtering of inactive records.
- Archive preserves history better than unlink.
- active is not a workflow state field.

Archive-aware searches

The lookup includes archived records deliberately, while the archive action hides completed courses without deleting them.

```
class AcademyCourse(models.Model):
    _name = "academy.course"

    active = fields.Boolean(default=True)

    def find_any_course(self, code):
        return self.with_context(active_test=False).search([("code", "=", code)], limit=1)

    def action_archive_old_courses(self):
        old_courses = self.search([
            ("state", "=", "done"),
            ("active", "=", True),
        ])
        old_courses.write({"active": False})
```

Common Mistakes

- Deleting historical records instead of archiving them.
- Forgetting active_test=False when searching for archived records.
- Using active to represent draft, confirmed, and done workflow states.
- Adding custom archive logic when the standard active behavior is enough.

Lab / Practice: Add active to academy.course, archive one record, prove normal search hides it, and prove active_test=False finds it.

O30 Environment self.env and Modifying Environment

Learning Objectives

- Use self.env to access models, user, company, context, and cursor.
- Modify environment with sudo, with_context, with_user, with_company, and with_env.
- Understand the security impact of environment changes.

Explanation

self.env is the gateway to the Odoo runtime. It carries the database cursor, current user, context, registry, company information, cache, and model accessors.

self.env['model.name'] returns a model recordset bound to the same environment. This is how custom code reaches other models without importing their Python classes directly.

with_context creates a new recordset with modified context. Context can drive defaults, language, timezone, active_test, company behavior, and many model-specific flags.

sudo changes access rights by using superuser mode or a specific user. It is necessary for some technical operations, but it can bypass record rules and expose or modify data the current user should not touch.

with_user and with_company are more explicit tools when behavior should be evaluated as another user or company. They are useful in tests, multi-company logic, and controlled automation.

Environment changes return new recordsets; they do not mutate every existing variable. A senior developer keeps the elevated or modified environment as narrow as possible and returns to normal permissions quickly.

Key Points

- self.env gives access to models, cursor, user, company, and context.
- with_context changes contextual behavior for a recordset.
- sudo bypasses normal access checks and must be narrow.
- with_user and with_company make user and company changes explicit.

Environment patterns

The method shows context changes, narrow sudo usage, company-scoped work, and model access through self.env.

```
def action_prepare_digest(self):
    Mail = self.env["mail.mail"]
    lang = self.env.user.lang or "en_US"

    courses = self.with_context(active_test=False, lang=lang).search([
        ("state", "=", "published"),
    ])

    admin_course_count = self.sudo().search_count([])

    company_courses = self.with_company(self.env.company).search([
        ("company_id", "=", self.env.company.id),
    ])

    Mail.sudo().create({
        "subject": "Course digest",
        "body_html": "Published: %s, total visible to admin: %s" % (
            len(courses),
            admin_course_count,
        ),
        "email_to": self.env.user.email,
    })
    return company_courses
```

Common Mistakes

- Using sudo everywhere instead of fixing access rights or record rules.
- Forgetting that with_context returns a new recordset.
- Using self.env.cr for direct SQL when ORM methods would be safer.

- Mixing companies in code without explicit `with_company` or company domains.

Lab / Practice: In Odoo shell, print `env.user`, `env.company`, and `env.context`, then compare search results with normal access, `sudo`, and `active_test=False`.

Part F Views, Security, Data & QWeb

This part contains 28 detailed lessons. Work in order. Complete each lab before moving forward when the lab unlocks the next dependency.

Presentation: Companion deck: see `training_materials/presentations/` for the Views, Security, Data & QWeb slides.

VS01 Intro to Odoo Views and XML Data Loading

Learning Objectives

- Explain how view records are stored and loaded.
- Choose reliable external ids for view and data records.
- Describe the relationship between model fields and UI architecture.

Explanation

Intro to Odoo Views and XML Data Loading is best understood as part of the contract between the ORM, the web client, and the business user. Odoo views are not templates copied into the browser; they are records in `ir.ui.view` that the server combines, validates, and sends to the web client. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Start from the model and workflow, then select the smallest view types needed to support the workflow instead of creating every possible view. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Declare XML under a data file, reference existing models through `ref`, and keep view ids stable because other modules and upgrades depend on them. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Install with demo data disabled and enabled, update the module, and confirm the view exists in developer mode with the expected inherited architecture. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor intro to odoo views and xml data loading decision appears during upgrades, audits, and support tickets. A predictable data-loading strategy makes migrations easier because broken XML ids and duplicated records are among the most common deployment failures. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- A view is an `ir.ui.view` record with arch XML.
- External ids are part of the module API.
- Data files load in manifest order.
- Views should represent workflows, not database tables only.
- Developer mode is essential for inspecting final architecture.

Minimal window action and menu

The action defines the model and ordered view modes; the menu only points users to that action.

```
<odoo>
  <record id="training_course_action" model="ir.actions.act_window">
    <field name="name">Courses</field>
    <field name="res_model">training.course</field>
    <field name="view_mode">tree,form,kanban</field>
    <field name="context">{'search_default_active': 1}</field>
  </record>

  <menuitem id="training_menu_root" name="Training"/>
  <menuitem id="training_menu_courses"
    name="Courses"
    parent="training_menu_root"
    action="training_course_action"/>
</odoo>
```

Common Mistakes

- Changing external ids after another module inherits them.
- Loading views before the model fields exist.
- Putting business rules only in XML instead of model methods.
- Forgetting that manifest data file order is significant.

Lab / Practice: Create a model action, menu, tree view, and form view for a small training.course model, then update the module twice to verify idempotency.

VS02 List Views, Sorting, and Limits

Learning Objectives

- Build a readable list view for operational records.
- Use `default_order` and `limit` intentionally.
- Select list columns that support the primary user decision.

Explanation

List Views, Sorting, and Limits is best understood as part of the contract between the ORM, the web client, and the business user. The list view is usually the first screen where users scan volume, compare records, and decide what needs attention. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Choose columns by asking what a user must know before opening a form; do not mirror every field in the model. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use `default_order` on indexed or naturally sortable fields and reserve limit for dashboards or queues where a short list is part of the design. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Check list load time with realistic data, confirm sorting is stable, and verify that invisible technical fields are still available when used by decorations or buttons. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor list views, sorting, and limits decision appears during upgrades, audits, and support tickets. Poor list views create hidden operational cost because every extra column can increase read payloads and every confusing order creates manual work. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Tree views are now commonly referred to as list views in newer Odoo discussions.
- `default_order` belongs on the view arch.
- `limit` changes the initial row count for that view.
- Columns should reflect the user decision.
- Decorations and modifiers may require invisible helper fields.

Operational list with stable ordering

The list opens with recent courses first and only includes fields needed to triage the course catalog.

```
<record id="training_course_view_tree" model="ir.ui.view">
  <field name="name">training.course.tree</field>
  <field name="model">training.course</field>
  <field name="arch" type="xml">
    <tree string="Courses" default_order="start_date desc, name" limit="80">
      <field name="name"/>
      <field name="teacher_id"/>
      <field name="start_date"/>
      <field name="seat_count"/>
      <field name="state"/>
    </tree>
  </field>
</record>
```

Common Mistakes

- Using a non-indexed computed field as the default sort on a large table.
- Adding every field because it is available.
- Using `limit` to hide bad performance rather than fixing the cause.
- Forgetting multi-company or active filters when reviewing list content.

Lab / Practice: Replace a wide list view with a decision-focused list, add `default_order`, and compare the generated read fields in the browser network tab.

VS03 Editable Lists, Mass Editing, and the Handle Widget

Learning Objectives

- Enable safe inline edits for simple records.
- Understand when mass editing is appropriate.
- Use handle widgets for sequence-based ordering.

Explanation

Editable Lists, Mass Editing, and the Handle Widget is best understood as part of the contract between the ORM, the web client, and the business user. Editable lists reduce navigation when users are maintaining line-like records, but they also reduce the friction that normally makes users review a form. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Only make fields editable when validation is local and obvious; complex onchange chains, attachments, chatter, or approvals usually deserve a form. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use `editable='bottom'` for controlled creation, `multi_edit='1'` for deliberate mass changes, and `widget='handle'` with an integer sequence field for manual ordering. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Test create, edit, discard, required fields, access rights, and record rules with a non-admin user because inline editing exposes write paths quickly. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor editable lists, mass editing, and the handle widget decision appears during upgrades, audits, and support tickets. A safe editable list feels fast without bypassing business rules because constraints, onchange logic, and access rights still execute server-side. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Inline editing is a workflow choice, not a default.
- `multi_edit` applies one value to many selected rows.
- The handle widget requires an orderable sequence field.
- Server constraints remain mandatory.
- Editable lists are risky on approval records.

Editable line list with sequence handle

Users can reorder and maintain simple training lines without opening each form, while required fields remain enforced.

```
<tree editable="bottom" multi_edit="1" default_order="sequence, id">
  <field name="sequence" widget="handle"/>
  <field name="name" required="1"/>
  <field name="duration_hours"/>
  <field name="price"/>
</tree>
```

Common Mistakes

- Enabling editable lists on records with complex workflow transitions.
- Forgetting to include sequence in `_order` on the model.
- Assuming mass edit bypasses access rights.
- Relying on onchange instead of constraints for critical validation.

Lab / Practice: Add an editable list to course session lines, include a handle sequence, and prove that a negative duration is blocked by a Python constraint.

VS04 List Decorations, Field Decorations, Widths, and Hidden Labels

Learning Objectives

- Apply visual decorations without changing data semantics.
- Use invisible fields as decoration inputs.
- Tune field width and labels for dense operational screens.

Explanation

List Decorations, Field Decorations, Widths, and Hidden Labels is best understood as part of the contract between the ORM, the web client, and the business user. Decorations are a fast way to turn a list into an exception queue, but they should reinforce a business meaning already present in the data. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Design the visual language first: one color for danger, one for success, and a small number of badges so users do not learn to ignore noise. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use row decoration attributes, field-level decorations, optional fields, width hints, and nolabel where they clarify the screen without hiding important data. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Review decorations with sample records in every state and confirm helper fields are present in the view when expressions need them. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor list decorations, field decorations, widths, and hidden labels decision appears during upgrades, audits, and support tickets. Visual rules age badly when they are undocumented, so connect decoration expressions to stable state values rather than fragile names or dates embedded in text. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Row decorations use expressions evaluated by the client.
- Field decorations work well on status or KPI fields.
- Invisible helper fields can feed modifiers and decorations.
- Width hints are guidance, not a security feature.
- Do not encode business rules only as colors.

Exception-oriented decorated list

The list highlights cancellation, under-filled courses, and completion while keeping the comparison field available invisibly.

```
<tree decoration-danger="state == 'cancel'"
      decoration-warning="seat_count < minimum_seats"
      decoration-success="state == 'done'">
  <field name="name" width="240px"/>
  <field name="minimum_seats" invisible="1"/>
  <field name="seat_count" decoration-warning="seat_count < minimum_seats"/>
  <field name="state" widget="badge"/>
</tree>
```

Common Mistakes

- Using many colors for unrelated meanings.
- Forgetting XML escaping for comparison operators.
- Treating invisible fields as secure.

- Making labels disappear when the field meaning is not obvious.

Lab / Practice: Create three sample records that trigger danger, warning, and success decorations, then ask another user to explain the meaning without reading the XML.

VS05 List Totals, Averages, and Grouped Lists

Learning Objectives

- Use sum and avg footers on numeric fields.
- Understand how grouped list aggregations help operations.
- Avoid misleading totals on filtered data.

Explanation

List Totals, Averages, and Grouped Lists is best understood as part of the contract between the ORM, the web client, and the business user. Totals and averages make a list view useful for reconciliation, capacity planning, and quick financial checks. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Decide whether a metric should be summed, averaged, counted, or left out; not every numeric field communicates meaningful aggregate information. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Add sum and avg labels directly on fields, combine with groupby where appropriate, and ensure computed fields are stored if aggregation must happen in SQL. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Validate totals under filters, groups, multi-company access, and currency contexts so users know exactly what population is being summarized. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor list totals, averages, and grouped lists decision appears during upgrades, audits, and support tickets. Production incidents often happen when users export or approve based on a subtotal they misunderstood, so the label should name the business measure precisely. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- sum and avg attributes create list footers.
- Grouped lists aggregate per group and overall.
- Stored fields are safer for SQL aggregation.
- Currency and unit of measure context matters.
- Totals reflect the current domain and access rules.

Capacity and revenue footers

The footer gives immediate operational totals without requiring an export to a spreadsheet.

```
<tree default_order="start_date desc">
  <field name="name"/>
  <field name="teacher_id"/>
  <field name="seat_count" sum="Total Seats"/>
  <field name="price_total" sum="Total Revenue"/>
  <field name="satisfaction_score" avg="Average Score"/>
</tree>
```

Common Mistakes

- Summing percentages or scores that should be averaged.
- Aggregating non-stored computed fields that cannot be grouped efficiently.
- Ignoring currency mixing across companies.
- Letting totals imply a complete population when filters are active.

Lab / Practice: Add sum and average footers to a list, group by teacher, and compare grouped totals against a manual `read_group` call in the Odoo shell.

VS06 Search Views, Default Filters, and Dynamic Date Filters

Learning Objectives

- Build search filters that match daily work.
- Set default filters from actions.
- Use dynamic datetime domains safely.

Explanation

Search Views, Default Filters, and Dynamic Date Filters is best understood as part of the contract between the ORM, the web client, and the business user. A search view turns a generic model list into a work queue, a reporting entry point, or a compliance review screen. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Start with the questions users ask every day, then encode those questions as filters, separators, date filters, and search fields with understandable labels. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Declare filters in the search view, pass `search_default` keys from the action context, and use `context_today` or relative datetime helpers carefully in domains. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Verify defaults from every menu and action that opens the model because the same search view may serve multiple workflows with different assumptions. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor search views, default filters, and dynamic date filters decision appears during upgrades, audits, and support tickets. Dynamic filters are powerful during operations but risky during audits, so reports that require a fixed period should store or pass explicit dates. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Search filters are reusable domain snippets.
- `search_default_filter_name` activates a named filter.
- Dynamic dates are evaluated relative to context.
- Search fields should map to indexed or useful fields.
- Menus can share a search view with different contexts.

Default active and this-month filters

The search view offers direct text search, a reusable active filter, a dynamic date filter, and a group-by shortcut.

```
<search string="Courses">
  <field name="name"/>
  <field name="teacher_id"/>
  <filter name="active" string="Active" domain="[('active', '=', True)]"/>
  <filter name="this_month" string="Starts This Month"
    domain="[('start_date', '>=', context_today().strftime('%Y-%m-01'))]"/>
  <filter name="group_teacher" string="Teacher" context="{ 'group_by': 'teacher_id' }"/>
</search>
```

Common Mistakes

- Using dynamic filters for legally fixed reporting periods.
- Forgetting that action context controls default filters.
- Adding filters users cannot explain.
- Creating domains that ignore timezone expectations.

Lab / Practice: Create a menu that opens courses with Active and Starts This Month enabled by default, then add a second menu that opens the same model without those defaults.

VS07 Search Panel and Groupby UX

Learning Objectives

- Use search panels for guided filtering.
- Design groupby options that explain the data.
- Avoid filter combinations that create empty screens.

Explanation

Search Panel and Groupby UX is best understood as part of the contract between the ORM, the web client, and the business user. The search panel is a navigation aid for users who think in categories more than domains. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Use it when the model has meaningful many2one, many2many, selection, or hierarchy fields that users naturally use to narrow a dataset. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Define searchpanel fields separately from groupby filters and decide whether a user needs counters, multi-select, or category dependencies. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Test large datasets because counters and many category values can become expensive if the model or domains are not indexed well. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor search panel and groupby ux decision appears during upgrades, audits, and support tickets. A good search panel reduces training cost because new users discover the available dimensions without memorizing advanced search syntax. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Search panel fields guide filtering from the left side.
- Groupby filters change result organization, not just filtering.
- Counters can have performance cost.
- Hierarchies need clean parent relationships.
- Use labels that match business language.

Category and teacher search panel

The panel supports guided category and teacher filtering while groupby remains a separate analytical option.

```
<search string="Courses">
  <field name="name"/>
  <searchpanel>
    <field name="category_id" icon="fa-folder-open" enable_counters="1"/>
    <field name="teacher_id" select="multi" icon="fa-user"/>
  </searchpanel>
  <filter name="group_category" string="Category" context="{ 'group_by': 'category_id' }"/>
</search>
```

Common Mistakes

- Enabling counters on a model with slow domains without measuring.
- Duplicating every groupby as a search panel field.
- Using technical field names as labels.
- Assuming search panel visibility is a security boundary.

Lab / Practice: Add a search panel to a course model with categories and teachers, then measure the difference between counters enabled and disabled on sample data.

VS08 Form Customization with Groups, Tags, and Widgets

Learning Objectives

- Create clean form layouts with semantic grouping.
- Use tags and widgets where they improve comprehension.
- Keep forms aligned with business workflow.

Explanation

Form Customization with Groups, Tags, and Widgets is best understood as part of the contract between the ORM, the web client, and the business user. A form view is where the user understands and changes one business object, so layout communicates priority as much as labels do. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Group fields by decision area, place identity fields first, and use widgets such as `many2many_tags` only when the compact representation is easier to read. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use sheet, group, separator, field options, placeholder, widget, and help text in a way that supports scanning without turning the form into a dashboard. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Review the form at common screen sizes, with empty and populated data, and with readonly states so the layout does not collapse when fields disappear. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor form customization with groups, tags, and widgets decision appears during upgrades, audits, and support tickets. Stable form design reduces downstream customizations because partners can inherit predictable groups and users develop muscle memory. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Forms should tell a story about one record.
- sheet and group provide standard Odoo visual structure.
- `many2many_tags` is compact but can hide detail.
- Place technical fields only when they help the workflow.
- Readable anchors make later inheritance safer.

Structured course form

The form separates identity from delivery so users can understand the record before editing schedule details.

```
<form string="Course">
  <sheet>
    <group>
      <group string="Identity">
        <field name="name" placeholder="Functional Odoo Basics"/>
        <field name="category_id"/>
        <field name="tag_ids" widget="many2many_tags"/>
      </group>
      <group string="Delivery">
        <field name="teacher_id"/>
        <field name="start_date"/>
        <field name="duration_hours"/>
      </group>
    </group>
  </sheet>
</form>
```

Common Mistakes

- Making one enormous group with no visual hierarchy.
- Using tag widgets for records where users need columns or attributes.
- Hiding required context in tabs users rarely open.
- Letting inherited views depend on brittle positional anchors.

Lab / Practice: Redesign a flat form into two groups and one tag field, then inspect the final architecture after installing a second module that inherits it.

VS09 Notebook Pages, Button Containers, Header Buttons, and Statusbars

Learning Objectives

- Use notebook pages for secondary information.
- Place workflow buttons in the header.
- Represent state with a statusbar widget.

Explanation

Notebook Pages, Button Containers, Header Buttons, and Statusbars is best understood as part of the contract between the ORM, the web client, and the business user. The header and notebook areas turn a simple form into a workflow screen without requiring custom JavaScript. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Put state-changing actions where users expect workflow controls, and use notebook pages for details that matter after the primary identity is understood. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Implement object buttons for Python methods, action buttons for client actions or window actions, and `statusbar_visible` to show the intended lifecycle. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Verify every button in every state with access rights applied, because a visible button that fails server-side damages user trust. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor notebook pages, button containers, header buttons, and statusbars decision appears during upgrades, audits, and support tickets. A disciplined header prevents form sprawl: actions are discoverable, state is explicit, and secondary data stays organized. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Header buttons should map to business transitions.
- `statusbar` displays the lifecycle field.
- Notebook pages are for secondary or repeated information.
- Button visibility should match server-side rules.
- Use `button_box` for smart buttons, not random actions.

Workflow header and notebook

The header owns the lifecycle while the notebook separates operational details from notes.


```
<form string="Course">
  <header>
    <button name="action_confirm" type="object" string="Confirm"
      class="oe_highlight" invisible="state != 'draft'"/>
    <button name="action_done" type="object" string="Mark Done"
      invisible="state != 'confirmed'"/>
    <field name="state" widget="statusbar"
      statusbar_visible="draft,confirmed,done"/>
  </header>
  <sheet>
    <notebook>
      <page string="Sessions">
        <field name="session_ids"/>
      </page>
      <page string="Notes">
        <field name="internal_note"/>
      </page>
    </notebook>
  </sheet>
</form>
```

Common Mistakes

- Putting workflow buttons in random groups inside the form.
- Showing states in the statusbar that users cannot reach.
- Hiding buttons only in XML while Python methods remain too permissive.
- Using notebooks for required first-step data.

Lab / Practice: Add a draft-confirmed-done workflow with header buttons, then test every transition as a manager and as a read-only user.

VS10 Attrs Migration and Dynamic Readonly, Invisible, Required Rules

Learning Objectives

- Convert legacy attrs and states into modern modifier expressions.
- Use readonly, invisible, and required conditions safely.
- Design dynamic statusbar behavior without hiding business truth.

Explanation

Attrs Migration and Dynamic Readonly, Invisible, Required Rules is best understood as part of the contract between the ORM, the web client, and the business user. Odoo's view modifiers evolved from attrs dictionaries toward direct Python-like expressions in XML attributes. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Treat migration as a chance to simplify conditions rather than translating every old attrs expression mechanically. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Write direct invisible, readonly, and required expressions against fields loaded in the view, and keep dynamic statusbar_visible decisions tied to meaningful workflow states. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Compare the final architecture before and after migration and test records in each state because modifier errors often appear only with specific values. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor attrs migration and dynamic readonly, invisible, required rules decision appears during upgrades, audits, and support tickets. Clear modifier expressions survive upgrades better because reviewers can understand the condition without unpacking nested domain tuples. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Modern modifiers use direct expressions such as `invisible="state != 'draft'"`.
- Fields referenced by expressions must be available to the client.
- Required conditions should match model constraints where possible.
- Migration is a refactoring opportunity.
- Server-side validation remains authoritative.

Modern modifiers replacing attrs

The conditions are readable in place and no longer require a nested attrs dictionary.

```
<field name="cancel_reason"
      invisible="state != 'cancel'"
      required="state == 'cancel'"
      readonly="state in ('done', 'cancel')"/>
<field name="state" widget="statusbar"
      statusbar_visible="draft,confirmed,done,cancel"/>
```

Common Mistakes

- Migrating syntax without checking behavior in every state.
- Referencing a field not present in the view.
- Using invisible required fields without a server default.
- Relying on UI required rules instead of model constraints.

Lab / Practice: Take a legacy form using attrs, convert three modifiers to direct expressions, and run through draft, confirmed, done, and cancel examples.

VS11 Buttons that Open Forms, URL Actions, and Disabled Actions

Learning Objectives

- Return window actions from object buttons.
- Create URL actions for external systems.
- Disable buttons through state and group rules.

Explanation

Buttons that Open Forms, URL Actions, and Disabled Actions is best understood as part of the contract between the ORM, the web client, and the business user. Buttons connect views to behavior, so every button should make the next user step obvious and safe. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Choose object buttons for model methods, action buttons for existing actions, stat buttons for counters, and URL actions only when the target is trustworthy. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Return `ir.actions.act_window` dictionaries for opening forms, use `target='new'` for modal flows, and make visibility conditions mirror server-side checks. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Test buttons with missing related records, blocked access rights, and invalid states because button methods often become support hot spots. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor buttons that open forms, url actions, and disabled actions decision appears during upgrades, audits, and support tickets. The production rule is simple: a disabled or hidden button is only a usability hint, while Python access checks are the real control. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- `type='object'` calls a model method.
- Window actions can open a specific record form.
- URL actions should be generated from trusted values.
- Button visibility is not authorization.
- Use groups on buttons for role-specific actions.

Open the related partner form

The object button returns a precise form action instead of relying on a generic menu action.

```
def action_open_teacher_partner(self):
    self.ensure_one()
    return {
        "type": "ir.actions.act_window",
        "name": "Teacher",
        "res_model": "res.partner",
        "res_id": self.teacher_id.id,
        "view_mode": "form",
        "target": "current",
    }
```

Common Mistakes

- Building URLs from user input without validation.
- Hiding a button but leaving the method callable by any user.
- Returning an action without `res_id` when a specific form is expected.

- Using buttons for operations that should be computed automatically.

Lab / Practice: Add a smart button that opens the teacher partner and a separate URL action to documentation, then secure the object method with a group check.

VS12 Kanban Views, Templates, and Images

Learning Objectives

- Build kanban cards with QWeb templates.
- Render image fields efficiently.
- Choose card content that supports quick triage.

Explanation

Kanban Views, Templates, and Images is best understood as part of the contract between the ORM, the web client, and the business user. Kanban views are visual workspaces for records that benefit from cards, images, statuses, or stage-based scanning. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Design the card around the action users take next, not around showing every field in a compact rectangle. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Declare fields before templates, use `kanban_image` or image data URLs carefully, and keep card templates simple enough for inheritance. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Verify empty images, slow networks, mobile widths, and access to binary fields because kanban screens often become the first page users open. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor kanban views, templates, and images decision appears during upgrades, audits, and support tickets. A useful kanban card has strong information scent: a user can decide whether to open the record in a few seconds. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Kanban templates use QWeb under the kanban arch.
- Fields must be declared for use in templates.
- Images should have fallbacks.
- Cards should prioritize the next action.
- Keep templates easy to inherit.

Course kanban with image fallback

The card declares fields used by the QWeb template and uses the standard kanban image helper.

```
<kanban>
  <field name="name"/>
  <field name="teacher_id"/>
  <field name="image_128"/>
  <templates>
    <t t-name="kanban-box">
      <div class="oe_kanban_global_click o_kanban_record">
        
        <strong><field name="name"/></strong>
        <div><field name="teacher_id"/></div>
      </div>
    </t>
  </templates>
</kanban>
```

Common Mistakes

- Using undeclared fields inside the kanban template.
- Putting large binary images directly into crowded cards.

- Making every element clickable with unclear affordance.
- Forgetting fallback behavior when images are empty.

Lab / Practice: Create a kanban card with an image, title, teacher, and state badge, then test it with records that do and do not have images.

VS13 Kanban Drag and Drop, Handle Ordering, and Progressbars

Learning Objectives

- Enable stage-based drag and drop in kanban.
- Use sequence fields for manual ordering.
- Display progressbar summaries on kanban columns.

Explanation

Kanban Drag and Drop, Handle Ordering, and Progressbars is best understood as part of the contract between the ORM, the web client, and the business user. Kanban becomes operational when records can move between stages and users can see column-level progress. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Only enable drag and drop when a move is a valid business transition; otherwise the UI invites users to bypass workflow thinking. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use `default_group_by` for stages, a handle sequence for ordering, and progressbar definitions for quick aggregated status. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Test dragging with record rules, state constraints, and onchange side effects because kanban writes happen quickly from the client. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor kanban drag and drop, handle ordering, and progressbars decision appears during upgrades, audits, and support tickets. Production kanban design must balance speed and control: the faster the drag gesture, the more reliable the server-side transition rules must be. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- `default_group_by` creates a column-oriented kanban.
- Drag and drop writes the grouped field.
- Handle ordering needs a sequence field.
- Progressbars summarize grouped records.
- Server constraints must protect invalid moves.

Stage kanban with progressbar

The kanban groups by stage, allows drag ordering, and visualizes state distribution per column.

```
<kanban default_group_by="stage_id" records_draggable="1">
  <field name="stage_id"/>
  <field name="sequence"/>
  <field name="state"/>
  <progressbar field="state"
    colors="{ 'draft': 'muted', 'confirmed': 'warning', 'done': 'success' }"/>
  <templates>
    <t t-name="kanban-box">
      <div class="oe_kanban_global_click">
        <field name="sequence" widget="handle"/>
        <strong><field name="name"/></strong>
      </div>
    </t>
  </templates>
</kanban>
```

Common Mistakes

- Allowing drag and drop when state transitions require approvals.
- Forgetting access rules on the grouped field.
- Using progressbar colors with states that do not exist.
- Assuming client drag behavior replaces server validation.

Lab / Practice: Build a stage kanban, add a constraint that blocks moving done records back to draft, and verify the drag is rejected.

VS14 Graph, Pivot, Cohort, and Gantt Views

Learning Objectives

- Select the correct analytical view for the question.
- Configure measures and intervals.
- Understand enterprise-only planning views such as cohort and gantt.

Explanation

Graph, Pivot, Cohort, and Gantt Views is best understood as part of the contract between the ORM, the web client, and the business user. Analytical views answer different questions: graph shows shape, pivot supports exploration, cohort tracks retention, and gantt plans time. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Start from the business question and expected measure, then choose the view type rather than adding every reporting view automatically. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Declare measures, row and column fields, date intervals, and dependencies with stored fields so the database can aggregate predictably. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Validate analytical views with known fixtures and compare totals against `read_group` or SQL because a beautiful chart can still be wrong. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor graph, pivot, cohort, and gantt views decision appears during upgrades, audits, and support tickets. In production, reporting views become decision tools, so access rules, date ranges, and measure definitions must be as carefully reviewed as forms. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Graph views visualize aggregated measures.
- Pivot views support exploratory analysis.
- Cohort views track behavior over periods.
- Gantt views schedule records across dates.
- Stored fields make reporting reliable.

Revenue pivot and graph

The pivot supports analysis by teacher and month, while the graph shows capacity trends.

```
<pivot string="Course Revenue">
  <field name="teacher_id" type="row"/>
  <field name="start_date" interval="month" type="col"/>
  <field name="price_total" type="measure"/>
</pivot>

<graph string="Seats by Month" type="bar">
  <field name="start_date" interval="month"/>
  <field name="seat_count" type="measure"/>
</graph>
```

Common Mistakes

- Using non-stored computed measures.
- Mixing unrelated measures in one graph.
- Forgetting enterprise availability for cohort and gantt.

- Ignoring access-rule effects on reported totals.

Lab / Practice: Create graph and pivot views for courses, seed five months of data, and verify one total manually with `read_group`.

VS15 Security Overview: Six Components, Groups, Categories, and Hierarchy

Learning Objectives

- Describe the main layers of Odoo security.
- Organize groups under useful application categories.
- Explain parent-child group relationships to functional users.

Explanation

Security Overview: Six Components, Groups, Categories, and Hierarchy is best understood as part of the contract between the ORM, the web client, and the business user. Odoo security is layered: model access rights, record rules, groups, field/view/action group restrictions, sudo boundaries, and business-method checks all matter. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Design roles from real job responsibilities, then map them to groups and categories that users can understand in the Settings UI. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Create module categories for clean group presentation and define parent-child categories only when they reflect a real application hierarchy. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Verify each role with a separate user, not by switching an administrator's groups while staying in the same browser session. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor security overview: six components, groups, categories, and hierarchy decision appears during upgrades, audits, and support tickets. Security that is easy to explain is easier to audit; if a role requires a diagram to justify every permission, the design is probably too broad. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Security has more than CSV access rights.
- Groups collect permissions and UI visibility.
- Categories organize groups in the user form.
- Record rules restrict rows after model access is granted.
- Business methods must still check sensitive transitions.

Application category and groups

The category makes the group visible in the expected application section of the user form.

```
<record id="module_category_training" model="ir.module.category">
  <field name="name">Training</field>
  <field name="sequence">35</field>
</record>

<record id="group_training_user" model="res.groups">
  <field name="name">Training User</field>
  <field name="category_id" ref="module_category_training"/>
</record>
```

Common Mistakes

- Assuming menus are security controls.
- Designing groups from developer convenience instead of job roles.
- Granting broad access and trying to patch it later with record rules.
- Testing security only as the administrator.

Lab / Practice: Create three job-role groups for a training app and write a one-sentence business description for each before adding permissions.

VS16 Implied Groups and Combobox Group Selection

Learning Objectives

- Use `implied_ids` to build role inheritance.
- Understand group selection behavior in the user form.
- Avoid accidental privilege escalation through implied groups.

Explanation

Implied Groups and Combobox Group Selection is best understood as part of the contract between the ORM, the web client, and the business user. Implied groups let one role include another role, which keeps permissions maintainable when managers should also have normal user capabilities. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Design implication from smaller to larger roles and make the chain obvious; hidden implication is a common source of unexpected access. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Set `implied_ids` with the many2many command syntax and use exclusive category behavior carefully when groups are presented as a combobox-style selection. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Test adding and removing groups from a user because implied groups may remain or disappear in ways that surprise administrators. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor implied groups and combobox group selection decision appears during upgrades, audits, and support tickets. In production, implied groups are part of privilege design, so review them like code and document why each implication exists. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- `implied_ids` grants included groups automatically.
- Role inheritance should go from broad basic access to stronger roles.
- Group categories affect how selections appear on users.
- Removing an implying group can affect implied membership.
- Implication is not a substitute for record rules.

Manager implies user

A manager automatically receives the base user permissions instead of duplicating them.

```
<record id="group_training_manager" model="res.groups">
  <field name="name">Training Manager</field>
  <field name="category_id" ref="module_category_training"/>
  <field name="implied_ids" eval="[(4, ref('training.group_training_user'))]"/>
</record>
```

Common Mistakes

- Creating circular or confusing implication chains.
- Using implied groups to hide overly broad access rights.
- Forgetting that implied groups affect all users assigned the parent group.
- Not testing group removal behavior.

Lab / Practice: Create user, teacher, and manager groups where manager implies teacher and teacher implies user, then inspect the effective groups on a test user.

VS17 Access Rights in CSV and XML

Learning Objectives

- Write `ir.model.access.csv` rows correctly.
- Know when XML access records are useful.
- Separate model access from row-level record rules.

Explanation

Access Rights in CSV and XML is best understood as part of the contract between the ORM, the web client, and the business user. Access rights answer whether a group may create, read, write, or delete records of a model at all. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Use CSV for normal module permissions because it is compact, reviewable, and recognized by Odoo conventions. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use XML only for exceptional cases where you need update behavior, conditions around noupdate data, or integration with a generated data process. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Verify access with a user who has only the target group, because inherited administrator permissions can hide missing CSV rows. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor access rights in csv and xml decision appears during upgrades, audits, and support tickets. A clean access matrix is the foundation for every later record rule; record rules cannot grant access if model access denies it. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- CSV rows live in `security/ir.model.access.csv`.
- `perm_create/read/write/unlink` are independent booleans.
- The `model_id` value references `model_model_name`.
- Record rules filter rows after access rights pass.
- XML access records are possible but less common.

Access CSV for course users

Base users can read courses while managers receive full model-level access.

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_training_course_user,training.course user,model_training_course,training.group_training_user,1,0,0,0
access_training_course_manager,training.course manager,model_training_course,training.group_training_manager,1,1,1,1
```

Common Mistakes

- Using record rules to compensate for missing model access.
- Misspelling model external ids in CSV.
- Granting unlink by default to operational users.
- Forgetting access rows for transient models used by wizards.

Lab / Practice: Create an access CSV for course and session models, then confirm a read-only user cannot create from the UI or via RPC.

VS18 Record Rules: Why They Exist and How Domains Work

Learning Objectives

- Explain record rules as row-level filters.
- Write domains that match business ownership.
- Understand global and group-specific rule behavior.

Explanation

Record Rules: Why They Exist and How Domains Work is best understood as part of the contract between the ORM, the web client, and the business user. Record rules restrict which rows a user can see or change after model access rights have allowed the operation. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Design rules from ownership and responsibility: own records, company records, assigned team records, or records in an approved state. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Write domains using user, company, and field values, then assign rules to groups unless the rule truly applies globally. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Test read, write, create, and unlink separately because record rules can be operation-specific and may interact in surprising ways. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor record rules: why they exist and how domains work decision appears during upgrades, audits, and support tickets. In production, record rules are audit controls; a permissive domain can expose customer, payroll, or student data even when menus look correct. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Rules are domains on records.
- Model access must pass before record rules matter.
- Global rules combine with group rules.
- Rules can be limited by operation.
- Domains should be readable and testable.

Own-record rule

The rule limits teacher users to courses connected to their own user record.

```
<record id="rule_training_course_own_teacher" model="ir.rule">
  <field name="name">Teachers see their courses</field>
  <field name="model_id" ref="model_training_course"/>
  <field name="domain_force">[('teacher_user_id', '=', user.id)]</field>
  <field name="groups" eval="[(4, ref('training.group_training_teacher'))]">
</record>
```

Common Mistakes

- Forgetting that read rules also affect many2one display names.
- Creating global rules when a group rule was intended.
- Using domains that depend on fields users cannot access.
- Testing only one operation.

Lab / Practice: Write a rule that lets teachers read only their assigned courses, then verify the list, form, export, and many2one search behavior.

VS19 Record Rules Student and Teacher Example

Learning Objectives

- Model a realistic student-teacher security matrix.
- Combine group access and record ownership.
- Validate cross-role behavior with fixtures.

Explanation

Record Rules Student and Teacher Example is best understood as part of the contract between the ORM, the web client, and the business user. Education and training modules are excellent record-rule examples because students, teachers, and managers see overlapping but different data. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Give every role the smallest model access it needs, then use rules for ownership: students see their enrollments, teachers see their courses, managers see all. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use related fields such as `student_user_id` and `teacher_user_id` to make domains readable and avoid deep joins where possible. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Create fixture users for each role and run scripted checks so future developers do not accidentally widen a rule during maintenance. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor record rules student and teacher example decision appears during upgrades, audits, and support tickets. The goal is not to hide screens; the goal is to make every data path return only the records the business allows. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Students and teachers need different ownership fields.
- Managers often need a broader rule, not sudo everywhere.
- Related user fields simplify domains.
- Fixtures make rule testing repeatable.
- Rules affect UI and RPC access.

Student and teacher domains

The student rule follows direct ownership while the teacher rule follows the course relationship.

```
<record id="rule_enrollment_student_self" model="ir.rule">
  <field name="name">Students see own enrollments</field>
  <field name="model_id" ref="model_training_enrollment"/>
  <field name="domain_force">[('student_user_id', '=', user.id)]</field>
  <field name="groups" eval="[(4, ref('training.group_training_student'))]">
</record>

<record id="rule_enrollment_teacher_courses" model="ir.rule">
  <field name="name">Teachers see course enrollments</field>
  <field name="model_id" ref="model_training_enrollment"/>
  <field name="domain_force">[('course_id.teacher_user_id', '=', user.id)]</field>
  <field name="groups" eval="[(4, ref('training.group_training_teacher'))]">
</record>
```

Common Mistakes

- Letting students read the course roster through an unrestricted related model.
- Using sudo in controllers instead of fixing the rule design.

- Forgetting portal or external users if the module exposes them.
- Assuming a hidden menu blocks RPC reads.

Lab / Practice: Create three users and four enrollments, then prove each role sees the expected records through the UI and an XML-RPC read call.

VS20 Security Inheritance and Groups on Fields, Views, and Actions

Learning Objectives

- Apply groups attributes to UI elements.
- Understand how inherited security data changes effective permissions.
- Avoid confusing UI restrictions with real authorization.

Explanation

Security Inheritance and Groups on Fields, Views, and Actions is best understood as part of the contract between the ORM, the web client, and the business user. Security inheritance happens when modules add groups, access rows, rules, or UI restrictions on top of existing behavior. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Use groups on fields, buttons, menus, views, and actions to reduce clutter for users who cannot use those features anyway. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Keep actual data protection in access rights, record rules, and business methods; groups on views are a presentation and workflow control. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Test inherited modules in the full dependency stack because a later module may add a broad rule or imply a powerful group. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor security inheritance and groups on fields, views, and actions decision appears during upgrades, audits, and support tickets. Good security inheritance is additive and reviewable, while scattered group attributes become hard to audit during upgrades. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- groups attributes hide UI elements from users outside the group.
- Menus and actions can be group-restricted.
- Field-level groups can protect field visibility.
- Server-side checks remain necessary.
- Inherited modules can widen or narrow effective access.

Group-restricted margin field and action

Managers see finance information in the view and can access the reporting action, but model security must still protect the data.

```
<field name="margin_percent" groups="training.group_training_manager"/>

<record id="training_finance_action" model="ir.actions.act_window">
  <field name="name">Course Margins</field>
  <field name="res_model">training.course</field>
  <field name="view_mode">pivot,graph</field>
  <field name="groups_id" eval="[(4, ref('training.group_training_manager'))]"/>
</record>
```

Common Mistakes

- Believing hidden fields are secure if the model allows read.
- Adding group attributes in many inherited views without a central role design.
- Forgetting server checks on object buttons.
- Not reviewing implied groups when a module is installed.

Lab / Practice: Hide a margin field and finance action from normal users, then attempt to read the field by RPC to confirm whether server security is sufficient.

VS21 Odoo 19 Privilege Model

Learning Objectives

- Understand the direction of Odoo 19 privilege tightening.
- Identify where custom modules depend on broad administrative behavior.
- Plan privilege reviews during upgrades.

Explanation

Odoo 19 Privilege Model is best understood as part of the contract between the ORM, the web client, and the business user. Odoo 19 continues the trend of making privileges more explicit, especially around administrative operations, technical settings, and cross-app access. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Audit custom modules for places where code assumes that a user with one application role can manage technical records or operate outside their business scope. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Keep methods explicit about sudo usage, group checks, and record ownership so a framework privilege change does not silently break or overexpose behavior. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. During verification, run upgrade tests with real users for settings, imports, server actions, scheduled jobs, and any custom controller endpoint. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor odoo 19 privilege model decision appears during upgrades, audits, and support tickets. Privilege models evolve because production systems need stronger boundaries; custom modules should be ready by avoiding implicit administrator shortcuts. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Expect stronger separation between business and technical privileges.
- Audit sudo calls and administrative group checks.
- Do not assume old Settings access remains enough.
- Test server actions and scheduled jobs during upgrade.
- Treat privilege changes as security changes, not only migration work.

Explicit group check around sensitive action

The method checks the business role before using sudo for the narrow internal operation.

```
def action_recompute_certificates(self):
    if not self.env.user.has_group("training.group_training_manager"):
        raise AccessError("Only training managers can recompute certificates.")
    self.sudo()._recompute_certificate_numbers()
    return True
```

Common Mistakes

- Using sudo to make upgrade errors disappear.
- Assuming a technical user interface group equals business approval.
- Leaving server actions callable by broad groups.
- Skipping tests for scheduled and automated actions.

Lab / Practice: Review one custom module and list every sudo call, then classify each as required, removable, or needing a preceding group check.

VS22 Odoo 18 to 19 Migration Notes for Views and Security

Learning Objectives

- Plan view modifier migration work.
- Review deprecated XML patterns.
- Regression-test security after framework upgrades.

Explanation

Odoo 18 to 19 Migration Notes for Views and Security is best understood as part of the contract between the ORM, the web client, and the business user. Migration from Odoo 18 to 19 should be treated as a behavioral review, not a search-and-replace exercise. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Inventory view modifiers, JavaScript widgets, access files, record rules, automated actions, and reports before changing code. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Update XML syntax where required, remove obsolete patterns, and prefer direct modifier expressions and clear inherited views over compatibility clutter. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Run upgrade tests from a fresh restored database, then test real role journeys because view errors and security regressions often appear only after registry reload. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor odoo 18 to 19 migration notes for views and security decision appears during upgrades, audits, and support tickets. A good migration leaves the module easier to understand than before; do not preserve unshipped branch behavior by layering shims around it. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Inventory before editing.
- Migrate modifiers deliberately.
- Review groups, access rights, and record rules together.
- Test reports and server actions, not only menus.
- Keep migration commits focused.

Migration checklist fragment

The first pass locates legacy modifiers, privilege-sensitive code, and security XML that deserve manual review.

```
# Suggested review commands
rg "attrs=|states=" custom_addons/training
rg "sudo\(|has_group\(" custom_addons/training
rg "ir.rule|ir.model.access|groups=" custom_addons/training/security custom_addons/training/views
```

Common Mistakes

- Migrating syntax without comparing behavior.
- Leaving obsolete inherited views that no longer match upstream anchors.
- Testing only with admin users.
- Combining migration with unrelated refactors.

Lab / Practice: Build a migration checklist for one module, convert two legacy modifiers, and document one security rule that needs a functional owner review.

VS23 XML Data, CSV Data, and Choosing Between Them

Learning Objectives

- Choose XML or CSV based on data shape.
- Load reference data idempotently.
- Understand noupdate implications.

Explanation

XML Data, CSV Data, and Choosing Between Them is best understood as part of the contract between the ORM, the web client, and the business user. Odoo modules use data files to install configuration, security, records, actions, and sometimes initial business data. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Use XML when records need references, nested structure, eval expressions, noupdate control, or readable grouping; use CSV for flat bulk rows. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Keep reference data stable with external ids and be careful with noupdate because it protects user-edited records from future module updates. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Test install, update, uninstall where relevant, and import into a fresh database because data files often pass locally only due to leftover records. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor xml data, csv data, and choosing between them decision appears during upgrades, audits, and support tickets. The correct file format reduces future migration pain: XML is expressive, CSV is efficient, and mixing them intentionally is normal. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- XML is better for relational configuration records.
- CSV is better for large flat datasets.
- noupdate affects later module updates.
- External ids make data repeatable.
- Manifest order controls references.

Reference category XML

noupdate is appropriate when administrators may rename categories after installation.

```
<odoo noupdate="1">
  <record id="course_category_functional" model="training.course.category">
    <field name="name">Functional</field>
    <field name="sequence">10</field>
  </record>
  <record id="course_category_technical" model="training.course.category">
    <field name="name">Technical</field>
    <field name="sequence">20</field>
  </record>
</odoo>
```

Common Mistakes

- Using CSV for records that need complex references or eval.
- Putting user-editable configuration outside noupdate without a reason.
- Referencing records before their files load.

- Duplicating records by removing external ids.

Lab / Practice: Create the same three category records in XML and CSV, then decide which version should ship and explain the upgrade behavior.

VS24 View Inheritance Overview and XPath Positions

Learning Objectives

- Use inherited views instead of copying base views.
- Select robust XPath anchors.
- Apply position attributes safely.

Explanation

View Inheritance Overview and XPath Positions is best understood as part of the contract between the ORM, the web client, and the business user. View inheritance lets modules extend or change existing screens without replacing the upstream view entirely. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Choose XPath anchors that express intent and are likely to survive upgrades, such as named fields or groups with stable names. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use positions before, after, inside, replace, attributes, and move carefully, keeping each inherited view focused on one business change. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Verify the final combined architecture in developer mode and run module updates after upstream changes because broken XPaths fail at load time. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor view inheritance overview and xpath positions decision appears during upgrades, audits, and support tickets. Small inherited views make customizations easier to remove during migrations and easier for another developer to review. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- `inherit_id` points to the parent view.
- XPath anchors should be stable and specific.
- `position='attributes'` changes attributes only.
- Avoid copying full upstream views.
- One inherited view should have one clear purpose.

Add a field after teacher

The extension adds one field and changes one attribute without copying the whole parent form.

```
<record id="training_course_form_inherit_capacity" model="ir.ui.view">
  <field name="name">training.course.form.inherit.capacity</field>
  <field name="model">training.course</field>
  <field name="inherit_id" ref="training.training_course_view_form"/>
  <field name="arch" type="xml">
    <xpath expr="//field[@name='teacher_id']" position="after">
      <field name="seat_count"/>
    </xpath>
    <xpath expr="//field[@name='price_total']" position="attributes">
      <attribute name="groups">training.group_training_manager</attribute>
    </xpath>
  </field>
</record>
```

Common Mistakes

- Using positional XPaths like `//group[2]` that break after small upstream changes.
- Replacing large blocks when attributes would be enough.
- Combining unrelated changes in one inherited view.

- Ignoring priority when multiple modules inherit the same target.

Lab / Practice: Inherit a form twice from two small modules, then inspect how priority and XPath anchors affect the final architecture.

VS25 ir.sequence for Business Numbers

Learning Objectives

- Create reliable sequence records.
- Assign sequence values in create.
- Handle company-specific numbering when needed.

Explanation

ir.sequence for Business Numbers is best understood as part of the contract between the ORM, the web client, and the business user. Business users expect order numbers, certificate numbers, and ticket references to be unique, readable, and stable. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Decide whether the number is legal, operational, or cosmetic because that determines gaps, prefixes, date ranges, and company separation. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Define an ir.sequence record with code, prefix, padding, and implementation, then call `next_by_code` in create only when the placeholder value is still new. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Test imports, duplicate creation, multi-company behavior, and rollback scenarios so the sequence does not create confusing references. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor ir.sequence for business numbers decision appears during upgrades, audits, and support tickets. A sequence is part of external communication; changing it carelessly can break integrations, printed reports, and customer support procedures. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Use code as the stable lookup key.
- Assign numbers server-side in create.
- Do not recompute sequence values on write.
- Date ranges and companies change numbering behavior.
- Legal sequences may require stricter gap handling.

Certificate sequence

The XML defines the sequence and create assigns it once when the record is created.

```
<record id="seq_training_certificate" model="ir.sequence">
  <field name="name">Training Certificate</field>
  <field name="code">training.certificate</field>
  <field name="prefix">CERT/%(year)s</field>
  <field name="padding">5</field>
</record>

@api.model_create_multi
def create(self, vals_list):
    for vals in vals_list:
        if vals.get("name", "New") == "New":
            vals["name"] = self.env["ir.sequence"].next_by_code("training.certificate")
    return super().create(vals_list)
```

Common Mistakes

- Calling `next_by_code` in `default_get` and consuming numbers for abandoned forms.
- Changing sequence prefixes without migration communication.

- Using one sequence when companies require separate ranges.
- Allowing users to edit legal sequence numbers casually.

Lab / Practice: Add a certificate number sequence, create records through UI and import, and confirm every saved record receives one unique number.

VS26 QWeb Intro and Core Directives

Learning Objectives

- Explain how QWeb renders XML templates.
- Use t-call, t-set, t-esc, and t-att safely.
- Separate template logic from business logic.

Explanation

QWeb Intro and Core Directives is best understood as part of the contract between the ORM, the web client, and the business user. QWeb is Odoo's XML templating engine for reports, website fragments, mail snippets, and many kanban templates. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Keep templates focused on presentation and put calculations, permissions, and data preparation in models or report helpers. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use t-call for composition, t-set for local values, t-esc for escaped output, and t-att or t-attf for attributes that come from safe values. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Render reports with representative records and check HTML output before PDF rendering because QWeb errors can be easier to diagnose in HTML. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor qweb intro and core directives decision appears during upgrades, audits, and support tickets. Clean QWeb survives design changes because layout can move without rediscovering hidden business logic buried in template conditions. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- QWeb templates are XML and must be valid XML.
- t-esc escapes output and is safer for user data.
- t-call composes reusable templates.
- t-att builds dynamic attributes.
- Business calculations belong outside templates.

Reusable certificate block

The template escapes record values and uses a dynamic class for styling without doing heavy calculation in QWeb.

```
<template id="training_certificate_block">
  <div class="certificate">
    <h2><t t-esc="doc.name"/></h2>
    <p>Issued to <span t-esc="doc.student_id.name"/></p>
    <p t-att-class="'grade grade-%s' % doc.grade_code">
      Score: <span t-esc="doc.score"/>
    </p>
  </div>
</template>
```

Common Mistakes

- Using t-raw for user-provided data.
- Putting complex calculations in template expressions.
- Forgetting XML escaping rules.
- Copying templates instead of using t-call.

Lab / Practice: Create a QWeb template that renders a certificate title, student name, teacher name, and escaped note field.

VS27 QWeb Conditionals, Loops, and Template Inheritance

Learning Objectives

- Use t-if and t-foreach for report structures.
- Inherit QWeb templates with XPath.
- Keep repeated report sections maintainable.

Explanation

QWeb Conditionals, Loops, and Template Inheritance is best understood as part of the contract between the ORM, the web client, and the business user. Most useful QWeb reports need conditional sections and repeated lines, but that does not mean the template should become a program. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Prepare clean recordsets in Python and let QWeb decide only whether to show sections and how to repeat display rows. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use t-if, t-elif, t-else, t-foreach, t-as, and inherited templates with XPath to extend report layouts without copying them. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Verify empty lists, one-row lists, and many-row reports because loop boundaries and page breaks are common report defects. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor qweb conditionals, loops, and template inheritance decision appears during upgrades, audits, and support tickets. Template inheritance keeps local changes upgradeable; full report copies are expensive when Odoo changes base layouts or assets. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- t-if controls conditional rendering.
- t-foreach repeats over recordsets or lists.
- Template inheritance uses XPath like view inheritance.
- Prepare complex data before rendering.
- Test empty and long datasets.

Loop and inherit report template

The template handles both a roster and an empty state with simple QWeb directives.

```
<template id="report_course_roster">
  <t t-call="web.external_layout">
    <h2><span t-esc="doc.name"/></h2>
    <table>
      <tr t-foreach="doc.enrollment_ids" t-as="line">
        <td><span t-esc="line.student_id.name"/></td>
        <td><span t-esc="line.state"/></td>
      </tr>
    </table>
    <p t-if="not doc.enrollment_ids">No students enrolled.</p>
  </t>
</template>
```

Common Mistakes

- Looping over unsorted recordsets when report order matters.
- Using t-raw to inject HTML without sanitizing.
- Copying external_layout instead of inheriting or calling it.
- Forgetting the empty state.

Lab / Practice: Build a roster report that shows a table for enrolled students and a clear message when no students are enrolled.

VS28 Paper Formats, Barcodes, Images, Server Actions, and Email Templates

Learning Objectives

- Configure report paper formats and media assets.
- Add barcode or QR output to PDFs.
- Connect server actions and email templates to document workflows.

Explanation

Paper Formats, Barcodes, Images, Server Actions, and Email Templates is best understood as part of the contract between the ORM, the web client, and the business user. Printed documents are often where Odoo leaves the browser and becomes part of legal, warehouse, or customer-facing process. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Define paper format before fine-tuning CSS, because margins, orientation, and DPI change how every report element behaves. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use report barcode routes for QR codes, serve images from binary fields or attachments, and trigger emails through templates or server actions only after permissions are clear. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Verification is not only clicking the happy path as an administrator. Test PDFs with missing images, long names, different companies, and email previews so failures are visible before a customer receives a broken document. Test with at least two real security profiles, reload the module from a clean database, and check that exported or imported data behaves the same way as data created through the UI.

In production, the cost of a poor paper formats, barcodes, images, server actions, and email templates decision appears during upgrades, audits, and support tickets. Production reporting is operational communication; once a certificate, invoice, or label is emailed, users expect it to be stable and traceable. Keep the XML boring, make domains readable, document unusual choices in field help or comments, and treat permissions as part of the release checklist.

Key Points

- Paper format controls margins, orientation, and page size.
- Barcode and QR images can be rendered through report routes.
- Binary images need fallbacks and size control.
- Server actions should run with deliberate privileges.
- Email templates should be previewed with real records.

QR code and email send

The report renders a QR code from a verification URL and the model action sends the matching email template.

```


def action_send_certificate(self):
    self.ensure_one()
    template = self.env.ref("training.email_template_certificate")
    template.send_mail(self.id, force_send=True)
    return True
```

Common Mistakes

- Hard-coding paper margins in CSS while ignoring paperformat.
- Embedding huge images that slow every PDF render.
- Running server actions with sudo without business checks.
- Sending emails without previewing language and company context.

Lab / Practice: Create a certificate PDF with a custom paper format, company logo, QR verification link, and an email template action.

Part G OWL Frontend, POS & External APIs

This part contains 20 detailed lessons. Work in order. Complete each lab before moving forward when the lab unlocks the next dependency.

Presentation: Companion deck: see [training_materials/presentations/](#) for the OWL Frontend, POS & External APIs slides.

W01 OWL Architecture and Reactive Rendering

Learning Objectives

- Explain how OWL components render from reactive state.
- Identify the role of services, registries, and templates.
- Avoid direct DOM manipulation in normal component logic.

Explanation

OWL Architecture and Reactive Rendering belongs to the client-side layer where user intent, browser state, and Odoo services meet. OWL is Odoo's component framework and provides the reactive mental model behind many modern web client features. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Decide which component owns state, which services provide data, and which children only render props. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Use Component classes, static templates, setup methods, hooks, and the registry rather than creating global scripts. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Inspect rerenders, service calls, and asset loading in browser developer tools after every meaningful change. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, owl architecture and reactive rendering decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A maintainable OWL customization feels native because it composes with the framework instead of fighting it. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Reactive state changes trigger rerendering.
- Templates describe UI; components own behavior.
- Services centralize shared capabilities such as rpc and notifications.
- Registries let addons extend the web client.
- Direct DOM writes are a last resort.

Small reactive counter component

The state object is reactive, so incrementing value rerenders the template that displays it.

```
/** @odoo-module **/  
  
import { Component, useState } from "@odoo/owl";  
import { registry } from "@web/core/registry";  
  
export class TrainingCounter extends Component {  
    static template = "training.TrainingCounter";  
  
    setup() {  
        this.state = useState({ value: 0 });  
    }  
  
    increment() {  
        this.state.value += 1;  
    }  
}  
  
registry.category("actions").add("training_counter", TrainingCounter);
```

Common Mistakes

- Changing random DOM nodes instead of changing component state.

- Putting global variables in asset files for shared state.
- Forgetting to register the component under the correct registry category.
- Treating OWL like jQuery with templates attached.

Lab / Practice: Create a small client action that increments a counter and logs when the component rerenders.

W02 Component Structure and Lifecycle Hooks

Learning Objectives

- Organize component files and templates cleanly.
- Use lifecycle hooks for setup and cleanup.
- Keep asynchronous loading predictable.

Explanation

Component Structure and Lifecycle Hooks belongs to the client-side layer where user intent, browser state, and Odoo services meet. Component structure matters because Odoo assets are bundled, upgraded, translated, and extended by other modules. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Keep JavaScript behavior, XML templates, and SCSS names aligned so another developer can find the complete feature quickly. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Use setup for hooks, onWillStart for initial asynchronous work, onMounted for browser-only integration, and onWillUnmount for cleanup. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Test navigation away from the component while requests are pending and verify that timers or listeners do not leak. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, component structure and lifecycle hooks decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Lifecycle discipline prevents slow memory leaks and race conditions that otherwise appear only during long back-office sessions. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- setup is the main hook registration point.
- onWillStart can await data before first render.
- onMounted is safe for DOM-dependent integrations.
- onWillUnmount cleans up listeners and timers.
- Asset paths should mirror feature names.

Lifecycle with cleanup

The component loads initial data before rendering and removes its refresh timer when it is destroyed.

```
/** @odoo-module **/  
  
import { Component, onMounted, onWillStart, onWillUnmount, useState } from "@odoo/owl";  
import { useService } from "@web/core/utils/hooks";  
  
export class CourseDashboard extends Component {  
    static template = "training.CourseDashboard";  
  
    setup() {  
        this.orm = useService("orm");  
        this.state = useState({ stats: null });  
        let timer;  
  
        onWillStart(async () => {  
            this.state.stats = await this.orm.call("training.course", "dashboard_stats", []);  
        });  
        onMounted(() => {  
            timer = setInterval(() => this.refresh(), 60000);  
        });  
        onWillUnmount(() => clearInterval(timer));  
    }  
  
    async refresh() {  
        this.state.stats = await this.orm.call("training.course", "dashboard_stats", []);  
    }  
}
```

Common Mistakes

- Starting intervals without clearing them.
- Calling browser APIs before the component is mounted.
- Loading data in the constructor instead of setup hooks.
- Ignoring navigation races during slow RPC responses.

Lab / Practice: Build a dashboard component that loads statistics before render and refreshes every minute without leaking timers.

W03 useState, useRef, Events, and Props

Learning Objectives

- Use state for mutable component data.
- Use refs for specific DOM elements.
- Pass behavior through events and props cleanly.

Explanation

useState, useRef, Events, and Props belongs to the client-side layer where user intent, browser state, and Odoo services meet. OWL gives developers clear tools for local state, DOM references, parent-to-child data, and child-to-parent communication. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Before writing code, decide whether a value is owned by the component, passed by the parent, derived from services, or simply stored in the DOM. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Use useState for reactive values, useRef for controlled DOM access, props for inputs, and callback props or triggered events for outputs. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Verify that a child component still works when props change and that refs are not accessed before mount. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, usestate, useref, events, and props decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Correct ownership keeps components reusable and avoids the common trap of several components mutating the same data without a single source of truth. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- State is mutable and reactive inside the component.
- Props are inputs from the parent.
- Refs point to DOM nodes after render.
- Events should describe user intent.
- Derived values should not be duplicated unnecessarily.

Filter input with ref and callback prop

The component owns the input text, calls the parent callback when applied, and uses a ref only to restore focus.

```
/** @odoo-module **/  
  
import { Component, useRef, useState } from "@odoo/owl";  
  
export class CourseFilter extends Component {  
    static template = "training.CourseFilter";  
    static props = { onApply: Function };  
  
    setup() {  
        this.state = useState({ text: "" });  
        this.inputRef = useRef("filterInput");  
    }  
  
    apply() {  
        this.props.onApply(this.state.text.trim());  
        this.inputRef.el.focus();  
    }  
}
```

Common Mistakes

- Mutating props directly.
- Using refs as general state storage.
- Passing large objects when a small prop would do.
- Creating event names that describe implementation instead of intent.

Lab / Practice: Create a child filter component that receives an onApply callback and prove it remains reusable in two parent components.

W04 QWeb XML Directives: t-if, t-foreach, and t-model

Learning Objectives

- Render conditional content in OWL XML templates.
- Loop over arrays safely with keys.
- Bind form controls with t-model where appropriate.

Explanation

QWeb XML Directives: t-if, t-foreach, and t-model belongs to the client-side layer where user intent, browser state, and Odoo services meet. OWL templates use QWeb-style XML directives to describe dynamic browser UI. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Keep templates declarative: conditions should select presentation, loops should render prepared lists, and form bindings should update local state. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Use t-if for conditions, t-foreach with t-as and t-key for repeatable data, and t-model for simple two-way form input binding. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Test empty arrays, duplicate labels, and rapid typing because template mistakes often appear as unstable DOM updates. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, qweb xml directives: t-if, t-foreach, and t-model decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Good templates are readable enough that a reviewer can understand the UI states without opening the JavaScript file first. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Every t-foreach should have a stable t-key.
- t-if removes or inserts template branches.
- t-model is useful for simple local form values.
- Complex formatting belongs in component methods or getters.
- XML must remain valid and escaped.

OWL template directives

The template handles search input, empty state, and stable list rendering with a key per course.

```
<templates xml:space="preserve">
  <t t-name="training.CourseList">
    <input t-ref="filterInput" t-model="state.query" placeholder="Search courses"/>
    <p t-if="!state.courses.length">No courses found.</p>
    <ul>
      <li t-foreach="state.courses" t-as="course" t-key="course.id">
        <span t-esc="course.name"/>
        <span class="text-muted" t-esc="course.teacher"/>
      </li>
    </ul>
  </t>
</templates>
```

Common Mistakes

- Looping without t-key.
- Putting business queries inside template expressions.
- Forgetting XML escaping for comparison operators.
- Using t-model against props instead of local state.

Lab / Practice: Write a template that lists courses, shows an empty state, and filters from a t-model input stored in component state.

W05 Custom Form Field Widgets

Learning Objectives

- Register a field widget in the web client.
- Respect standard field props and update behavior.
- Use widgets to improve editing without changing model semantics.

Explanation

Custom Form Field Widgets belongs to the client-side layer where user intent, browser state, and Odoo services meet. A custom field widget changes how one field is displayed or edited while keeping the model field as the source of truth. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Use a widget when the default field component is correct semantically but not expressive enough for the workflow. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Register the widget in the fields registry, accept standard props, and call `props.update` when the user changes the value. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Test readonly mode, required fields, invalid values, and list/form contexts because widgets may be reused outside the original screen. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, custom form field widgets decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Production widgets should be boringly compatible with Odoo's form renderer: they should not bypass validation, dirty state, or `onchange` behavior. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Field widgets are registered in the fields registry.
- `props.value` is the current field value.
- `props.update` writes a new value to the record buffer.
- Support readonly mode explicitly.
- Widgets should not own persistent business state.

Rating field widget

The widget follows the standard field contract and updates the form record through `props.update`.

```
/** @odoo-module **/  
  
import { Component } from "@odoo/owl";  
import { registry } from "@web/core/registry";  
import { standardFieldProps } from "@web/views/fields/standard_field_props";  
  
export class RatingField extends Component {  
    static template = "training.RatingField";  
    static props = { ...standardFieldProps };  
  
    setRating(value) {  
        if (!this.props.readonly) {  
            this.props.update(value);  
        }  
    }  
}  
  
registry.category("fields").add("training_rating", {  
    component: RatingField,  
    supportedTypes: ["integer", "float"],  
});
```

Common Mistakes

- Writing directly to the server from the widget instead of using props.update.
- Ignoring readonly mode.
- Registering a widget with a generic key likely to collide.
- Supporting field types the component cannot handle.

Lab / Practice: Create a five-star rating widget for an integer field and test it in readonly and editable form modes.

W06 Overriding JavaScript Components with the Patch Utility

Learning Objectives

- Patch existing components narrowly.
- Call super behavior when extending methods.
- Minimize upgrade risk when overriding Odoo web code.

Explanation

Overriding JavaScript Components with the Patch Utility belongs to the client-side layer where user intent, browser state, and Odoo services meet. The patch utility is the preferred way to adjust existing JavaScript classes when registries or props are not enough. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Patch only the method or getter you need, and first ask whether a registry extension, service, or template inheritance would be safer. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Import the target class and patch it with a small object that adds or extends behavior while preserving upstream logic where required. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Test against the exact Odoo version and read upstream changes during upgrades because patches depend on private implementation details more than normal extensions. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, overriding javascript components with the patch utility decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A good patch is almost invisible: one responsibility, no global side effects, and an easy removal path when upstream gains the needed hook. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Patch is powerful and should be narrow.
- Prefer public extension points first.
- Call super when preserving behavior matters.
- Version upgrades can break patches.
- Name modules and methods clearly for traceability.

Patch a method to add a training flag

The patch adds a small event before save while delegating the actual save behavior to the original controller.

```
/** @odoo-module */

import { patch } from "@web/core/utils/patch";
import { FormController } from "@web/views/form/form_controller";

patch(FormController.prototype, {
  async saveButtonClicked(params = {}) {
    this.env.bus.trigger("training:before-form-save", {
      resModel: this.props.resModel,
    });
    return super.saveButtonClicked(params);
  },
});
```

Common Mistakes

- Patching large methods by copying upstream source.
- Forgetting to call super when extending behavior.
- Using patch when a registry extension exists.

- Not reviewing patches during every major upgrade.

Lab / Practice: Patch a small controller method to emit a custom event, then remove the patch and confirm the rest of the feature is isolated.

W07 RPC and ORM Services

Learning Objectives

- Use orm and rpc services appropriately.
- Call model methods from OWL components.
- Handle loading and errors cleanly.

Explanation

RPC and ORM Services belongs to the client-side layer where user intent, browser state, and Odoo services meet. Odoo provides services so client code does not need to know every HTTP detail or session convention. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Use orm for model operations and rpc for custom endpoints or lower-level calls where the ORM service does not fit. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Inject services with `useService`, keep request code in explicit methods, and store loading or error state so the UI communicates what is happening. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Test server access rights, record rules, network failures, and repeated clicks because the browser can make invalid calls faster than a human can diagnose them. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, rpc and orm services decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Service-based code is easier to migrate because authentication, context, and error handling stay aligned with the framework. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- `useService` injects framework services.
- `orm.call` invokes model methods.
- `rpc` is useful for custom routes.
- Always represent loading and failure states.
- Server methods must still validate access.

Load dashboard data through orm

The component uses framework services and exposes loading and error feedback to users.

```
/** @odoo-module **/  
  
import { Component, useState } from "@odoo/owl";  
import { useService } from "@web/core/utils/hooks";  
  
export class TrainingDashboard extends Component {  
    static template = "training.TrainingDashboard";  
  
    setup() {  
        this.orm = useService("orm");  
        this.notification = useService("notification");  
        this.state = useState({ loading: false, stats: {} });  
    }  
  
    async load() {  
        this.state.loading = true;  
        try {  
            this.state.stats = await this.orm.call("training.course", "dashboard_stats", []);  
        } catch (error) {  
            this.notification.add("Could not load dashboard data.", { type: "danger" });  
        } finally {  
            this.state.loading = false;  
        }  
    }  
}
```

Common Mistakes

- Calling fetch directly for ORM work.
- Ignoring errors and leaving stale UI state.
- Trusting client parameters without server validation.
- Making one RPC call per row when a batched method would do.

Lab / Practice: Replace a direct fetch call with `orm.call` and add a loading indicator plus a notification on failure.

W08 Custom Kanban and Dashboard OWL Views

Learning Objectives

- Design a dashboard as a client action or custom view.
- Register components in the correct registry.
- Keep dashboard data server-side and aggregated.

Explanation

Custom Kanban and Dashboard OWL Views belongs to the client-side layer where user intent, browser state, and Odoo services meet. Dashboards should present decisions, exceptions, and navigation paths rather than becoming a second database browser. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Choose a client action for a standalone dashboard and a custom view only when it truly replaces how users inspect a model. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Register the component, define an action tag, load aggregated data from model methods, and keep cards reusable where possible. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Verify performance on realistic data because dashboards are often opened by managers every morning and must not run expensive per-card queries. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, custom kanban and dashboard owl views decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A production dashboard is an operations surface: it needs clear ownership, stable metrics, and failure behavior that does not block the rest of Odoo. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Client actions are good for standalone dashboards.
- Custom views are deeper integrations with the view system.
- Aggregate on the server, render on the client.
- Use action tags and registries consistently.
- Dashboard cards should link to filtered records.

Client action registration

The action tag connects the server-side `ir.actions.client` record to the OWL component.

```
/** @odoo-module **/  
  
import { registry } from "@web/core/registry";  
import { TrainingDashboard } from "../training_dashboard";  
  
registry.category("actions").add("training.dashboard", TrainingDashboard);  
  
// XML action  
// <record id="action_training_dashboard" model="ir.actions.client">  
//   <field name="name">Training Dashboard</field>  
//   <field name="tag">training.dashboard</field>  
// </record>
```

Common Mistakes

- Loading raw records in the browser and aggregating there.
- Creating dashboards without drill-down actions.
- Using a custom view when a client action is enough.
- Forgetting access rights on the server statistics method.

Lab / Practice: Build a client action dashboard with three KPI cards and one card that opens a filtered course list.

W09 POS Custom Buttons and Click Events

Learning Objectives

- Add a custom control button to a POS screen.
- Handle click events with OWL methods.
- Keep POS actions compatible with offline behavior.

Explanation

POS Custom Buttons and Click Events belongs to the client-side layer where user intent, browser state, and Odoo services meet. POS extensions run in a high-speed operational UI where every button must be obvious and resilient. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Decide which screen owns the action and whether the feature can work offline or must warn the cashier that connectivity is required. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Register a button component, attach it to the target POS screen, and handle click events with methods that use POS services or models. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Test with barcode scanning, touch screens, and rapid repeated clicks because POS users do not interact like back-office users. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, pos custom buttons and click events decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A good POS button adds one clear workflow step without slowing checkout or compromising order consistency. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- POS buttons should be screen-specific.
- Click handlers should be idempotent where possible.
- Offline behavior must be considered.
- Use POS registries and services instead of globals.
- User feedback must be immediate.

Simple POS control button

The button is added to the ProductScreen and handles the click through a component method.

```
/** @odoo-module **/  
  
import { Component } from "@odoo/owl";  
import { ProductScreen } from "@point_of_sale/app/screens/product_screen/product_screen";  
  
export class CourseInfoButton extends Component {  
    static template = "training.CourseInfoButton";  
  
    onClick() {  
        this.env.services.notification.add("Training mode is active.", {  
            type: "info",  
        });  
    }  
}  
  
ProductScreen.addControlButton({  
    component: CourseInfoButton,  
    condition() {  
        return true;  
    },  
});
```

Common Mistakes

- Adding a button globally when it belongs to one screen.
- Not debouncing expensive click operations.
- Assuming a cashier has back-office permissions.
- Ignoring offline mode when the click needs the server.

Lab / Practice: Add a POS ProductScreen button that shows a notification and disable it when no order is active.

W10 Hiding POS Buttons Safely

Learning Objectives

- Hide or show POS UI based on configuration.
- Distinguish visibility from authorization.
- Keep hidden actions protected server-side.

Explanation

Hiding POS Buttons Safely belongs to the client-side layer where user intent, browser state, and Odoo services meet. Hiding a POS button can simplify cashier flow, but it must not be confused with security. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Decide whether the control is hidden because it is irrelevant, disabled because a condition is missing, or forbidden because the user lacks a role. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Use button conditions, configuration values loaded into POS, and server-side validation for any sensitive operation. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Test every cashier role and POS configuration because a button hidden on one terminal may remain visible on another configuration. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, hiding pos buttons safely decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Production POS systems are distributed user interfaces; UI conditions must match central business policy and survive session reloads. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Visibility improves workflow but is not security.
- Button conditions can read POS configuration.
- Sensitive methods need server checks.
- Different POS configs may show different controls.
- Use clear disabled feedback when possible.

Configuration-based button condition

The button appears only for POS configurations where the feature is enabled.

```
ProductScreen.addControlButton({
  component: CourseInfoButton,
  condition() {
    const config = this.pos.config;
    return Boolean(config.module_training && config.show_training_button);
  },
});
```

Common Mistakes

- Treating a hidden button as access control.
- Hard-coding user ids in JavaScript.
- Forgetting to load the configuration field into POS data.
- Making the button disappear without explaining why.

Lab / Practice: Add a Boolean POS configuration field that controls a custom button and prove the server method still rejects unauthorized calls.

W11 POS Popups and Dialog Flow

Learning Objectives

- Show confirmation and input popups in POS.
- Handle popup results correctly.
- Design cashier-friendly dialog flows.

Explanation

POS Popups and Dialog Flow belongs to the client-side layer where user intent, browser state, and Odoo services meet. POS popups interrupt the fastest workflow in Odoo, so they must be short, intentional, and recoverable. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Use a popup when the cashier needs confirmation, small input, or a warning that cannot be ignored before continuing. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Call the dialog or popup service, await the result where the framework supports it, and update the order only after the user confirms. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Test keyboard, touch, barcode scanner focus, and cancel paths because popup usability is more important in POS than in most back-office screens. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, pos popups and dialog flow decisions affect performance, offline behavior, cache invalidation, and upgrade risk. In production, a popup should prevent mistakes without creating a line at the register. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Use popups for short high-value interruptions.
- Always handle cancel and confirm paths.
- Do not put long forms in POS popups.
- Keep focus behavior cashier-friendly.
- Validate data after the popup closes.

Confirmation before clearing order

The destructive action is performed only from the confirmation callback.

```
async clearCurrentOrder() {
  const order = this.pos.get_order();
  if (!order || !order.get_orderlines().length) {
    return;
  }
  this.dialog.add(ConfirmationDialog, {
    title: "Clear order",
    body: "Remove all current order lines?",
    confirm: () => {
      for (const line of [...order.get_orderlines()]) {
        order.removeOrderline(line);
      }
    },
  });
}
```

Common Mistakes

- Using a popup for information that could be a non-blocking notification.
- Forgetting the cancel path.

- Mutating orders before the user confirms.
- Creating popups that require too much typing at checkout.

Lab / Practice: Create a confirmation popup that asks before applying a manager discount or clearing an order.

W12 JavaScript Translations

Learning Objectives

- Mark JavaScript strings for translation.
- Avoid concatenation that breaks translators' context.
- Test translated POS and web screens.

Explanation

JavaScript Translations belongs to the client-side layer where user intent, browser state, and Odoo services meet. JavaScript translations matter because OWL and POS features often introduce user-facing labels outside Python and XML fields. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Write complete sentences for translators and pass variables as placeholders rather than building messages from fragments. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Import `_t` or the appropriate translation helper, mark strings in components, and export translation terms through the normal Odoo i18n process. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Test in a secondary language and look for layout overflow, untranslated strings, and strings generated dynamically from code values. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, javascript translations decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A translated customization feels like part of the product; untranslated JavaScript is one of the fastest ways to expose custom code quality. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Use `_t` for user-facing JavaScript strings.
- Translate complete messages, not fragments.
- Avoid concatenation with translated text.
- Test labels in narrow screens.
- Do not translate technical keys or registry names.

Translated notification

The full sentence is translatable and the course name is inserted as a placeholder.

```
/** @odoo-module */

import { _t } from "@web/core/l10n/translation";

this.notification.add(
  _t("Course %(course)s was added to the training order.", {
    course: course.name,
  }),
  { type: "success" }
);
```

Common Mistakes

- Concatenating translated fragments.
- Forgetting strings inside POS-only components.
- Translating technical identifiers.
- Not testing how long translated labels affect layout.

Lab / Practice: Mark three JavaScript labels and one notification for translation, export terms, and verify them in another language.

W13 POS RPC and Server Synchronization

Learning Objectives

- Call server methods from POS safely.
- Respect offline and cashier permissions.
- Batch data where possible.

Explanation

POS RPC and Server Synchronization belongs to the client-side layer where user intent, browser state, and Odoo services meet. POS RPC is tempting for every feature, but online calls can slow checkout and fail during network interruptions. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Decide whether data must be loaded at session start, cached in POS, sent with the order, or fetched on demand. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Use services to call model methods, batch requests, and show clear failure feedback when the POS must be online for the operation. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Test disconnected network behavior and low permissions because POS often runs with users that differ from back-office managers. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, pos rpc and server synchronization decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Production POS integrations should minimize synchronous calls during payment and order validation. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Prefer session-loaded data for frequently used reference data.
- Use RPC only when freshness is required.
- Batch calls to avoid checkout delays.
- Handle offline failures explicitly.
- Server methods must check permissions.

Fetch live loyalty balance

The method calls the server only when a partner exists and gives a cashier-friendly warning on failure.

```
async getLoyaltyBalance(partnerId) {
  if (!partnerId) {
    return 0;
  }
  try {
    return await this.orm.call(
      "training.loyalty",
      "pos_get_balance",
      [partnerId],
      { context: this.pos.user.context }
    );
  } catch {
    this.notification.add("Loyalty balance is unavailable while offline.", {
      type: "warning",
    });
    return null;
  }
}
```

Common Mistakes

- Making one RPC call per product scan.
- Blocking payment on optional online data.
- Ignoring the POS user's access rights.
- Not deciding what happens offline.

Lab / Practice: Implement one POS RPC method for live data and document exactly what the cashier sees when the network is down.

W14 Rendering Dynamic POS Data

Learning Objectives

- Load additional fields into POS data.
- Render dynamic data on products or order lines.
- Keep display values synchronized with order state.

Explanation

Rendering Dynamic POS Data belongs to the client-side layer where user intent, browser state, and Odoo services meet. Dynamic POS rendering lets cashiers see availability, warnings, course metadata, or customer-specific information at the point of action. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. First decide whether the data is static for the session, changes during the session, or belongs on the order for accounting and audit. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Load required fields through POS data loaders or model extensions, store display-ready values on POS models, and render them through OWL templates. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Test reloads, order restore, and product cache updates because stale POS data can create incorrect cashier decisions. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, rendering dynamic pos data decisions affect performance, offline behavior, cache invalidation, and upgrade risk. In production, dynamic POS UI should display enough information to guide action without becoming a back-office report. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Session-loaded data is fast but can become stale.
- Order-line data should be stored if needed later.
- Templates should render prepared values.
- Reload and restore behavior must be tested.
- Do not overload product tiles with too much data.

Product tile showing course level

The template extension displays an extra product field only when the POS product data includes it.

```
<t t-name="training.ProductCard" t-inherit="point_of_sale.ProductCard" t-inherit-mode="extension">
  <xpath expr="//div[class='product-name']" position="after">
    <div t-if="props.product.training_level" class="training-level">
      <t t-esc="props.product.training_level"/>
    </div>
  </xpath>
</t>
```

Common Mistakes

- Rendering fields that were never loaded into POS.
- Showing stale stock or capacity without a refresh policy.
- Putting audit-critical values only in the UI.
- Making product cards unreadable with too many badges.

Lab / Practice: Load a training_level field into POS products and render it on the product card with an empty-state fallback.

W15 Clearing Order Lines and Mutating POS Orders

Learning Objectives

- Mutate current orders through POS model APIs.
- Avoid unsafe direct array changes.
- Protect destructive actions with confirmation.

Explanation

Clearing Order Lines and Mutating POS Orders belongs to the client-side layer where user intent, browser state, and Odoo services meet. POS order mutation must use framework methods so totals, taxes, discounts, and UI state remain consistent. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Before clearing or changing lines, decide whether the action is reversible and whether a manager confirmation is required. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Iterate over a copy of the current order lines and remove them with the POS order API rather than splicing internal arrays. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Test lines with discounts, lots, taxes, notes, and combo products because destructive actions often fail on less common line types. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, clearing order lines and mutating pos orders decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A safe mutation preserves the invariants that payment, receipts, and order export depend on. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Use order APIs, not internal array mutation.
- Copy line arrays before removing while iterating.
- Confirm destructive actions.
- Recompute totals through framework behavior.
- Test special line types.

Clear all order lines safely

The method iterates over a copy and delegates removal to the POS order API.

```
clearOrderLines(order) {
  if (!order) {
    return;
  }
  for (const line of [...order.get_orderlines()]) {
    order.removeOrderline(line);
  }
  this.notification.add("The order was cleared.", { type: "success" });
}
```

Common Mistakes

- Splicing `order.orderlines` directly.
- Removing lines while iterating over the live array.
- Skipping confirmation for destructive cashier actions.
- Forgetting special cases such as lot-tracked products.

Lab / Practice: Add a clear-order control that confirms first, removes lines through the order API, and verifies totals return to zero.

W16 XML-RPC Integration Fundamentals

Learning Objectives

- Authenticate with Odoo over XML-RPC.
- Call model methods through `execute_kw`.
- Understand database, uid, password, model, method, and args.

Explanation

XML-RPC Integration Fundamentals belongs to the client-side layer where user intent, browser state, and Odoo services meet. XML-RPC remains a practical integration interface for scripts, legacy systems, and training labs. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Design integrations around explicit service users with narrow groups, not administrator credentials copied into scripts. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Authenticate against common, then call `object.execute_kw` with model, method, positional args, and keyword options such as fields and limit. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Verify access rights and record rules with the same service user because XML-RPC observes normal Odoo security unless code uses `sudo` internally. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, xml-rpc integration fundamentals decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A production XML-RPC integration should have credentials, logging, retries, and data validation treated like any other external system. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Authenticate on `/xmlrpc/2/common`.
- Call models through `/xmlrpc/2/object execute_kw`.
- Use service users with limited groups.
- XML-RPC respects Odoo access rules.
- Never hard-code production passwords in code.

Python XML-RPC login and search_read

The script authenticates once and calls `search_read` as the integration user.

```
import xmlrpc.client

url = "https://odoo.example.com"
db = "prod"
username = "integration@example.com"
password = "change-me"

common = xmlrpc.client.ServerProxy(f"{url}/xmlrpc/2/common")
uid = common.authenticate(db, username, password, {})

models = xmlrpc.client.ServerProxy(f"{url}/xmlrpc/2/object")
courses = models.execute_kw(
    db, uid, password,
    "training.course", "search_read",
    [{"state", "=", "confirmed"}],
    {"fields": ["name", "start_date"], "limit": 20},
)
```

Common Mistakes

- Using the admin account for integrations.
- Forgetting that record rules affect API results.
- Fetching all fields and all records by default.
- Logging passwords or full credential URLs.

Lab / Practice: Create a script that authenticates over XML-RPC and reads the first ten confirmed courses with only three fields.

W17 External API Authentication

Learning Objectives

- Select an authentication pattern for external integrations.
- Store secrets outside source code.
- Validate callbacks and inbound requests.

Explanation

External API Authentication belongs to the client-side layer where user intent, browser state, and Odoo services meet. External APIs add another trust boundary around Odoo, so authentication design is part of the feature, not deployment cleanup. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Choose API keys, OAuth2, signed webhooks, or basic authentication based on the provider and risk of the data being exchanged. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Store secrets in system parameters, environment variables, or a secret manager, and wrap outbound calls in a service class or model method. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Test expired credentials, revoked tokens, replayed webhooks, and provider downtime so the integration fails safely. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, external api authentication decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Production API authentication should be observable: operators need enough logs to diagnose failures without exposing secrets. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Do not commit API credentials.
- Use least-privilege provider scopes.
- Validate webhook signatures when available.
- Handle token expiry deliberately.
- Log identifiers, not secrets.

Signed outbound request helper

The token is read from configuration at runtime and never appears in the source file.

```
import requests
from odoo import models

class TrainingExternalClient(models.AbstractModel):
    _name = "training.external.client"
    _description = "Training External API Client"

    def _headers(self):
        token = self.env["ir.config_parameter"].sudo().get_param("training.api_token")
        return {"Authorization": f"Bearer {token}", "Accept": "application/json"}

    def fetch_certificates(self):
        response = requests.get(
            "https://api.example.com/certificates",
            headers=self._headers(),
            timeout=10,
        )
        response.raise_for_status()
        return response.json()
```

Common Mistakes

- Committing secrets to the repository.
- Using broad OAuth scopes because they are easier during testing.
- Accepting inbound webhooks without signature validation.
- Logging Authorization headers during debugging.

Lab / Practice: Design an authentication plan for one outbound API and one inbound webhook, including where each secret is stored.

W18 Fetch and Export Workflows with XML-RPC

Learning Objectives

- Export selected records over XML-RPC.
- Paginate large result sets.
- Make fetch jobs restartable.

Explanation

Fetch and Export Workflows with XML-RPC belongs to the client-side layer where user intent, browser state, and Odoo services meet. Export integrations fail in production when they assume the dataset is small and the network is perfect. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Define the export domain, fields, ordering, and checkpoint strategy before writing the loop. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Use search with offset or keyset-style domains, read only required fields, and persist the last successful checkpoint outside the transient process. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Verify duplicates, skipped records, and interrupted runs by killing the process midway and restarting it against the same database. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, fetch and export workflows with xml-rpc decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A robust export job values correctness over raw speed because downstream systems often cannot detect missing records. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Export only required fields.
- Paginate results instead of fetching everything.
- Use deterministic ordering.
- Persist checkpoints for restartability.
- Design for duplicates or idempotent downstream writes.

Paged XML-RPC export

The loop exports in deterministic batches instead of loading the complete table at once.

```
offset = 0
batch_size = 100
while True:
    rows = models.execute_kw(
        db, uid, password,
        "training.enrollment", "search_read",
        [{"write_date", ">=", "2026-01-01 00:00:00"}],
        {
            "fields": ["id", "student_id", "course_id", "state", "write_date"],
            "order": "id",
            "offset": offset,
            "limit": batch_size,
        },
    )
    if not rows:
        break
    export_rows(rows)
    offset += batch_size
```

Common Mistakes

- Exporting all records into memory.
- Using no order while paginating.
- Not handling retries or partial downstream failure.
- Exporting display names when stable ids are required.

Lab / Practice: Write a paginated export for enrollments and simulate a restart after two batches.

W19 CRUD Operations over XML-RPC

Learning Objectives

- Create, read, update, and delete records through XML-RPC.
- Pass relational field commands correctly.
- Respect constraints and security during API writes.

Explanation

CRUD Operations over XML-RPC belongs to the client-side layer where user intent, browser state, and Odoo services meet. XML-RPC CRUD uses the same ORM methods that server code uses, so constraints, onchange gaps, access rights, and record rules all matter. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. Design write payloads from the model contract, not from the current form layout, because UI-only onchange logic may not run during API calls. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Use create, write, unlink, search_read, and relational command lists for one2many or many2many fields when needed. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Verify bad payloads, duplicate submissions, and permission failures because integrations often retry after ambiguous network errors. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, crud operations over xml-rpc decisions affect performance, offline behavior, cache invalidation, and upgrade risk. Production CRUD integrations should be idempotent where possible and should log remote ids, local ids, and operation outcomes. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- create returns the new record id.
- write returns True when the update succeeds.
- unlink deletes records if access and rules allow it.
- Relational commands use tuple-like command lists.
- Onchange behavior is not a substitute for server validation.

Create and update enrollment

The API creates a draft enrollment and then updates its state through normal ORM security and constraints.

```
enrollment_id = models.execute_kw(
    db, uid, password,
    "training.enrollment", "create",
    [{
        "student_id": student_id,
        "course_id": course_id,
        "state": "draft",
    }],
)

models.execute_kw(
    db, uid, password,
    "training.enrollment", "write",
    [[enrollment_id], {"state": "confirmed"}],
)
```

Common Mistakes

- Expecting form onchange methods to run during create.

- Sending display names instead of database ids for many2one fields.
- Deleting records with unlink when archiving would preserve audit history.
- Retrying creates without an external idempotency key.

Lab / Practice: Create a course and enrollment over XML-RPC, update one field, and attempt an invalid write to confirm constraints are enforced.

W20 Postman Authentication and CRUD Testing

Learning Objectives

- Use Postman to document and test API calls.
- Authenticate against Odoo endpoints deliberately.
- Build repeatable CRUD request collections.

Explanation

Postman Authentication and CRUD Testing belongs to the client-side layer where user intent, browser state, and Odoo services meet. Postman is useful for training and integration discovery because it makes request shape, authentication, and responses visible. Senior developers treat JavaScript changes as product behavior, not decoration, because a small patch can alter every user session that loads the bundle.

The design starts with ownership of state and boundaries. For Odoo JSON-RPC or controller endpoints, define environment variables for base URL, database, credentials, token, and record ids. Keep components small, pass data through props where possible, and call services at clear workflow edges instead of scattering RPC calls through templates.

Implementation should follow Odoo's asset, registry, service, and template patterns. Create a collection that authenticates, stores returned values in variables, and then performs create, read, update, and delete or archive requests. Favor explicit imports, named templates, stable registry keys, and patches that are narrow enough to survive upstream changes.

Verification requires more than refreshing the page once. Verify negative cases such as missing auth, wrong database, invalid ids, and insufficient rights because a collection that only proves success is incomplete. Test with production-like data, translated labels, slow network behavior, and users with realistic access rights so browser success does not hide server-side failures.

In production, postman authentication and crud testing decisions affect performance, offline behavior, cache invalidation, and upgrade risk. A good Postman collection becomes living integration documentation when examples, tests, and environment variables are kept clean. Keep examples traceable to business needs, document assumptions near extension points, and prefer framework services over ad hoc browser globals.

Key Points

- Use environments for base URL and credentials.
- Store ids from responses for later requests.
- Add negative authorization tests.
- Do not export real secrets in shared collections.
- Document request purpose in the collection.

JSON-RPC body for search_read

The request uses Postman environment variables so the same collection can run against staging or local databases.

```
{
  "jsonrpc": "2.0",
  "method": "call",
  "params": {
    "service": "object",
    "method": "execute_kw",
    "args": [
      "{{db}}",
      {{uid}},
      "{{password}}",
      "training.course",
      "search_read",
      [[{"state", "=", "confirmed"}]],
      {"fields": ["name", "start_date"], "limit": 5}
    ]
  },
  "id": 1
}
```

Common Mistakes

- Exporting collections with real passwords.
- Hard-coding record ids that differ by database.
- Testing only successful CRUD operations.
- Confusing web session authentication with integration authentication.

Lab / Practice: Build a Postman collection with login, search_read, create, write, and negative-access requests using environment variables only.

Part H Infrastructure, Docker, VPS & Performance

This part contains 20 detailed lessons. Work in order. Complete each lab before moving forward when the lab unlocks the next dependency.

Presentation: Companion deck: see [training_materials/presentations/](#) for the Infrastructure, Docker, VPS & Performance slides.

101 Containers vs Virtual Machines

Learning Objectives

- Explain the difference between OS virtualization and process isolation.
- Choose an appropriate runtime for Odoo development and deployment.
- Identify what containers do not solve.

Explanation

Containers vs Virtual Machines is infrastructure work that directly affects reliability, data safety, and developer speed. Containers package a process with its filesystem and dependencies, while virtual machines package an entire guest operating system. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Use containers to make Odoo services repeatable and portable, but still design the host, network, storage, and backup strategy deliberately. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Run Odoo, PostgreSQL, Nginx, and workers as separate services where appropriate and keep persistent data outside disposable container layers. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify isolation assumptions by restarting containers, removing images, and confirming that database and filestore data remain intact. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, containers vs virtual machines choices become long-lived operational contracts. Containers reduce drift, but they do not replace monitoring, patching, secrets management, or restore testing. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Containers share the host kernel.
- VMs provide stronger OS-level separation.
- Container filesystems are disposable by design.
- Persistent data belongs in volumes or managed storage.
- Containers improve repeatability, not application design.

Mental model comparison

The model highlights why containers start quickly but still depend on host and storage decisions.

```
# Container mindset
app process + image filesystem + mounted volumes + network namespace

# VM mindset
guest OS + kernel + virtual disk + virtual network + app processes
```

Common Mistakes

- Storing database data inside a disposable container layer.
- Assuming containers are automatically secure.
- Using one giant container for every service.
- Forgetting host-level patching and monitoring.

Lab / Practice: Draw an architecture for local Odoo using containers and explain which files survive when the Odoo container is deleted.

102 Installing Docker Compose for Odoo Workloads

Learning Objectives

- Install Docker Engine and Compose plugin.
- Verify versions and permissions.
- Prepare a host for repeatable compose projects.

Explanation

Installing Docker Compose for Odoo Workloads is infrastructure work that directly affects reliability, data safety, and developer speed. Docker Compose is the most common entry point for running Odoo, PostgreSQL, and supporting services together on a single host. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Decide whether the host is for development or production because package sources, user permissions, filesystem paths, and firewall rules differ. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Install Docker from official packages, enable the service, add only trusted operators to the docker group, and verify compose is available as docker compose. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Run hello-world, inspect docker info, and create a minimal compose project to prove the daemon, plugin, network, and volume drivers work. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, installing docker compose for odoo workloads choices become long-lived operational contracts. A clean Docker installation prevents confusing deployment bugs where the application is blamed for host-level package or permission problems. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Use the Compose plugin command: docker compose.
- Membership in the docker group is root-equivalent.
- Pin host setup in documentation or automation.
- Verify daemon and compose versions.
- Keep production hosts minimal.

Ubuntu installation outline

The commands install Docker Engine and verify that the Compose plugin is available.

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
docker compose version
```

Common Mistakes

- Installing old docker-compose v1 and mixing command syntax.
- Adding every user to the docker group casually.
- Skipping a reboot or new login after group changes.
- Debugging compose files before verifying the daemon works.

Lab / Practice: Install Docker and Compose on a disposable VM, run hello-world, and document the exact version output.

103 Docker CLI, Images, Containers, and Docker Hub

Learning Objectives

- Use core Docker CLI commands confidently.
- Explain images, containers, tags, and registries.
- Inspect running Odoo services.

Explanation

Docker CLI, Images, Containers, and Docker Hub is infrastructure work that directly affects reliability, data safety, and developer speed. The Docker CLI is the operational vocabulary for seeing what is running, what image it came from, and where logs are produced. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Use tags deliberately: latest is convenient for experiments but too ambiguous for production upgrade control. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Pull images, run containers, inspect metadata, read logs, execute shells, and remove unused resources with clear intent. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify the image digest, environment, mounts, and network settings when a container behaves differently from another host. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, docker cli, images, containers, and docker hub choices become long-lived operational contracts. Production operators should be able to answer which image tag is running and how to recreate it before they make changes. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Images are immutable templates.
- Containers are running or stopped instances.
- Registries store and distribute images.
- Tags are names, not guarantees of immutability.
- docker inspect is essential for debugging.

Useful Docker inspection commands

These commands show running services, images, recent logs, interactive access, mounts, and disk usage.

```
docker ps
docker images
docker logs --tail=100 odoo-web
docker exec -it odoo-web bash
docker inspect odoo-web --format '{{json .Mounts}}'
docker system df
```

Common Mistakes

- Using latest tags in production.
- Removing volumes while trying to clean images.
- Ignoring image provenance.
- Debugging from assumptions instead of docker inspect output.

Lab / Practice: Run an Odoo container, inspect its environment and mounts, then explain the difference between stopping and removing it.

I04 Dockerfile for Odoo

Learning Objectives

- Build a custom Odoo image.
- Install Python and system dependencies predictably.
- Keep addons and requirements layered for cache efficiency.

Explanation

Dockerfile for Odoo is infrastructure work that directly affects reliability, data safety, and developer speed. A custom Dockerfile turns an Odoo deployment from a mutable server into a versioned artifact. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Decide which dependencies belong in the image and which configuration belongs at runtime through environment or mounted files. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Start from a trusted Odoo base image, copy requirements before addons for cache efficiency, install packages as root, and run Odoo as the intended user. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Build the image from a clean context and run it without bind mounts to prove the image contains everything needed. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, dockerfile for odoo choices become long-lived operational contracts. A production image should be reproducible, small enough to deploy quickly, and explicit about every dependency that custom code requires. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Images should include code and dependencies, not secrets.
- Layer order affects build cache.
- Use requirements.txt for Python dependencies.
- Keep runtime config outside the image.
- Build and test images in CI.

Custom Odoo image

The image installs Python dependencies first and then copies custom addons with the correct ownership.

```
FROM odoo:18.0

USER root
COPY requirements.txt /tmp/requirements.txt
RUN pip3 install --no-cache-dir -r /tmp/requirements.txt

USER odoo
COPY --chown=odoo:odoo ./addons /mnt/extra-addons
ENV OD00_RC=/etc/odoo/odoo.conf
```

Common Mistakes

- Copying secrets into the image.
- Installing dependencies interactively inside a running container.
- Invalidating the build cache by copying the whole repository too early.
- Running the final container as root without need.

Lab / Practice: Create a Dockerfile for custom addons, build it locally, and run Odoo from the image without mounting addon code.

I05 Volumes for PostgreSQL, Filestore, and Addons

Learning Objectives

- Map persistent Odoo data to volumes.
- Separate database, filestore, and code storage.
- Avoid accidental data loss during container replacement.

Explanation

Volumes for PostgreSQL, Filestore, and Addons is infrastructure work that directly affects reliability, data safety, and developer speed. Odoo persistence is split between PostgreSQL rows and the filestore, and both must be backed up together. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Design storage paths so database data, Odoo filestore, configuration, logs, and custom addons have clear ownership and backup policy. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Use named volumes or host bind mounts intentionally, and never rely on the writable container layer for anything important. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify by deleting and recreating the application container while confirming that database records and attachments still exist. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, volumes for postgresql, filestore, and addons choices become long-lived operational contracts. Production restore depends on matching database dumps with the correct filestore, so volume layout is a backup design decision. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- PostgreSQL data and Odoo filestore are both persistent.
- Named volumes are managed by Docker.
- Bind mounts expose host paths explicitly.
- Container layers are disposable.
- Backups must include database and filestore together.

Compose volumes for Odoo data

The database and filestore use persistent volumes while custom addons are mounted read-only.

```
services:
  db:
    image: postgres:16
    volumes:
      - pgdata:/var/lib/postgresql/data
  odoo:
    image: odoo:18.0
    volumes:
      - odoo-data:/var/lib/odoo
      - ./addons:/mnt/extra-addons:ro

volumes:
  pgdata:
  odoo-data:
```

Common Mistakes

- Backing up only the database and forgetting attachments.
- Mounting writable addon code in production.
- Using anonymous volumes without knowing their names.

- Deleting volumes during image cleanup.

Lab / Practice: Attach a PDF to an Odoo record, recreate the Odoo container, and confirm the attachment still opens.

I06 Docker Networking for Odoo

Learning Objectives

- Explain service discovery in Docker Compose.
- Connect Odoo to PostgreSQL by service name.
- Expose only required ports to the host.

Explanation

Docker Networking for Odoo is infrastructure work that directly affects reliability, data safety, and developer speed. Compose networking gives services DNS names and private connectivity without exposing every port to the outside world. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Decide which services need host access, which only need internal network access, and which should be isolated from public traffic. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Use service names such as db as hostnames, publish only Nginx or Odoo ports needed by the environment, and avoid hard-coded container IP addresses. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify connectivity with `docker exec`, `getent hosts`, and application logs rather than guessing at network failures. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, docker networking for odoo choices become long-lived operational contracts. A clean network design reduces attack surface and makes local, staging, and production compose files easier to compare. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Compose services share a project network by default.
- Service names resolve through Docker DNS.
- ports publishes to the host; expose documents internal ports.
- Do not depend on container IP addresses.
- Limit public exposure.

Internal database network

Odoo connects to PostgreSQL using the db service name without publishing PostgreSQL to the host.

```
services:
  odoo:
    image: odoo:18.0
    environment:
      HOST: db
      USER: odoo
      PASSWORD: odoo
    ports:
      - "8069:8069"
    depends_on:
      - db
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: postgres
      POSTGRES_USER: odoo
      POSTGRES_PASSWORD: odoo
```

Common Mistakes

- Publishing PostgreSQL publicly without a reason.

- Using container IPs that change after recreation.
- Assuming depends_on means the database is ready for connections.
- Mixing unrelated stacks on the same exposed ports.

Lab / Practice: Create a compose network where only Odoo is published, then prove PostgreSQL is reachable from Odoo but not from the host port list.

107 Docker Compose Fundamentals

Learning Objectives

- Read and write compose service definitions.
- Use environment, volumes, ports, and depends_on.
- Operate a compose project safely.

Explanation

Docker Compose Fundamentals is infrastructure work that directly affects reliability, data safety, and developer speed. Compose turns a multi-command deployment into a declarative project that can be started, stopped, inspected, and versioned. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Design compose files so the default command is safe for the target environment and overrides are explicit. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Define services, images or build contexts, environment variables, volumes, networks, healthchecks, and restart policies where appropriate. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify with config rendering, logs, service status, and restart tests because valid YAML can still describe a poor deployment. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, docker compose fundamentals choices become long-lived operational contracts. Production compose usage should be predictable enough that an engineer can recreate the stack from the repository and documented secrets. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- docker compose config validates and expands configuration.
- Services are the main units.
- Environment can come from .env and env_file.
- Restart policy controls daemon behavior after failure.
- Compose project names affect resource names.

Common compose operations

These commands validate, start, inspect, restart, and stop a compose project.

```
docker compose config
docker compose up -d
docker compose logs -f --tail=100 odoo
docker compose ps
docker compose restart odoo
docker compose down
```

Common Mistakes

- Editing running containers instead of compose files.
- Skipping docker compose config before deployment.
- Confusing down with volume removal.
- Letting project names change between runs unexpectedly.

Lab / Practice: Write a minimal compose file with Odoo and PostgreSQL, run docker compose config, and explain every generated service field.

108 Multi-Container Odoo and PostgreSQL

Learning Objectives

- Run Odoo and PostgreSQL as separate services.
- Configure database credentials and health checks.
- Understand startup ordering and readiness.

Explanation

Multi-Container Odoo and PostgreSQL is infrastructure work that directly affects reliability, data safety, and developer speed. Separating Odoo and PostgreSQL mirrors the real dependency between an application server and a database server. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Choose explicit database credentials, persistent storage, healthchecks, and restart behavior before adding custom addons or proxy layers. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Use environment variables or config files for `db_host`, `db_user`, and `db_password`, and define a PostgreSQL healthcheck because `depends_on` is not readiness. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify by restarting only the database, then only Odoo, and checking how each service logs reconnection or startup behavior. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, multi-container odoo and postgresql choices become long-lived operational contracts. A stable two-container baseline is the foundation for adding workers, Nginx, backups, and monitoring later. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Odoo and PostgreSQL should be separate services.
- `depends_on` controls order, not readiness.
- Healthchecks make service state visible.
- Use persistent PostgreSQL storage.
- Restart behavior should be intentional.

Odoo and PostgreSQL compose stack

The stack separates services and exposes database readiness through a healthcheck.

```
services:
  db:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: postgres
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 10s
      timeout: 5s
      retries: 5

  odoo:
    image: odoo:18.0
    restart: unless-stopped
    depends_on:
      - db
    environment:
      HOST: db
      USER: ${POSTGRES_USER}
      PASSWORD: ${POSTGRES_PASSWORD}
```

Common Mistakes

- Putting PostgreSQL inside the Odoo container.
- Using default passwords in shared environments.
- Assuming startup order means readiness.
- Restarting the whole stack for a single-service issue.

Lab / Practice: Run the two-container stack, restart PostgreSQL, and capture the relevant Odoo logs during reconnect.

109 .env Files and Runtime Configuration

Learning Objectives

- Use .env files for non-secret and secret runtime values carefully.
- Understand Compose variable substitution.
- Separate committed examples from real credentials.

Explanation

.env Files and Runtime Configuration is infrastructure work that directly affects reliability, data safety, and developer speed. .env files make compose projects easier to configure, but they also become a common source of leaked secrets. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Decide which variables are safe defaults, which are environment-specific, and which should come from a secret manager in production. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Commit .env.example, ignore real .env files, and use explicit variable names for database credentials, domain names, image tags, and worker settings. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify the rendered configuration with docker compose config and check repository status before committing. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, .env files and runtime configuration choices become long-lived operational contracts. A disciplined configuration approach lets staging and production differ without maintaining separate hand-edited compose files. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- .env supports Compose variable substitution.
- Commit examples, not real secrets.
- docker compose config shows resolved values.
- Use clear variable names.
- Production secrets may need a stronger mechanism.

.env.example

The example documents required settings without pretending that the sample password is production-ready.

```
ODOO_IMAGE=odoo:18.0
POSTGRES_USER=odoo
POSTGRES_PASSWORD=replace-in-real-env
ODOO_DB_HOST=db
ODOO_WORKERS=4
ODOO_LIMIT_MEMORY_HARD=2684354560
DOMAIN=odoo.example.com
```

Common Mistakes

- Committing real .env files.
- Assuming .env variables are automatically available inside containers.
- Using vague names such as PASSWORD for multiple services.
- Skipping rendered config review before deployment.

Lab / Practice: Create .env.example and .gitignore entries for a compose project, then render the final compose config with local values.

110 Upgrading Containers Safely

Learning Objectives

- Plan image and module upgrades.
- Back up database and filestore before changes.
- Roll forward or roll back from a known state.

Explanation

Upgrading Containers Safely is infrastructure work that directly affects reliability, data safety, and developer speed. Container upgrades are easy to start and expensive to recover from if database migrations and filestore compatibility are ignored. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Separate base image upgrades, custom addon changes, and database migrations into a planned sequence with a tested backup. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Pull or build the new image, stop traffic, take backups, run Odoo update commands on staging first, and keep the previous image tag available. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify module updates, logs, scheduled jobs, reports, and core user journeys before reopening traffic. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, upgrading containers safely choices become long-lived operational contracts. Safe upgrades are boring because every step has already been rehearsed on a restored copy of production. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Pin image tags before upgrading.
- Back up database and filestore together.
- Test migrations on a restored copy.
- Keep previous image available for rollback.
- Review logs before declaring success.

Upgrade command sequence

The sequence backs up before starting the new image and runs an explicit module update.

```
docker compose pull odoo
docker compose stop odoo
./scripts/backup_odoo.sh
docker compose up -d odoo
docker compose exec odoo odoo -d prod -u training --stop-after-init
docker compose logs --tail=200 odoo
```

Common Mistakes

- Using latest tags and not knowing what changed.
- Upgrading production before staging.
- Backing up the database without the filestore.
- Skipping post-upgrade functional checks.

Lab / Practice: Write a one-page upgrade runbook for an Odoo container stack, including rollback criteria and validation checks.

111 CI/CD with GitHub Actions Basics

Learning Objectives

- Create a basic CI workflow for Odoo modules.
- Run linting and tests on pull requests.
- Build container images from commits.

Explanation

CI/CD with GitHub Actions Basics is infrastructure work that directly affects reliability, data safety, and developer speed. CI/CD turns local confidence into repeatable checks that run before code reaches shared environments. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Start with fast checks that catch common failures: syntax, manifest validity, unit tests, and image build problems. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Use GitHub Actions workflows with pinned action versions, service containers for PostgreSQL where needed, and artifacts for logs or coverage. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify that a failing test fails the workflow and that secrets are only available on trusted events. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, ci/cd with github actions basics choices become long-lived operational contracts. Production deployment should build on CI results, but the first goal is preventing broken custom addons from being merged unnoticed. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Run checks on pull_request and push.
- Use PostgreSQL services for integration tests.
- Cache dependencies cautiously.
- Protect secrets from untrusted forks.
- Keep workflows readable and incremental.

Basic GitHub Actions workflow

The workflow starts with lightweight checks that catch syntax and manifest problems.

```
name: odoo-ci

on:
  pull_request:
  push:
    branches: [main]

jobs:
  python-checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: python -m compileall custom_addons
      - run: python scripts/check_manifests.py custom_addons
```

Common Mistakes

- Putting deployment secrets in pull_request workflows from forks.
- Running huge end-to-end jobs before cheap syntax checks.

- Ignoring workflow failure logs.
- Letting CI use a different dependency set than production.

Lab / Practice: Create a CI workflow that compiles custom addon Python files and intentionally break one file to prove the check fails.

I12 VPS Provisioning and Production Directory Layout

Learning Objectives

- Prepare a VPS for Odoo deployment.
- Design a clear production directory structure.
- Separate app code, config, backups, logs, and secrets.

Explanation

VPS Provisioning and Production Directory Layout is infrastructure work that directly affects reliability, data safety, and developer speed. A VPS is simple enough for small Odoo deployments but still needs disciplined layout and host preparation. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Provision with a supported OS, enough memory and disk IOPS, a non-root deploy user, firewall rules, time sync, and monitoring access. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Create predictable directories such as `/opt/odoo/app`, `/opt/odoo/config`, `/opt/odoo/backups`, `/var/log/odoo`, and `/srv/odoo/filestore` depending on the deployment style. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify permissions by running the service as the target user and confirming that backups can write only where intended. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, vps provisioning and production directory layout choices become long-lived operational contracts. A good layout makes incidents calmer because engineers can find the compose file, config, backups, and logs without searching the whole server. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Use a non-root deploy or odoo user.
- Keep configuration and secrets separate from code.
- Define backup and log paths explicitly.
- Document the directory layout.
- Provision disk and memory for growth.

Production layout example

The layout keeps operational assets separate and easy to back up or audit.

```
/opt/odoo/  
  app/           # compose file or deployment repository  
  config/        # odoo.conf, nginx snippets, env files  
  backups/       # database and filestore backups  
  scripts/       # backup, restore, health checks  
/var/log/odoo/   # application logs  
/srv/odoo/filestore/ # optional external filestore path
```

Common Mistakes

- Running everything from a developer's home directory.
- Mixing backups into the application repository.
- Leaving important files owned by root after manual commands.
- Not documenting which path is authoritative.

Lab / Practice: Provision a small test VPS or VM and create the directory layout with ownership and permissions suitable for an odoo deploy user.

I13 SSH Hardening

Learning Objectives

- Reduce common SSH attack surface.
- Use key-based authentication.
- Protect administrative access without locking out operators.

Explanation

SSH Hardening is infrastructure work that directly affects reliability, data safety, and developer speed. SSH is usually the front door to a VPS, so weak SSH posture undermines every application hardening effort. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Design access around named users, public keys, sudo policy, and an emergency recovery path from the VPS provider console. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Disable password login, disable root login, restrict users where practical, install fail2ban or equivalent protection, and keep authorized keys reviewed. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify a second session can connect before closing the first and test sudo access for the deploy process. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, ssh hardening choices become long-lived operational contracts. Good SSH hardening balances security and operability: locked-out administrators cause downtime just as surely as attackers do. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Use public key authentication.
- Disable direct root login.
- Keep an active session while testing changes.
- Limit sudo and user access.
- Maintain an emergency recovery path.

sshd hardening fragment

The fragment disables password and root login while allowing a named deploy user.

```
# /etc/ssh/sshd_config.d/10-hardening.conf
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowUsers deploy
ClientAliveInterval 300
ClientAliveCountMax 2
```

Common Mistakes

- Restarting SSH after changes without an open backup session.
- Sharing one private key between operators.
- Disabling root login before creating a sudo-capable user.
- Forgetting provider console recovery.

Lab / Practice: Harden SSH on a disposable VM, prove key login works in a second session, and document the rollback path.

114 Nginx Reverse Proxy for Odoo

Learning Objectives

- Proxy public HTTP traffic to Odoo.
- Set headers required for correct URL generation.
- Configure timeouts for long Odoo requests.

Explanation

Nginx Reverse Proxy for Odoo is infrastructure work that directly affects reliability, data safety, and developer speed. Nginx usually terminates public HTTP traffic and forwards application requests to Odoo on an internal port. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Decide where TLS terminates, which hostnames are valid, and whether Odoo runs in `proxy_mode` so it trusts forwarded headers correctly. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Configure `proxy_pass`, `Host`, `X-Forwarded` headers, body size, buffering, and timeouts appropriate for Odoo imports and reports. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify real client IPs, generated URLs, file uploads, long report renders, and backend logs after enabling the proxy. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, nginx reverse proxy for odoo choices become long-lived operational contracts. A correct reverse proxy prevents subtle bugs around redirects, mixed content, incorrect base URLs, and client IP audit logs. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Enable `proxy_mode` in Odoo when behind a proxy.
- Forward `Host` and `X-Forwarded` headers.
- Set body size for imports and attachments.
- Tune timeouts for reports and upgrades.
- Keep upstream ports private when possible.

Nginx Odoo proxy server

The proxy forwards standard headers and allows long Odoo requests to complete.

```
server {
    listen 80;
    server_name odoo.example.com;

    client_max_body_size 100m;

    location / {
        proxy_pass http://127.0.0.1:8069;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 720s;
        proxy_connect_timeout 720s;
    }
}
```

Common Mistakes

- Forgetting `proxy_mode` in Odoo configuration.
- Leaving Odoo publicly exposed when Nginx should be the entry point.

- Using low timeouts that break large reports or imports.
- Not forwarding X-Forwarded-Proto, causing bad redirects.

Lab / Practice: Put Nginx in front of Odoo, enable `proxy_mode`, and confirm generated `web.base.url` uses the public hostname.

115 Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt

Learning Objectives

- Route public HTTP and HTTPS traffic to Odoo.
- Handle longpolling or websocket-style notification endpoints.
- Install TLS certificates with Certbot.

Explanation

Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt is infrastructure work that directly affects reliability, data safety, and developer speed. Users should reach Odoo on standard web ports while the application listens internally on ports such as 8069 and longpolling on 8072. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Plan firewall rules, Nginx server blocks, TLS termination, renewal automation, and the route for bus or longpolling traffic. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Use Certbot for Let's Encrypt certificates, redirect HTTP to HTTPS, and add a separate location for longpolling when the deployment uses a dedicated port. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify certificate renewal, live chat or bus notifications, redirect behavior, and firewall exposure from outside the server. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, ports 80 and 443 to 8069, longpolling 8072, and let's encrypt choices become long-lived operational contracts. A deployment without reliable TLS and notification routing will look healthy until users report broken chat, Discuss, or real-time updates. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Public users should use 80 and 443.
- Odoo backend often listens on 8069 internally.
- Longpolling commonly uses 8072 in worker deployments.
- Certbot can automate Let's Encrypt certificates.
- Test certificate renewal before relying on it.

Nginx HTTPS and longpolling locations

The server routes normal requests to 8069 and longpolling requests to 8072 under HTTPS.

```
server {
    listen 443 ssl http2;
    server_name odoo.example.com;

    ssl_certificate /etc/letsencrypt/live/odoo.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/odoo.example.com/privkey.pem;

    location /longpolling/ {
        proxy_pass http://127.0.0.1:8072;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
    }

    location / {
        proxy_pass http://127.0.0.1:8069;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

Common Mistakes

- Opening 8069 and 8072 publicly when only Nginx should be exposed.
- Forgetting automatic certificate renewal.
- Routing longpolling to the wrong upstream.
- Not testing real-time notifications after enabling workers.

Lab / Practice: Configure HTTPS for a test hostname, add a longpolling location, and run `certbot renew --dry-run`.

I16 systemd odoo.service

Learning Objectives

- Run Odoo as a systemd service.
- Define restart behavior and service user.
- Use journalctl for logs and diagnosis.

Explanation

systemd odoo.service is infrastructure work that directly affects reliability, data safety, and developer speed. Some deployments run Odoo directly on a host instead of containers, and systemd is the standard process supervisor for that model. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Decide the service user, config file path, Python environment, addons path, restart policy, and log destination before writing the unit. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Create a unit file with ExecStart, User, WorkingDirectory, Restart, and dependency on PostgreSQL or network availability as appropriate. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify daemon reload, enable, start, restart, failure restart, and journal logs so operations are repeatable. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, systemd odoo.service choices become long-lived operational contracts. A clear systemd unit makes a non-container deployment manageable and avoids fragile shell sessions or manual startup commands. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Run Odoo under a dedicated user.
- Keep ExecStart explicit.
- Use systemctl enable for boot startup.
- Use journalctl -u for logs.
- Restart policy should match operational expectations.

Odoo systemd unit

The unit runs Odoo from a virtual environment with a dedicated user and restarts after failures.

```
[Unit]
Description=Odoo Application Server
After=network.target postgresql.service

[Service]
Type=simple
User=odoo
Group=odoo
WorkingDirectory=/opt/odoo/server
ExecStart=/opt/odoo/venv/bin/python3 /opt/odoo/server/odoo-bin -c /etc/odoo/odoo.conf
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Common Mistakes

- Running Odoo manually in screen or tmux as the production supervisor.
- Starting the service as root without need.
- Forgetting systemctl daemon-reload after editing the unit.

- Hiding logs in an unmanaged file path.

Lab / Practice: Create a systemd unit on a test VM, start it, force a failure, and confirm restart behavior in journalctl.

117 Backup Cron for Database and Filestore

Learning Objectives

- Back up PostgreSQL and Odoo filestore together.
- Schedule backups with cron.
- Test restore, retention, and failure alerts.

Explanation

Backup Cron for Database and Filestore is infrastructure work that directly affects reliability, data safety, and developer speed. Backups are only useful when they can be restored, and Odoo requires both database and filestore for a complete restore. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Design backup frequency, retention, encryption, offsite storage, and restore objectives before writing the cron line. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Use `pg_dump` or `pg_dumpall` as appropriate, archive the filestore matching the database, name backups with timestamps, and prune old files safely. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify by restoring to a separate database and opening attachments, not by checking that a backup file exists. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, backup cron for database and filestore choices become long-lived operational contracts. Production backup policy should include alerting because a silent backup failure is discovered at the worst possible time. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Database and filestore must be backed up as a pair.
- Timestamp backup names.
- Store copies off the application host.
- Test restore regularly.
- Alert on backup failures.

Backup script fragment

The script writes a timestamped database dump and matching filestore archive, then prunes old local backups.

```
#!/usr/bin/env bash
set -euo pipefail

DB=prod
STAMP=$(date +%Y%m%d-%H%M%S)
DEST=/opt/odoo/backups/$STAMP
mkdir -p "$DEST"

pg_dump -Fc "$DB" > "$DEST/$DB.dump"
tar -C /var/lib/odoo/filestore -czf "$DEST/filestore.tgz" "$DB"
find /opt/odoo/backups -mindepth 1 -maxdepth 1 -type d -mtime +14 -exec rm -rf {} +
```

Common Mistakes

- Backing up only PostgreSQL.
- Never testing restore.
- Keeping all backups on the same disk as production.
- Writing cron jobs that fail silently.

Lab / Practice: Create a backup script and cron entry, then restore the output to a separate test database and open an attachment.

118 Odoo Workers Architecture and Worker Calculation

Learning Objectives

- Explain threaded versus multi-worker Odoo modes.
- Calculate an initial worker count.
- Reserve capacity for cron and longpolling.

Explanation

Odoo Workers Architecture and Worker Calculation is infrastructure work that directly affects reliability, data safety, and developer speed. Odoo workers let the application handle concurrent HTTP requests with multiple processes instead of one threaded development-style server. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Base the initial worker count on CPU cores, memory, traffic profile, cron load, and whether the host also runs PostgreSQL. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Set workers, max_cron_threads, proxy_mode, and gevent or longpolling configuration according to the Odoo version and deployment model. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify under load with realistic requests, not only by checking that processes exist, because workers can be CPU-bound, memory-bound, or database-bound. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, odoo workers architecture and worker calculation choices become long-lived operational contracts. Worker sizing is an iterative capacity practice: start conservatively, measure, and adjust before users feel saturation. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- workers > 0 enables multi-process mode.
- A common starting formula is CPU cores * 2 + 1, adjusted for memory.
- Cron workers consume capacity.
- Longpolling or bus traffic may need separate handling.
- Measure before and after changes.

odoo.conf worker settings

The settings enable worker mode with conservative time and memory limits for a modest host.

```
[options]
workers = 5
max_cron_threads = 1
proxy_mode = True
limit_time_cpu = 120
limit_time_real = 240
limit_memory_soft = 2147483648
limit_memory_hard = 2684354560
```

Common Mistakes

- Using the CPU formula without checking memory.
- Forgetting cron capacity.
- Running many workers against too few PostgreSQL connections.
- Changing workers without load testing.

Lab / Practice: Calculate initial workers for a 4-core, 8 GB VPS running Odoo and PostgreSQL, then justify memory reserves.

119 Memory Limits, Time Limits, PostgreSQL Pooling, and max_connections

Learning Objectives

- Configure Odoo request memory and time guards.
- Relate Odoo workers to PostgreSQL connections.
- Plan pooling or max_connections changes safely.

Explanation

Memory Limits, Time Limits, PostgreSQL Pooling, and max_connections is infrastructure work that directly affects reliability, data safety, and developer speed. Odoo and PostgreSQL capacity problems often appear together because more workers can mean more database connections and more memory pressure. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Design limits from the host resources, request profile, scheduled jobs, and PostgreSQL memory settings instead of copying values from another deployment. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Set limit_memory_soft, limit_memory_hard, limit_time_cpu, limit_time_real, db_maxconn, and PostgreSQL max_connections with enough headroom for maintenance sessions. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify by monitoring worker recycling, slow requests, database connection counts, and out-of-memory events under realistic load. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, memory limits, time limits, postgresql pooling, and max_connections choices become long-lived operational contracts. Production tuning is a balancing act: too loose risks host instability, too tight kills legitimate reports and imports. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- Odoo workers consume memory independently.
- db_maxconn controls Odoo's database connection pool per process.
- PostgreSQL max_connections must cover all clients.
- Memory limits recycle runaway workers.
- Long reports may need time limit review.

Connection budget example

The calculation reminds operators to budget for all database clients, not only HTTP workers.

```
# Example budget
workers = 5
max_cron_threads = 1
db_maxconn = 16

# Potential Odoo connections: (workers + cron + main process) * db_maxconn
# Add admin tools, monitoring, backup jobs, and migration sessions before
# setting PostgreSQL max_connections.
```

Common Mistakes

- Increasing workers without increasing database capacity.
- Setting max_connections high without enough PostgreSQL memory.
- Disabling time limits to hide slow code.
- Ignoring cron and migration connections.

Lab / Practice: Create a connection and memory budget for an 8-worker Odoo deployment and propose safe PostgreSQL `max_connections`.

120 Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana

Learning Objectives

- Identify slow SQL with pg_stat_statements.
- Explain when missing indexes hurt Odoo.
- Describe Redis sessions, load balancing, and monitoring basics.

Explanation

Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana is infrastructure work that directly affects reliability, data safety, and developer speed. At scale, Odoo performance is usually constrained by database queries, worker saturation, cache behavior, or lack of visibility. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Design observability before firefighting: know which metrics, logs, and query statistics reveal the difference between slow Python, slow SQL, and overloaded infrastructure. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Enable pg_stat_statements, inspect high-total-time queries, add indexes only after understanding domains, and consider Redis-backed sessions or sticky load balancing for multi-node deployments. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Verification must prove behavior, not only syntax. Verify every performance change with before-and-after measurements, because indexes can slow writes and load balancers can break session assumptions. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

In production, indexes, pg_stat_statements, redis sessions, load balancing, and prometheus grafana choices become long-lived operational contracts. Prometheus and Grafana provide the operational feedback loop: collect metrics, create dashboards, alert on symptoms, and use traces or logs for root cause. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

Key Points

- pg_stat_statements shows expensive query patterns.
- Indexes should match frequent domains and joins.
- Redis sessions can help multi-node session consistency.
- Load balancing requires shared filestore and session strategy.
- Prometheus and Grafana turn metrics into alerts and dashboards.

Inspect slow queries

The query identifies expensive SQL patterns; the example index matches a common state and date domain.

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

SELECT calls,
       total_exec_time,
       mean_exec_time,
       rows,
       query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;

CREATE INDEX IF NOT EXISTS training_course_state_start_idx
ON training_course (state, start_date);
```

Common Mistakes

- Adding indexes blindly for every slow query.

- Load balancing without shared filestore or session planning.
- Creating dashboards with no alert thresholds.
- Ignoring write overhead from extra indexes.

Lab / Practice: Enable `pg_stat_statements` on a staging database, identify one slow query pattern, and propose a measured index or code change.

Z Curriculum Closure

You now have the full written body of the Odoo Full-Stack Developer Program. Continue with the section presentation decks for lecture pacing, then execute a capstone under the rubric in Part A. Keep a personal lab journal: commands run, errors seen, and fixes applied—this journal becomes your operational runbook.

Document Control

Field	Value
Title	Odoo Full-Stack Developer Training Documentation
Edition	2026.07
Lesson count	160
Classification	Complete teaching documentation